

TokTickIT

CPE334 Lab 2 — Requester Ticketing Sprint

Student	Thawat Boonsuk
Student ID	67070501024
GitHub	Chessuker
Repository	https://github.com/Chessuker/TokTickIT
Branch	main at 9936b8c
Peer reviewer	Theehathat (67070501019, TeekhathatTT)
Tests on main	226 passing — 130 backend, 80 frontend, 16 end-to-end

1. Answer Part 1: Git Use with Engineering Workflow
2. Answer Part 2: Spec-Driven Development
3. Answer Part 3: Test-Driven Development and Traceability
4. Answer Part 4: AI Use with Reflection
5. Answer Part 5: Development Requester Select Screen
6. Answer Part 6: Working Ticket Screen — Create Mode
7. Answer Part 7: Working My Tickets Screen
8. Answer Part 8: Working Ticket Screen — View Mode and Attachments
9. Answer Part 9: Zen Green UI and Responsive Evidence

Answer Part 1:

Git Use with Engineering Workflow · 10 points

Answer Part 1 — Git Use with Engineering Workflow

Repository: <https://github.com/Chessuker/TokTickIT> · Final branch: `main` at `9936b8c`

1. Branch and merge history on `main`

Every issue was built on its own feature branch, merged into `lab2-staging` through a pull request, and reached `main` only through a second pull request from staging. No commit was pushed straight to `main`.

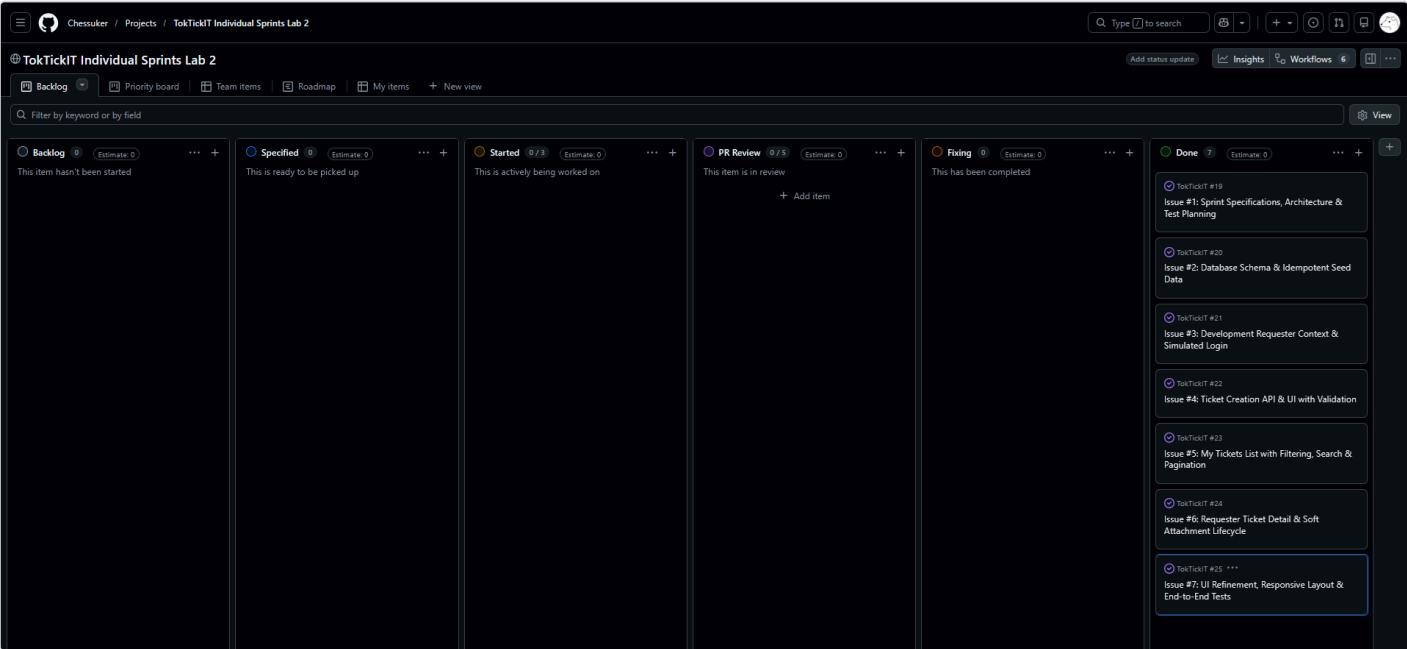
```
$ git log --graph --oneline --decorate -25
* 9936b8c (HEAD -> main, origin/main, origin/HEAD) Merge pull request #33 from Chessuker/lab2-staging
|\
| * 229659e (origin/lab2-staging) Merge pull request #32 from Chessuker/feature/7-ui-responsive-e2e
| |\
| | * dad25b8 (origin/feature/7-ui-responsive-e2e, feature/7-ui-responsive-e2e) feature 7: refine zen green UI, responsive styles, and
implement Playwright E2E tests
| | /
| * f097031 (lab2-staging) Merge pull request #31 from Chessuker/feature/6-ticket-detail-attachments
| |\
| | * 4db72b0 (origin/feature/6-ticket-detail-attachments, feature/6-ticket-detail-attachments) fix MIME check
| | * f7f2a07 feature 5: implement ticket detail view, attachment upload, download, soft removal, and ownership guards
| | /
| * accbb70 Merge pull request #30 from Chessuker/feature/5-my-tickets-list
| |\
| | * 91ed7cb (origin/feature/5-my-tickets-list, feature/5-my-tickets-list) feature 5: implement My Tickets listing with search, filtering,
pagination, and empty/no-results states
| | /
| * 7f597a1 Merge pull request #29 from Chessuker/feature/4-create-ticket
| |\
| | * 8fbc0b0 (origin/feature/4-create-ticket) feature 4: implement ticket creation API, resilient create form UI, and automated tests
* | | f8d1155 Merge pull request #28 from Chessuker/feature/3-requester-context
|\ \ \
| | /
| | /
| * | 3a7f601 (origin/feature/3-requester-context) feature 3: implement requester context, selector screen, and change requester shell action
| | /
| * d20a79e Merge pull request #27 from Chessuker/feature/2-db-schema-seed
| |\
| | * 1ae37ec (origin/feature/2-db-schema-seed, feature/2-db-schema-seed) resolve feature2
| | * b07fd8e Merge branch 'lab2-staging' into feature/2-db-schema-seed
| | \
| | /
| | /
| * | 7a9224c Merge pull request #26 from Chessuker/feature/1-spec-docs
| / |
| * | 61089de (origin/feature/1-spec-docs) add api_spec and reviewer md
| | * a7cbd3f fix md
| | * f8302fc edit gitignore
| | * 27939be feature2: implement database schema, idempotent seed data, and unit tests
| | /
| * de27ca3 docs: complete sprint engineering spec, ui-spec, test plan, and ai-use tracking
| * 5096b1d docs: complete sprint engineering spec, ui-spec, test plan, and ai-use tracking
| /
* 4d334fa Fix repository link in README
* 08d187f Merge pull request #15 from Chessuker/edit-md-lab1
|\
| * 8086598 (origin/edit-md-lab1) Merge branch 'main' into edit-md-lab1
| |\
| | /
| /
```

The graph above is the verbatim output of `git log --graph --oneline --decorate -25`, kept as text in [test-runs/git-history.txt](#). Read from the bottom up it shows seven feature branches converging on `lab2-staging`, and `lab2-staging` converging on `main` in a single merge at the top.

Issue	Feature branch	PR	Merged into	Date
#1 Spec & test planning	feature/1-spec-docs	#26	lab2-staging	2026-08-23
#2 Database schema & seed	feature/2-db-schema-seed	#27	lab2-staging	2026-08-27
#3 Requester context	feature/3-requester-context	#28	lab2-staging	2026-08-28
#4 Ticket creation	feature/4-create-ticket	#29	lab2-staging	2026-08-29
#5 My Tickets list	feature/5-my-tickets-list	#30	lab2-staging	2026-08-30
#6 Ticket detail & attachments	feature/6-ticket-detail-attachments	#31	lab2-staging	2026-09-04
#7 UI refinement & E2E	feature/7-ui-responsive-e2e	#32	lab2-staging	2026-09-05
— sprint release	lab2-staging	#33	main	2026-09-05

The same shape is visible on GitHub: the repository home page below shows `main` with **16 branches**, its latest commit being the release merge [#33](#) from `lab2-staging` .

2. GitHub Project and final Kanban



Project **TokTickIT Individual Sprints Lab 2**. All seven sprint issues sit in **Done** and every other column — Backlog, Specified, Started, PR Review, Fixing — is empty:

Card	Sprint issue
TokTickIT #19	Issue #1: Sprint Specifications, Architecture & Test Planning
TokTickIT #20	Issue #2: Database Schema & Idempotent Seed Data
TokTickIT #21	Issue #3: Development Requester Context & Simulated Login
TokTickIT #22	Issue #4: Ticket Creation API & UI with Validation
TokTickIT #23	Issue #5: My Tickets List with Filtering, Search & Pagination
TokTickIT #24	Issue #6: Requester Ticket Detail & Soft Attachment Lifecycle
TokTickIT #25	Issue #7: UI Refinement, Responsive Layout & End-to-End Tests

The card numbers are GitHub issue numbers; the sprint numbering used throughout this documentation is the "Issue #1 ... #7" in each title.

3. Peer review

The reviewer's identity, the comments given and received, the responses and the approvals are recorded in [reviewer.md](#), which is submitted rendered alongside this part.

Two of the seven pull requests were sent back before they were approved. PR #31 is the clearest record of the whole cycle on one page — review, fix, approval, merge:

feature 6: implement ticket detail view, attachment upload, download, soft removal, and ownership guards #31

Merged TeekhathatTT merged 2 commits into lab2-staging from feature/6-ticket-detail-attachments 18 hours ago

Conversation 3 Commits 2 Checks 0 Files changed 35

Chessuker commented yesterday • edited

Owner

Reviews

Assignees

Labels

Projects

Milestone

Development

Notifications

2 participants

Lock conversation

Closes #6.

Adds the Requester Ticket Detail screen (read-only) and the full soft attachment lifecycle: upload, download, and removal that keeps the record.

What changed

API (server/src/app.ts, attachmentRules.ts, attachmentUpload.ts)

Endpoint	Behaviour
GET /api/tickets/:id	One owned ticket with its attachments, active first. 403 if it belongs to someone else, 404 if it does not exist
POST /api/tickets/:id/attachments	One file per request. 415 wrong type, 413 over 5 MB, 400 ATTACHMENT_LIMIT_REACHED on the sixth active file
GET /api/attachments/:id	Metadata, including for removed files, with downloadUrl: null
GET /api/attachments/:id/download	Streams an active file. 403 for a removed one, 403 for another requester's, 404 if the row exists but the file does not
PATCH /api/attachments/:id/remove	Sets isRemoved, removedReason, removedAt. 400 without a reason. Nothing is deleted

UI (client/src/components/TicketDetail.tsx, route /tickets/:id)

All ticket fields read-only, attachment list split into active and removed, upload zone, removal modal, and an access-denied screen.

Also wires CreateTicketForm to actually upload the files staged during creation. The placeholder text saying uploads were not implemented yet was no longer true once this endpoint existed.

Three decisions worth arguing with

1. Multer runs inside the route handler, not as middleware. As middleware, an oversized body from a requester who does not own the ticket would answer 413 before ownership was ever checked. Running the ticket lookup first keeps the check order that api-spec.md §3.7 fixes, and refuses an intruder before accepting a single byte from them.

2. File type comes from the leading bytes, not the extension or Content-Type. sniffMimeType reads the JPEG/PNG/WEBP/PDF signature. An executable renamed to invoice.pdf is refused with 415; there is a test for exactly that.

3. A removed attachment answers 403, not 404. The resource genuinely exists and its owner can still read its metadata. What is refused is the state, not the existence. Happy to switch to 404 if you read BR-09 differently, but then the metadata endpoint has to change too.

Against the review checklist

• RC-04 contract — every path, status code and error envelope follows api-spec.md §3.6 to §3.10. storagePath is never in a response body.

• RC-05 ownership server-side — enforced in loadOwnedTicket / loadOwnedAttachment, one place each, so a new attachment route cannot forget it. Proven twice over: API-05 and E2E-03 hit GET /api/tickets/:id and GET /api/attachments/:id/download directly as the wrong requester and expect 403 with no Content-Disposition and no ticket content in the body.

• RC-02 / RC-03 tests — every AC claimed below has a test named in tests.md. No skips, no .only.

• RC-07 / RC-08 UI — Zen Green tokens only, no new colours. Detail screen measured at 375 px and 1280 px: document.body.scrollWidth equals the viewport width in both, so no horizontal overflow.

• RC-06 migrations — none. The Attachment table already landed in Issue Implement the API health check #2; this PR only starts writing to it.

• RC-09 / RC-10 — no .env, no build output. server/uploads/, e2e/report/ and test-results/ are gitignored. Two dependencies added: multer (server) and playwright/test (root).

Tests

Suite	Result	Added this PR
npm run test:server	124 passed, 6 files	API-05, API-07 ... API-11, API-14
npm run test:client	80 passed, 6 files	UI-05, UI-06
npm run test:e2e	6 passed, 2 files	E2E-02, E2E-03

attachments.api.test.ts mocks Prisma but deliberately does not mock the file system: it points UPLOAD_DIR at a temp folder and lets the route write and read real bytes, because "the file reaches disk and comes back" is the thing the download and removal rules are about. It also asserts that a refused upload leaves the directory untouched and that a removed file is still on disk afterwards.

E2E needs the stack up first:

docker compose up -d && npm run prisma:migrate && npm run prisma:seed && npm run test:e2e

playwright.config.ts starts the API and Vite itself but never the database, so a run cannot quietly pass against an empty schema.

Evidence

PR #31 review and approval — 1 of 2

prisma-ignite.com/ign... stores the ID and the user but never the database, so a user cannot query pass against an empty schema.

Evidence

Screenshots and the walkthrough are in `docs/lab-02/answer-part8-screenshots.md`, captured against the live stack by `docs/lab-02/screenshots/capture-issue6.mjs`. Ticket `TKT-2026-000041` is a real row. The downloaded PDF is saved next to the screenshots and is byte-identical to the file that was uploaded.

`docs/lab-02/tests.md` is updated with the results, and API-D-01 in `api-spec.md` is resolved: local disk under `server/uploads/`, uuid file names, `UPLOAD_DIR` to override.

Out of scope, on purpose

Public Comments, Internal Notes, Actions Taken and any status-change control. The issue excludes them and the detail screen is read-only this sprint. E2E-01 and E2E-04 belong to Issue #7; this PR adds the Playwright harness they will use.

What I would most like you to check

Whether the check order in `POST /api/tickets/:id/attachments` is right, and whether the access-denied screen leaks anything it should not. It renders the selected requester's own name, which felt safe, but it is the one place where a refusal still prints something.

👤 feature 5: implement ticket detail view, attachment upload, download, ... f7f2a07

 **TeekhathatTT** requested changes 19 hours ago View reviewed changes

TeekhathatTT left a comment Collaborator ...

แก้ไข: `server/src/app.ts` + `attachmentUpload.ts`

ปัญหา: ไฟล์อัปโหลด + เก็บขนาด ในโค้ดแจ้ง error 413 (`api-spec.md`) นอกควรต้องเป็น 415 นะจ๊ะ

แก้ไข: ปรับให้ MIME check มาก่อน size check

ที่เหลือถูกแล้ว เก่งมากโคตรปัสโดนบอย

👤 fix MIME check 4db72b0

 **Chessuker** commented 19 hours ago Owner Author ...

จัดไปพี่ชาย รอลี้กครุ

👤 Commit + Push ใหม่แล้ว

  **TeekhathatTT** approved these changes 18 hours ago View reviewed changes

TeekhathatTT left a comment Collaborator ...

เช็คแล้ว ถูกต้อง 67

  **TeekhathatTT** merged commit `fe97031` into `lab2-staging` 18 hours ago Revert

 Pull request successfully merged and closed Delete branch

You're all set — the `feature/6-ticket-detail-attachments` branch can be safely deleted.

PR #31 review and approval — 2 of 2

Reading down the conversation: TeekhathatTT **requested changes** on the 413/415 defect, the fix landed in commit `4db72b0`, and TeekhathatTT then **approved these changes** and merged. PR #32 shows a clean approval on the final issue:

feature 7: refine zen green UI, responsive styles, and implement Play... #32

Merged

TeekhatatTT merged 1 commit into lab2-staging from feature/7-ui-responsive-e2e 5 hours ago

Conversation 0

Commits 1

Checks 0

Files changed 32

Chessuker commented 5 hours ago

Owner

Reviewers

Assignees

Labels

Projects

Milestone

Development

Notifications

2 participants

Lock conversation

Closes #7. Last issue of the sprint.

Finishes the end-to-end suite, fixes what a measured pass over the UI found at three breakpoints, and fills in every remaining TBD in the lab documentation except the peer-review sections that are yours to write.

What changed

End-to-end (e2e/) — the two missing journeys, so all four planned specs now exist.

Spec	Test ID	Journey
create-ticket.spec.ts	E2E-01	Select requester → fill form → submit → read the ticket number → open the detail screen and find the same data
requester-context.spec.ts	E2E-04	Guard redirects every route with no requester selected; switching requester swaps the visible ticket set
attachments.spec.ts	E2E-02 (from #6)	
ownership.spec.ts	E2E-03 (from #6)	

E2E-01 threads the server-generated ticket number through three screens rather than asserting each screen on its own. If the same number appears on the confirmation, on the detail screen beside the same summary and category, and in a search of My Tickets, the ticket was really persisted.

UI (zen-green.css , CreateTicketForm.tsx , MyTickets.tsx) — five fixes, all of them things a script found by reading the page rather than by looking at it.

Finding	Fix
Mobile controls 28-38 px tall	44 px minimum below 768 px on buttons, page numbers and the header toggle
Create Ticket ran one field per row at tablet width	Two-column grid from 768 px; Summary, Description and the upload area opt out with zg-field-wide
Header wrapped to two lines at 820 px	Nav and requester block pinned to one line; the small "Requester" caption drops at that width
Ticket number ran under the Created Date at tablet width	.zg-table minimum 56 rem → 66 rem, Ticket No. minimum 12.5% → 13.5%
Pagination pushed Next onto a line of its own at 375 px	Previous and Next share the first row, numbers below

Docs — tests.md every planned test now says Pass with the final run table; ui-spec.md V-01 ... V-10 filled in with what was measured for each; new answer-part3-test-runs.md and answer-part9-screenshots.md ; the tap-target TBD in ui-spec #4 and the last two open decisions in api-spec #6 resolved.

Three decisions worth arguing with

1. 44 px is scoped to below 768 px, not applied everywhere. WCAG 2.2 AA (2.5.8) asks for 24 px and every control already cleared that, so this is about touch, not compliance. Making the table's Detail action 44 px on desktop would cost a row of vertical space on every screen to solve a problem a mouse does not have. If you think the rule should be global, say so — it is one media query either way.

2. The table's minimum width is the container width (66 rem), not a round number. Wider and the desktop table scrolls, which hides the Detail action off the right edge. Narrower and the percentage columns squeeze until .txt-2026-000056 runs under the date beside it. It is a measured value, so it will need revisiting if the container's max width ever changes — there is a comment saying exactly that.

3. The tablet header drops the "Requester" caption rather than shrinking the font further. The name underneath already says what the caption says. The alternative was another font-size step, which starts fighting the type scale.

Against the review checklist

• RC-01 — the diff is UI CSS, two component files, two new E2E specs, two capture scripts, and documentation. No API changes, no schema changes, no new dependencies.

• RC-02 / RC-03 — every row in the tests.md planned table now says Pass, and each names the file that proves it. No skips, no .only , no test disabled to make the suite green.

• RC-07 UI matches the tokens — no new colours. The audit read the computed values back: header rgb(0, 107, 60) , page rgb(245, 247, 246) , cards rgb(255, 255, 255) , focus ring 2px solid rgb(11, 122, 70) , field errors #DC2626 with a matching border, all straight from the token table.

• RC-08 responsive — nine screen-and-width combinations, document.body.clientWidth equal to the viewport width in every one. Every leaf element's scrollWidth checked against its clientWidth; the only hit is the deliberately visually-hidden file input. Lowest contrast ratio anywhere is 4.83:1, above the 4.5:1 AA floor.

• RC-09 / RC-10 — no .env , no build output. test-results/ and e2e/report/ stay gitignored; docs/lab-02/test-runs/*.txt are committed on purpose as the Part 3 evidence.

Tests

Suite	Command	Files	Tests	Result
Backend	npm run test-server	6	130	pass

PR #32 approval — 1 of 2

CPE334 Lab 2 — TokTickIT — Thawat Boonsuk 67070501024

6 / 103

Suite	Command	Files	Tests	Result
Backend	<code>npm run test:server</code>	6	130	pass
Frontend	<code>npm run test:client</code>	6	80	pass
End-to-end	<code>npm run test:e2e</code>	4	16	pass
Total		16	226	226 pass, 0 fail

The verbatim output of all three commands is committed under `docs/lab-02/test-runs/`. E2E needs the stack up first:

```
docker compose up -d && npm run prisma:migrate && npm run prisma:seed && npm run test:e2e
```

Two things to know before you read the diff

- `ai-use.md` is a draft I still need to rewrite in my own words, particularly P-01 ... P-04, which cover Issues [Set up the TokTickIT project foundation #1](#) and [Implement the API health check #2](#). Review it for whether it describes the sprint accurately; the wording is going to change before submission.
- The screenshots show ticket summaries beginning `E2E`. Those are genuine rows the end-to-end suite created through the real API against the same database, not fixtures. I can reseed and re-capture if you think the report reads better without them.

What I would most like you to check

Whether the V-01 ... V-10 table in `ui-spec.md` claims anything the evidence column does not actually support. Some rows are measurements and some are judgements, and I would rather you catch a judgement dressed up as a measurement now than have it in the report.



`feature 7: refine zen green UI, responsive styles, and implement Play...`

`dad25b8`



TeekhathatTT approved these changes [5 hours ago](#)

[View reviewed changes](#)

TeekhathatTT left a comment

[Collaborator](#)

GOOD JOB FROM SAMSUNG



TeekhathatTT merged commit `229659e` into `lab2-staging` [5 hours ago](#)

[Revert](#)



Pull request successfully merged and closed

You're all set — the `feature/7-ui-responsive-e2e` branch can be safely deleted.

[Delete branch](#)

PR #32 approval — 2 of 2

Approvals on PR [#26](#) and PR [#29](#) are captured too, and are reproduced under Answer Part 2 where their merge dates are the point.

4. README

The repository README is at `README.md` — 153 lines covering what the project is, the stack, the directory layout, how to run the database, server and client, and the test commands.


TokTickIT
Public
Pin

main
16 Branches
0 Tags

Add file
Code


TeekhathatTT

Merge pull request #33 from Chessuker/lab2-staging

9936b8c · 1 hour ago

51 Commits

.claude	feature 4: implement ticket creation API, resilient create form ...	last week
client	feature 7: refine zen green UI, responsive styles, and impleme...	5 hours ago
docs	feature 7: refine zen green UI, responsive styles, and impleme...	5 hours ago
e2e	feature 7: refine zen green UI, responsive styles, and impleme...	5 hours ago
server	fix MIME check	19 hours ago
.env.example	resolve feature2	last week
.gitattributes	Initial commit	last month
.gitignore	feature 5: implement ticket detail view, attachment upload, d...	yesterday
Feature1.png	edit md lab1	3 weeks ago
Feature2.png	edit md lab1	3 weeks ago
Feature3.png	edit md lab1	3 weeks ago
Feature4.png	edit md lab1	3 weeks ago
KanbanLab1.png	edit md lab1	3 weeks ago
Lab_02_labsheet.pdf	feature2: implement database schema, idempotent seed data...	2 weeks ago
README.md	resolve feature2	last week
docker-compose.yml	resolve feature2	last week
package-lock.json	feature 5: implement ticket detail view, attachment upload, d...	yesterday
package.json	feature 5: implement ticket detail view, attachment upload, d...	yesterday
playwright.config.ts	feature 5: implement ticket detail view, attachment upload, d...	yesterday

TokTickIT

TokTickIT is a full-stack IT ticketing/status portal built with a React + Express + PostgreSQL stack, wrapped in a deliberately over-the-top "1999 cyber portal" retro UI. It exposes a health-check endpoint that reports live database connectivity, a category list backed by Postgres, and a simple user directory with create support.

Table of contents

- [Features](#)
- [Tech stack](#)
- [Project structure](#)
- [Requirements](#)
- [Installation](#)
- [Configuration](#)
- [Usage](#)
- [Available scripts](#)
- [Tests](#)
- [Lab documentation](#)
- [Contact](#)

Repository home page and rendered README — 1 of 4

- [Lab documentation](#)
- [Contact](#)

Features

- **Live health check** — `GET /api/health` runs `SELECT 1` against Postgres and returns `200 / ok` or `503 / error` based on real DB connectivity, not a hardcoded response.
- **Category list** — categories are seeded in Postgres and served through `GET /api/categories`, rendered by the frontend with loading and error states.
- **User directory** — list existing users (`GET /api/users`) and create new ones from the UI (`POST /api/users`), with server-side email validation.
- **Retro cyber UI** — a Bootstrap 5-themed “late-90s internet” interface (marquee ticker, window chrome, blinking badges) as the frontend shell.

Tech stack

Layer	Technology
Frontend	React 19, TypeScript, Vite, Bootstrap 5
Backend	Node.js, Express, TypeScript
Database / ORM	PostgreSQL, Prisma
Testing	Vitest, React Testing Library (frontend), Vitest + Supertest (backend)
Linting	oxlint

Project structure

```
TokTickIT/
├── client/           # React + Vite frontend
│   └── src/
│       ├── App.tsx
│       └── components/CategoryList.tsx
├── server/          # Express + Prisma backend
│   ├── src/
│   │   ├── app.ts    # Express app & routes
│   │   ├── index.ts  # Entrypoint
│   │   ├── db.ts     # Prisma client
│   │   └── seed.ts   # DB seed script
│   ├── prisma/schema.prisma
│   └── tests/app.test.ts
├── docs/lab-01/     # CPE334 Lab 1 write-ups (tests, AI usage, peer review)
└── package.json     # Root scripts that fan out to client/server
```

Requirements

- Node.js `>= 18` (developed on v24)
- npm `>= 10`
- PostgreSQL `>= 13` — either a local instance, a hosted connection string, or Docker (a `docker-compose.yml` is included)

Installation

```
git clone https://github.com/Chessuker/TokTickIT.git
cd TokTickIT
npm --prefix client install
npm --prefix server install
```

Configuration

Start a local PostgreSQL with Docker (listens on host port 5433 so it does not clash with an existing 5432 instance):

```
docker compose up -d
```

Repository home page and rendered README — 2 of 4

```
docker compose up -d
```

Then copy the example env file and fill in real values:

```
cp server/.env.example server/.env
```

`server/.env`:

```
PORT=5000
DATABASE_URL="postgresql://postgres:postgres@localhost:5433/toktickit?schema-public"
```

The value above matches the bundled `docker-compose.yml`. Point it at your own instance if you are not using Docker.

The frontend reads `VITE_API_URL` (defaults to `http://localhost:5000` if unset) — set it in a `client/.env` file if your API isn't running on the default port.

Then generate the Prisma client, apply the migrations, and seed the database. The seed is idempotent — running it again never duplicates rows:

```
npm run prisma:generate
npm run prisma:migrate
npm run prisma:seed
```

Usage

Run backend and frontend in separate terminals:

```
npm run dev:server # http://localhost:5000
npm run dev:client # http://localhost:5173
```

Open `http://localhost:5173` in a browser. The app checks `/api/health` and loads `/api/categories` and `/api/users` on load.

Available scripts

Run from the repo root (each forwards to the relevant workspace):

Script	Description
<code>npm run dev:client</code>	Start the Vite dev server
<code>npm run dev:server</code>	Start the Express server with <code>tsx watch</code>
<code>npm run build:client</code>	Type-check and build the frontend
<code>npm run build:server</code>	Compile the backend with <code>tsc</code>
<code>npm run test:client</code>	Run frontend tests (Vitest)
<code>npm run test:server</code>	Run backend tests (Vitest + Supertest)
<code>npm run prisma:generate</code>	Regenerate the Prisma client
<code>npm run prisma:db:push</code>	Push <code>schema.prisma</code> to the database
<code>npm run prisma:migrate</code>	Apply pending migrations (<code>prisma migrate deploy</code>)
<code>npm run prisma:seed</code>	Seed reference data — safe to re-run

Additional server-only scripts (`npm --prefix server run ...`): `prisma:migrate`, `prisma:seed`.

Tests

```
npm run test:server # Supertest against Express, Prisma mocked – no live DB needed
npm run test:client # Vitest + React Testing Library
```

Both suites currently pass (7 backend / 3 frontend tests). See [docs/lab-01/tests.md](#) for the full breakdown of what each test covers.

Both suites currently pass (7 backend / 3 frontend tests). See [docs/lab-01/tests.md](#) for the full breakdown of what each test covers.

Lab documentation

This repo doubles as the CPE334 Lab 1 submission. Supporting docs live in [docs/lab-01/](#) :

- [tests.md](#) — test environment and results
- [ai_use.md](#) — AI tool usage log
- [reviewer.md](#) — peer review records

Contact

Project author: [Chessuker](#) — repo: github.com/Chessuker/TokTickIT

Repository home page and rendered README — 4 of 4

The same capture doubles as evidence for the repository layout below: the file listing above the README shows the tracked root — `.claude/`, `client/`, `docs/`, `e2e/`, `server/`, `docker-compose.yml`, `playwright.config.ts`, `package.json` — with each entry's last commit message beside it.

5. .gitignore

```
# Dependencies
node_modules/

# Environment & Secrets
.env
.env.local
.env.*.local

# Build output
dist/
build/
*.tsbuildinfo

# Logs
npm-debug.log*
yarn-debug.log*
yarn-error.log*
logs/
*.log

# IDE & OS files
.vscode/
.idea/
.DS_Store
Thumbs.db

# Course material, not project source
Lab_02_labsheet.pdf

# Uploaded attachments (local disk storage, API-D-01)
server/uploads/

# Playwright
e2e/report/
test-results/
playwright/.cache/
```

Three entries are worth pointing at, because they are decisions rather than boilerplate:

- **.env** — the database URL and port live there and are never committed. `server/.env` is required to run the stack; the README says what it must contain.
- **server/uploads/** — attachments are stored on local disk (decision API-D-01 in [api-spec.md](#)). The files uploaded during development and testing are real user data as far as the repository is concerned, so they stay out of it.
- **test-results/**, **e2e/report/** — Playwright's traces, videos and HTML report are regenerated on every run. The one piece of test output that *is* committed is `docs/lab-02/test-runs/*.txt`, because that is the Part 3 evidence.

6. Repository directory structure

Tracked files only, screenshots omitted for length. The full listing is in [test-runs/directory-structure.txt](#).

```

TokTickIT/
├── client/                                React 19 + Vite frontend
│   ├── public/
│   └── src/
│       ├── assets/
│       ├── components/                AppShell, RequesterSelector, CreateTicketForm,
│                                       MyTickets, TicketDetail, RequireRequester
│                                       (each with its .test.tsx beside it)
│       ├── context/                  RequesterProvider and the requester context
│       ├── pages/                    Route wrappers: CreateTicket, MyTickets,
│                                       TicketDetail, SystemStatus
│       ├── styles/                   zen-green.css - the whole theme in one file
│       └── test/                      Vitest setup
├── server/                             Express + Prisma backend
│   ├── prisma/
│   │   ├── migrations/                Two migrations: init, lab02_requester_ticketing
│   │   └── schema.prisma
│   ├── src/
│   │   ├── app.ts, db.ts, httpErrors.ts, requesterContext.ts,
│   │   ├── ticketNumber.ts, ticketValidation.ts,
│   │   ├── ticketListQuery.ts, attachmentRules.ts,
│   │   ├── attachmentUpload.ts, seed.ts, seedData.ts
│   │   └── tests/                     app, requesters, tickets, attachments, seed,
│                                       ticketNumber
├── e2e/                                Playwright: create-ticket, attachments,
│                                       ownership, requester-context, helpers
├── docs/
│   ├── lab-01/
│   └── lab-02/                        specification, api-spec, ui-spec, tests,
│                                       ai-use, reviewer, answer-part* and screenshots
├── docker-compose.yml                 PostgreSQL 16 on host port 5433
├── playwright.config.ts
├── package.json                      Workspace scripts: dev, build, test, prisma
└── README.md

```

Two things the layout is deliberate about. Component tests sit beside the component they test rather than in a separate tree, so a file and its test move together. Server source is split by concern rather than by layer — `attachmentRules.ts` holds the rules with no Express or Prisma in it, which is what lets them be read and tested on their own.

Screenshot to add: the same tree in the IDE's file explorer, which is the form the lab sheet asks for.

Evidence index for this part

Item	File
Branch and merge history on <code>main</code>	git-history.png , text in test-runs/git-history.txt
Final Kanban, all seven issues Done	kanban-final.png
PR #26 — changes requested, then approved	pr-approval-#26.png
PR #29 — approved	pr-approval-#29.png
PR #31 — changes requested, fixed, approved	pr-approval-#31.png
PR #32 — approved	pr-approval-#32.png
Repository home page, file listing and rendered README	README-screenshot.png

Item	File
Full tracked-file listing	test-runs/directory-structure.txt
Peer review record	reviewer.md

Lab 2 — Peer Review Documentation

Record of the peer review performed on this sprint: who reviewed, which pull requests, what was raised, and how it was resolved.

1. Peer Reviewer Information

Field	Detail
Name	Theehathat
Student ID	67070501019
GitHub Username	TeekhathatTT
Their repository, which I reviewed	https://github.com/TeekhathatTT/CPE334-TokTickIT
My repository, which they reviewed	https://github.com/Chessuker/TokTickIT

My Information

Field	Detail
Name	Thawat Boonsuk
Student ID	67070501024
GitHub Username	Chessuker

2. Reviewed PR Links

Pull requests submitted by me and reviewed by my partner, and vice versa. One row per PR.

Every PR below was opened by **Chessuker** and reviewed by **TeekhathatTT**. Issue branches went to `lab2-staging` first and reached `main` only through the release PR at the bottom of the table; #28 is the exception, merged to `main` directly before the staging workflow settled.

Two of the seven were sent back before they were approved. That is the part of this table worth reading: #27 and #31 both changed the code.

#	Issue	PR Link	Branch	Base	Reviewed By	Merged	Review Outcome
1	#1 Spec & test planning	#26	<code>feature/1-spec-docs</code>	<code>lab2-staging</code>	TeekhathatTT	2026-08-23	Changes requested → Approved
2	#2 Database schema & seed	#27	<code>feature/2-db-schema-seed</code>	<code>lab2-staging</code>	TeekhathatTT	2026-08-27	Changes requested → fixed
3	#3 Requester context	#28	<code>feature/3-requester-context</code>	<code>main</code>	TeekhathatTT	2026-08-28	Approved
4	#4 Ticket creation	#29	<code>feature/4-create-ticket</code>	<code>lab2-staging</code>	TeekhathatTT	2026-08-29	Approved

#	Issue	PR Link	Branch	Base	Reviewed By	Merged	Review Outcome
5	#5 My Tickets list	#30	feature/5-my-tickets-list	lab2-staging	TeekhathatTT	2026-08-30	Approved
6	#6 Ticket detail & attachments	#31	feature/6-ticket-detail-attachments	lab2-staging	TeekhathatTT	2026-09-04	Changes requested → Approved
7	#7 UI refinement & E2E	#32	feature/7-ui-responsive-e2e	lab2-staging	TeekhathatTT	2026-09-05	Approved
8	Sprint release	#33	lab2-staging	main	— release merge of already-reviewed work	2026-09-05	Merged

3. My PRs (submitted by me, reviewed by my partner)

PR Link	Partner's Review Comments	My Response	Outcome
#26 — Issue #1, spec & test plan	Changes requested. Two structural items: split the API contract out of <code>specification.md</code> into its own <code>docs/lab-02/api-spec.md</code> , and create <code>docs/lab-02/reviewer.md</code> to record the peer review	Both done in the same branch. Splitting the contract out turned out to matter more than it looked: <code>api-spec.md</code> became the document every later PR was argued against	Approved — "Everything in this feat is done. Good job!" Merged 2026-08-23
#27 — Issue #2, schema & seed	Changes requested. "1. ใน PR บอกว่าเพิ่ม AD-07/08/09/11 แต่ใน spec ยังมีแค่ AD-01 ถึง AD-06 เหมือนเดิม 2. typo <code>toktikit</code> (ขาด "c") ใน <code>docker-compose</code> และ <code>.env.example</code> "	Fixed both in commit <code>1ae37ec</code> . The first one was the PR description claiming decisions the specification did not actually contain — the description was ahead of the document	Fixed and merged 2026-08-27. The typo would have broken the database name in two config files at once
#28 — Issue #3, requester context	Approved. "ตรงตามที่ระบุใน description ทุกอย่างแล้ว เก่งมาก bro"	—	Approved , merged 2026-08-28
#29 — Issue #4, ticket creation	Approved. "T major เช็คแล้วตรงกับ PR ทุกจุด: - Commit history ตรง (<code>8fbc0b</code> บน <code>3a7f601</code> บน <code>d20a79e</code>) - Ticket number gen ในทราจแซดชันเดียวกับ insert, retry - <code>status</code> ไม่เขียนจาก request, <code>requesterId</code> มาจาก header - Validation ปฏิเสธ <code>status / ticketNumber</code> ที่ client ส่งมา - <code>/api/categories</code> → <code>{ data }</code> , เพิ่ม <code>/api/related-systems</code> - ไม่มี migration ใหม่, ไม่มี secret หลุด"	—	Approved , merged 2026-08-29
#30 — Issue #5, My Tickets list	Approved. "เช็คแล้วตรงกับ PR ทุกจุด เก่งมาก" — with the specifics named: file line counts matched the diff, <code>requesterId</code> read only from the header and never from the query string, duplicate query parameters rejected rather than last-one-wins, the <code>id:asc</code> tie-break that stops a row appearing on two pages, search covering <code>ticketNumber/summary/description</code> with the 300 ms debounce and the request-id guard, and no migration	"แน่นอนครับ เก่งอยู่แล้ว ขอบพระทัย"	Approved — "ขอบคุณแล้ว merge ได้". Merged 2026-08-30

PR Link	Partner's Review Comments	My Response	Outcome
#31 — Issue #6, detail & attachments	Changes requested. "แก้ที่: <code>server/src/app.ts</code> + <code>attachmentUpload.ts</code> / ปัญหา: ไฟล์ผิดชนิด+เกินขนาด ในโค้ดแจ้ง error 413 (<code>api-spec.md</code>) บอกว่าต้องเป็น 415 นะจ๊ะ / แก้: ปรับให้ MIME check มาก่อน size check / ที่เหลือถูกต้องแล้ว เก่งมาก"	"จัดไปพี่ชาย รอลักครู่" → fixed → "Commit + Push ไปแล้วครับ". Confirmed and reproduced first: the check order in the route was already MIME-before-size; the real cause was <code>multer</code> 's <code>limits.fileSize</code> , which aborts the request before the route can read any of the content, so an oversized file could never reach the type check. Replaced it with a storage engine that retains the first 5 MB + 1 bytes and counts the rest	Approved — "เช็คแล้ว ถูกต้อง". Fixed in <code>4db72b0</code> : six regression tests added for requests that break two rules at once, re-verified over real HTTP (6 MB <code>.txt</code> → 415, 6 MB PNG → 413, 50 MB PNG → 413, 6 MB PNG into another requester's ticket → 403). <code>api-spec.md</code> §3.7 corrected, since its claim that the middleware enforced the size limit was no longer true. Merged 2026-09-04
#32 — Issue #7, UI & E2E	Approved. "GOOD JOB FROM SAMSUNG"	—	Approved , merged 2026-09-05

Quotes are the reviewer's own words as they appear on the pull request. The #29 row is the exception: it summarises a longer point-by-point review rather than quoting it. The full #31 exchange — request, fix, approval, merge — is visible in [pr-approval-#31.png](#).

4. Partner's PRs (submitted by my partner, reviewed by me)

Partner repository: <https://github.com/TeekhathatTT/CPE334-TokTickIT>

PR Link	My Review Comments	Partner's Response	Outcome
#19 — <code>feature/spec-tests</code> → <code>lab2-staging</code> , Lab 2 documentation	Approved. "The Sprint Goal, FRs, BRs, AC, API contract, UI design, and test plan are well-integrated and will guide implementation cleanly. Looks ready to merge once you have the peer review feedback recorded." The one thing I asked for before merge was the peer-review record itself, which is the same gap my own #26 was sent back for	—	Approved 2026-08-22, merged 2026-08-23
#20 — <code>feature/lab2-requester</code> → <code>lab2-staging</code> , Development Requester	Changes requested, twice. First pass, 2026-08-24: the test imports and the module-mock path did not match, so <code>vi.mock</code> was pointing at a path the code never imported and the Prisma mock was never applied — the suite was green while talking to a real database. I gave the exact import and mock target: <code>import app from '../src/app.js', import { getPrisma } from '../src/prisma.js', vi.mock('../src/prisma.js')</code>. Second pass, 2026-08-26: asked to confirm the <code>vi.mock()</code> call ordering in <code>requesters.test.ts</code>, and — marked optional — whether the Prisma 5 → 6 upgrade was deliberate and whether migrations and seed still ran in CI	"แก้แล้ว แต่ตรง optional ไม่ทำ" — the required fix was made, the two optional checks were left	Approved 2026-08-27 — "Ok son, if you said so, Approve." Merged the same day

The #20 finding is the one worth keeping. A mock aimed at the wrong path fails silently: nothing errors, the tests pass, and the only symptom is that a suite which claims to need no database quietly needs one. It is the mirror image of what my partner found on my [#31](#) — both defects were invisible to a green test run, and both were caught by a person reading what the code actually did against what the document said it should do.

5. Review Checklist

What each review was checked against. Referenced from the review comments above.

#	Item	Source
RC-01	The PR description matches what the diff actually changes	—

#	Item	Source
RC-02	Every acceptance criterion claimed as done has a test that proves it	tests.md
RC-03	Tests are not skipped, flaky, or unrelated to the acceptance criteria	Lab sheet §9
RC-04	The API matches the documented contract — paths, status codes, error shapes	api-spec.md
RC-05	Ownership is enforced server-side, not only hidden in the UI	BR-04, AC-03
RC-06	Database migrations are reviewed, reversible in intent, and re-runnable	AC-13
RC-07	UI matches the Zen Green tokens and component rules	ui-spec.md
RC-08	Responsive at all three breakpoints, no clipping or horizontal overflow	AC-11
RC-09	No secrets, credentials, or <code>.env</code> files committed	Lab sheet §10
RC-10	No unrelated files, build output, or compiled artifacts in the diff	—

6. Screenshot Evidence

6.1 PR Approval Evidence

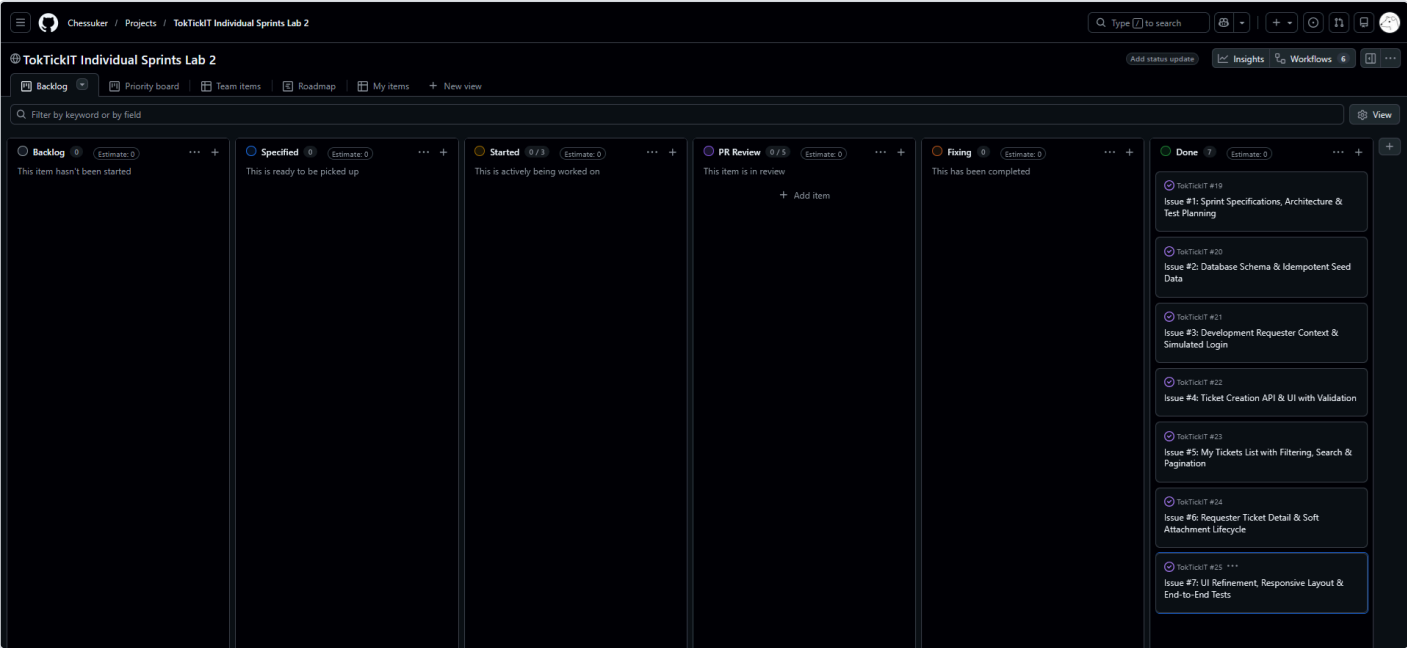
Screenshots of each approved PR. One per merged pull request.

#	PR	Screenshot
1	#31 — Issue #6: changes requested for the 413/415 defect, fixed in <code>4db72b0</code> , then approved	pr-approval-#31.png
2	#32 — Issue #7: approved, "GOOD JOB FROM SAMSUNG"	pr-approval-#32.png
3	#26 — Issue #1: changes requested (split out <code>api-spec.md</code> , add <code>reviewer.md</code>), then approved	pr-approval-#26.png
4	#29 — Issue #4: approved after a point-by-point check	pr-approval-#29.png

6.2 Kanban Board Evidence

Final GitHub Project board with every Issue in Done.

Screenshot
kanban-final.png



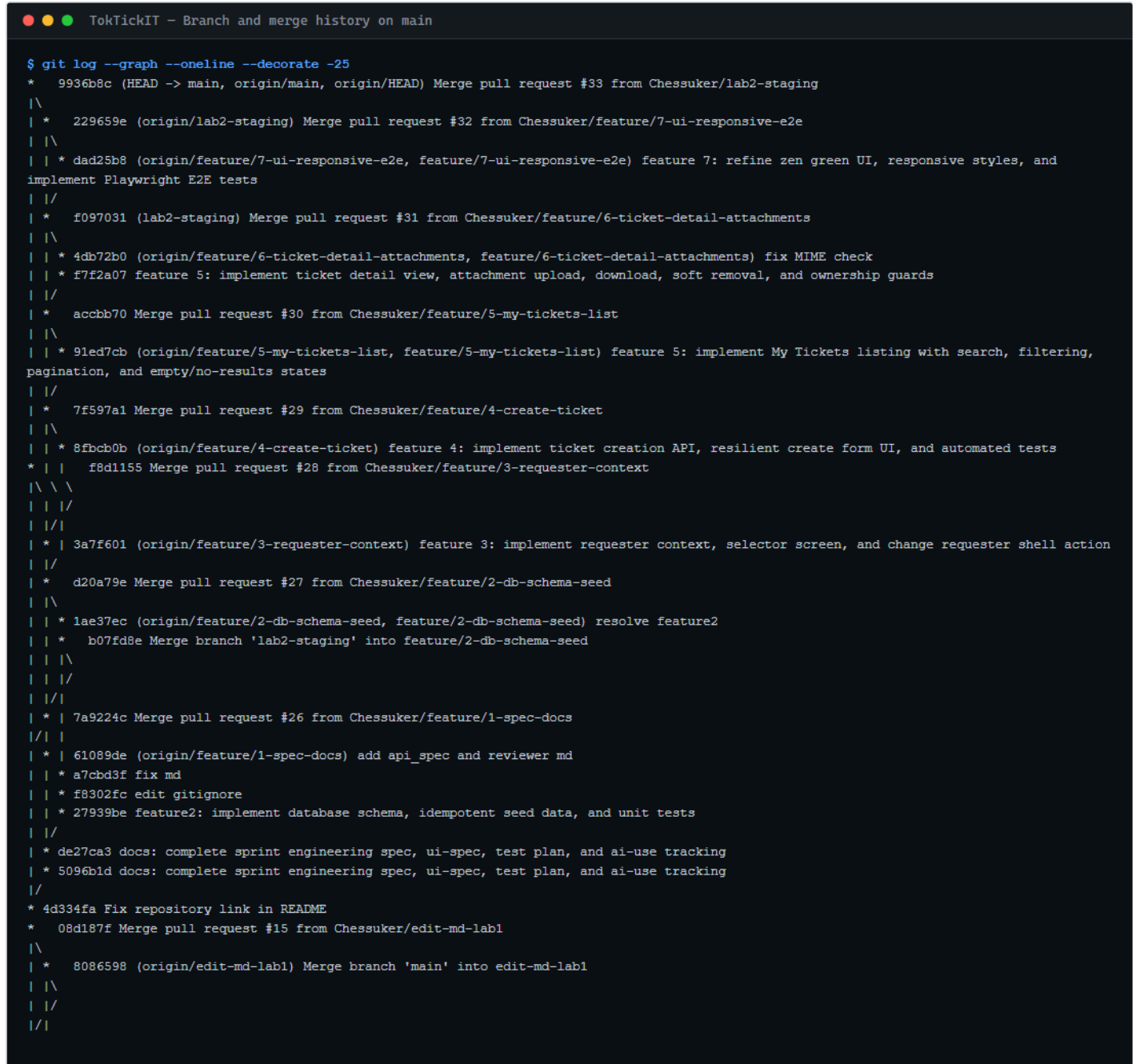
All seven sprint issues in **Done**, every other column empty.

6.3 Branch and Merge History

Commit history on `main` showing feature branches merged through staging into `main`.

Screenshot

<docs/lab-02/screenshots/git-history.png>



```
TokTickIT - Branch and merge history on main

$ git log --graph --oneline --decorate -25
* 9936b8c (HEAD -> main, origin/main, origin/HEAD) Merge pull request #33 from Chessuker/lab2-staging
|\
| * 229659e (origin/lab2-staging) Merge pull request #32 from Chessuker/feature/7-ui-responsive-e2e
| |\
| | * dad25b8 (origin/feature/7-ui-responsive-e2e, feature/7-ui-responsive-e2e) feature 7: refine zen green UI, responsive styles, and
| | implement Playwright E2E tests
| | /
| * f097031 (lab2-staging) Merge pull request #31 from Chessuker/feature/6-ticket-detail-attachments
| |\
| | * 4db72b0 (origin/feature/6-ticket-detail-attachments, feature/6-ticket-detail-attachments) fix MIME check
| | * f7f2a07 feature 5: implement ticket detail view, attachment upload, download, soft removal, and ownership guards
| | /
| * accbb70 Merge pull request #30 from Chessuker/feature/5-my-tickets-list
| |\
| | * 91ed7cb (origin/feature/5-my-tickets-list, feature/5-my-tickets-list) feature 5: implement My Tickets listing with search, filtering,
| | pagination, and empty/no-results states
| | /
| * 7f597a1 Merge pull request #29 from Chessuker/feature/4-create-ticket
| |\
| | * 8fbc0b0 (origin/feature/4-create-ticket) feature 4: implement ticket creation API, resilient create form UI, and automated tests
| | * f8d1155 Merge pull request #28 from Chessuker/feature/3-requester-context
| | \ \ \
| | / / /
| | /
| * | 3a7f601 (origin/feature/3-requester-context) feature 3: implement requester context, selector screen, and change requester shell action
| | /
| * d20a79e Merge pull request #27 from Chessuker/feature/2-db-schema-seed
| |\
| | * 1ae37ec (origin/feature/2-db-schema-seed, feature/2-db-schema-seed) resolve feature2
| | * b07fd8e Merge branch 'lab2-staging' into feature/2-db-schema-seed
| | /
| | /
| | /
| * | 7a9224c Merge pull request #26 from Chessuker/feature/1-spec-docs
| / |
| * | 61089de (origin/feature/1-spec-docs) add api_spec and reviewer md
| | * a7cbd3f fix md
| | * f8302fc edit gitignore
| | * 27939be feature2: implement database schema, idempotent seed data, and unit tests
| | /
| * de27ca3 docs: complete sprint engineering spec, ui-spec, test plan, and ai-use tracking
| * 5096b1d docs: complete sprint engineering spec, ui-spec, test plan, and ai-use tracking
| /
* 4d334fa Fix repository link in README
* 08d187f Merge pull request #15 from Chessuker/edit-md-lab1
|\
| * 8086598 (origin/edit-md-lab1) Merge branch 'main' into edit-md-lab1
| |\
| | /
| | /
```

Seven feature branches into `lab2-staging`, then `lab2-staging` into `main` in a single merge ([#33](#)). Nothing was pushed straight to `main`. The same evidence, with the full PR table, is in [answer-part1-git-workflow.md](#).

7. Review Summary

A short account of what the review process caught this sprint: the most useful comment received, the most useful comment given, and anything that would have shipped broken without it.

Most useful comment I received: The 413/415 comment from TeekhathatTT on PR [#31](#). It named two files, the rule and the line of the spec the code disagreed with, which is why it took minutes to confirm rather than an afternoon to argue about.

Second place goes to the review on [#26](#), which sent the very first PR back to split the API contract out of `specification.md` into its own file. That looked like filing at the time. It was not: `api-spec.md` became the document every later PR was argued against, and the #31 defect was only reviewable *because* the status codes lived somewhere a reviewer could point at.

Most useful comment I gave: On my partner's [#20](#): their test file mocked a module path the code never imported, so the Prisma mock silently did nothing and a suite advertised as database-free was talking to a real database. Everything was green, which is exactly why nobody had noticed. I gave the correct import and mock target rather than only naming the problem, and it was fixed in the next commit.

What the review caught that tests did not: A defect that lived in the *interaction* between two validation rules. Every individual rule in the upload path had a test — oversized file, disallowed type, sixth attachment, missing reason — and all of them passed. Nothing tested a request that broke two rules at once, so nobody noticed that an oversized file of a disallowed type answered 413 when `api-spec.md` §3.7 says the more specific 415 wins. The route's check order had been correct since the day it was written; the cause was upload middleware aborting the request before the route could read the content at all.

The lesson generalises past this one bug: a test plan built one row per acceptance criterion tests each rule in isolation, which is exactly the shape that misses ordering defects. Any endpoint with more than one validation now deserves at least one test that breaks two rules at once and asserts which error wins — six of those were added in `4db72b0`.

Answer Part 2:

Spec-Driven Development · 5 points

Answer Part 2 — Spec-Driven Development Evidence

The specification is [docs/lab-02/specification.md](#), submitted rendered alongside this part. Its companions are [api-spec.md](#) for the API contract and [ui-spec.md](#) for the visual and screen-level rules.

1. Numbered requirements, rules and criteria

Everything in the specification is numbered, and the numbering is what the rest of the sprint hangs off. Test names, commit messages, PR descriptions and source comments all cite these identifiers, so it is always possible to ask which requirement a line of code exists for.

Series	Count	What it covers
FR-01 ... FR-08	8	Functional requirements: create a ticket, requester context, list, detail, attachments, change requester, resilient failure, seed data
BR-01 ... BR-13	13	Business rules: ticket-number format, server-owned status, ownership protection, the three attachment rules, the three soft-removal rules, requester directory rules
AC-01 ... AC-15	15	Acceptance criteria in Given/When/Then form

The Definition of Done is section 10 of the specification, and section 11 records the assumptions and decisions taken along the way rather than leaving them implicit.

2. The specification existed before implementation

The specification was written and merged first. This is not a claim about intent — it is in the commit history:

Event	Commit	Date
specification.md , ui-spec.md and tests.md first committed	5096b1d	2026-08-22
Spec and test-plan PR #26 merged	7a9224c	2026-08-23
First implementation PR — ticket creation, #29	7f597a1	2026-08-29
Last implementation PR — UI and E2E, #32	229659e	2026-09-05

Six days separate the merged specification from the first implementation PR, and every implementation PR lands after it.

Verify it directly:

```
git log --diff-filter=A --format="%h %ad %s" --date=short -- docs/lab-02/specification.md
```

```
git log --oneline --merges --date=short --format="%h %ad %s" origin/main
```

The two pull requests side by side are the proof. The specification PR:

docs(lab-02): sprint specification, UI spec, and test plan #26

Merged TeekhatatIT merged 3 commits into lab2-staging from feature/1-spec-docs 2 weeks ago

Conversation 2 Commits 3 Checks 0 Files changed 6



Chessuker commented 2 weeks ago

Owner

What

Added the complete Spec-Driven Development documentation for CPE334 Lab 2 under docs/lab-02/ across all four files (documentation only — there is no production code in this PR yet).

File	Content
specification.md	Engineering contract covering 11 sections according to Appendix A
ui-spec.md	Zen Green Theme and screen behavior according to Appendix C
tests.md	Test plan and traceability matrix according to Appendix B
ai-use.md	AI usage record and reflection (Answer Part 4)

Why

Issue #1 requires all documentation to be completed before starting development, so that Issues #2-#7 have a specification, acceptance criteria, and test plan that serve as verifiable contracts from the beginning.

Details

specification.md

- Scope S-01...S-09 (included) and X-01...X-05 (excluded — no real authentication, IT staff workflow, comments, or status lifecycle beyond New)
- Functional requirements FR-01...FR-08
- Business rules BR-01...BR-13 — backend-generated ticket numbers, initial status New, ownership, attachment restrictions (JPG/PNG/WEBP/PDF, <5 MB, maximum 5 files), and soft removal
- Data model: RequesterUser, Ticket, Attachment, Category, and RelatedSystem with soft-removal fields, FKs, and indexes, including seed data requirements (4 categories, >6 related systems, >4 active + >1 inactive requester)
- API contract with status codes 200 / 201 / 400 / 403 / 404 / 413 / 415 / 500
- Acceptance criteria AC-01...AC-15 written in Given-When-Then format
- Definition of Done separated into product and course-delivery requirements
- Appendix: Mapping table between GitHub Issues Set up the TokTickIT project foundation #1-#7 Revert 6 feature/2 health check #7 and FR / BR / AC

ui-spec.md

- Complete color token set (primary #00683C, secondary #8B7A46, pale #EAF6EF, background #F5F7F6, surface #FFFFFF, border / text / error / warning)
- Component rules: labels above inputs, red asterisks, errors below fields, busy states, and focus rings
- Application shell (with Change Requester button) and four screens: Requester Selection, Create Ticket, My Tickets, and Ticket Detail
- Responsive breakpoints: Desktop >= 992 px, Tablet 768-991 px, Mobile < 768 px
- Visual inspection checklist V-01...V-10

tests.md

- Four-level test strategy: Unit, Integration/API (Supertest), UI Component (RTL), and E2E (Playwright)
- 24 planned tests with the corresponding test file paths
- Traceability matrix mapping AC-01...AC-15 to Test IDs
- Responsive checklist with screenshot paths and issue coverage table

ai-use.md

- LLM / agent information
- Key prompt table organized by phase and issue
- Reflection topics

Notes for Reviewer

- AC numbering has been aligned with the issue references already in place — Display the IT request category list #4

PR #26 — specification and test plan — 1 of 2

Notes for Reviewer

- AC numbering has been aligned with the issue references already in place — [Display the IT request category list #4](#) → AC-01, [Create and seed IT request categories #3](#) → AC-02, [Feature/2 health check #6](#) → AC-03, so the issues do not need to be modified later.
- Additional requirements identified during the second review of the issues: Change Requester + data reload (FR-06, BR-13, AC-02), preserving form values when the backend is unavailable (FR-07, AC-12), seed idempotency (BR-12, AC-13), hiding inactive requesters (BR-11, AC-14), and distinguishing the no-results state from the empty state (AC-15).
- Query parameters for `GET /api/tickets` use the names specified in Issue [Set up the TokTickIT project foundation #5](#): `search`, `category`, `status`, `sort`, `page`, `pageSize`.
- Items for which real data is not yet available are marked as `_TBD_` — test run results, screenshots, key prompts, reflection, attachment storage location, and default page size. These will be filled in when Issue [Revert 6 feature/2 health check #7](#) is completed without changing the document structure.

Related Issues

Closes [#1](#). Provides the foundation for [#2](#), [#3](#), [#4](#), [#5](#), [#6](#), and [#7](#).

👤

📁 Chessuker added 2 commits 2 weeks ago

📄 docs: complete sprint engineering spec, ui-spec, test plan, and ai-us... 5096b1d

📄 docs: complete sprint engineering spec, ui-spec, test plan, and ai-us... de27ca3

👤 TeekhathatTT requested changes 2 weeks ago View reviewed changes

TeekhathatTT left a comment Collaborator ...

Must fix before merge

- Create a separate `docs/lab-02/api-spec.md` file (currently embedded inside specification.md)
- Create `docs/lab-02/reviewer.md` to record peer review

👤

📄 add api_spec and reviewer md 61089de

👤 TeekhathatTT approved these changes 2 weeks ago View reviewed changes

TeekhathatTT left a comment Collaborator ...

Everything in this feat is done. Good job!

👤

🔗 TeekhathatTT merged commit 7a9224c into lab2-staging 2 weeks ago Revert

🔗 Pull request successfully merged and closed Delete branch

You're all set — the `feature/1-spec-docs` branch can be safely deleted.

PR #26 — specification and test plan — 2 of 2

and the first implementation PR, merged six days later:

Feature/4 create ticket #29

Merged TeekhathatTT merged 2 commits into `lab2-staging` from `feature/4-create-ticket` last week

Conversation 1 Commits 2 Checks 0 Files changed 41



Chessuker commented last week

Owner ...

Issue #4 — Ticket Creation API & UI with Validation

Closes #4

Implements Create Mode end to end: `POST /api/tickets` with server-owned ticket numbering, and the Zen Green Create Ticket screen with inline validation, a busy submit state, a success panel, and a form that survives backend failure.

⚠️ Two things to settle before reviewing

1. Base branch is `lab2-staging`, not `main`. `main` is still at Lab 1 (4d334fa); all Lab 2 work integrates through `lab2-staging`, which is where #2 was merged.

2. This branch carries Issue #3's commit. #3 is not merged anywhere yet, so `feature/4-create-ticket` includes it:

Commit	Issue	Scope
3a7f601	#3	Requester context, selector screen, app shell, Zen Green theme
8fbc80b	#4	Everything this PR is about

Cleanest order: merge the #3 PR first, then this diff collapses to 8fbc80b alone (25 files). If you'd rather review both at once, `git show 8fbc80b` isolates #4. I merged #3 in because #4 cannot function without the requester context — it needs the selected requester for `X-Requester-Id`.

Backend

File	What it does
<code>server/src/requesterContext.ts</code>	Resolves <code>X-Requester-Id</code> once, in middleware — 400 <code>REQUESTER_REQUIRED</code> if missing/malformed, 403 <code>FORBIDDEN</code> if unknown or inactive
<code>server/src/ticketValidation.ts</code>	Pure, DB-free validation of the five required fields; rejects client-supplied <code>status</code> / <code>ticketNumber</code>
<code>server/src/httpErrors.ts</code>	The single error envelope from api-spec §1.2; <code>sendInternalServerError</code> is the only path to a 500
<code>server/src/app.ts</code>	<code>POST /api/tickets</code> , <code>GET /api/related-systems</code> , categories moved to the spec's <code>{ data: [...] }</code> envelope

Points worth your attention:

- **Ticket number (BR-01)**: generated inside the *same transaction* as the insert, so no other create can slip between the read and the write. The unique constraint is the final guard; a P2002 collision is retried up to 3×. `server/src/ticketNumber.ts` already existed from [Implement the API health check #2](#) — I only wired it up.
- **Status (BR-02)**: never written. The route omits `status` entirely and lets the schema default own `New`, so there is no code path that can create a ticket in another state. A client that sends `status` gets a 400 rather than having it ignored.
- **Ownership (RC-05)**: `requesterId` comes only from the header, never the body. A body containing `requesterId` is ignored — there's a test for exactly that.
- **Safe errors (RC-04)**: 500s return the generic message plus a correlation id; the real cause is logged server-side. Asserted in tests that the Prisma error text does not appear in the response.

Frontend

`client/src/components/CreateTicketForm.tsx`, `client/src/attachments.ts`, plus form styling appended to `zen-green.css`.

- Labels above inputs, red ✖ on all five required fields, errors in `--zg-error` directly under the offending field with the border turning error-colored (ui-spec §2).
- Submit shows a spinner, disables the button, and swaps the label — no double-submit.
- Success replaces the form with the server's ticket number, status and priority badges, and links onward.
- **AC-12**: nothing clears form state except a successful create. A 500, a 400, or a dead socket all leave every value in place and

PR #29 — ticket creation — 1 of 3

- Success replaces the form with the server's ticket number, status and priority badges, and links onward.
- AC-12: nothing clears form state except a successful create. A 500, a 400, or a dead socket all leave every value in place and return the button to idle.

Deliberate deviations — please push back if you disagree

1. `relatedSystemId` is now required on create. api-spec §3.4 previously said "optional, nullable". Issue #4 lists it among the mandatory fields with a red *, and a ticket naming no system is harder to route. I followed the issue and updated api-spec §3.4 with the reasoning. The column stays nullable in the schema — that's for a later sprint (a system can be retired; the FK is `SetNull`), not a licence to omit it at create time.
2. `GET /api/categories` now returns `{ data: [...] }`. It previously returned a bare array, which contradicted api-spec §3.2. This is a breaking change to an existing endpoint, so it also touches `CategoryList.tsx` and its test, and `app.test.ts`. `CategoryList` is Lab 1 leftover and no longer routed — say the word if you'd rather I delete it than keep updating it.

Not in this PR

Attachment upload. `POST /api/tickets/:id/attachments` is Issue #6. Files here are validated and staged client-side only — type, 5 MB cap, 5-file limit — and the success screen states plainly that staged files were not attached rather than dropping them silently. The server repeats every check when #6 lands; this is convenience, not enforcement.

Testing

Suite	Files	Tests	Result
<code>npm run test:server</code>	5	58	✅ all pass
<code>npm run test:client</code>	4	41	✅ all pass

Acceptance criteria claimed done, and what proves each (RC-02):

Test	AC / BR	Covers
API-01	AC-01	Valid create → 201 with ticket number, persisted
API-02	AC-01, BR-02	Status is <code>New</code> ; <code>status</code> never written; client-supplied <code>status</code> rejected
API-03	AC-01	Missing fields → 400 naming every offending field
UI-03	AC-01	Errors render under the fields; submit blocked, no POST fired
UI-08	AC-12, FR-07	Error callout on failure; all values still present; button retryable

UNIT-01 (ticket number format and uniqueness) already passed from #2 and still does.

Also covered beyond the plan: requester-header cases (400/403), reference-integrity 400s, the `P2002` retry path, safe-500 envelopes, and the six attachment rules.

Manual verification against a real database

Run the full stack — Vite, Express, seeded PostgreSQL 16 in Docker — and walked all six states. Six screenshots and a write-up are in `docs/lab-02/answer-part6-screenshots.md`; `docs/lab-02/screenshots/capture.mjs` regenerates them.

Tickets `TKT-2026-000001` ... `000005` are real rows, all with status `New`, numbers incrementing without repeats. AC-12 was also confirmed by genuinely killing the Express process mid-session — callout shown, values intact, and pressing Create again after restart produced the next number in sequence.

Checklist notes

- RC-09 — no secrets: only `.env.example` files are tracked; `server/.env` is gitignored and not in the diff.
- RC-10 — unrelated files: the diff does include `.claude/launch.json` (adds a `server-dev` entry so the API can be started for verification) and ~470 KB of evidence PNGs under `docs/lab-02/screenshots/`. Both are intentional; tell me if you'd rather the screenshots live outside the repo.
- RC-08 — responsive (AC-11): not verified in this PR. Screenshots are desktop at 1280 px only. The form uses the existing breakpoint rules and `zg-form-actions` stacks below 768 px, but I have not checked tablet or mobile for clipping or overflow. That belongs to Issue 🚩 Revert 6 feature/2 health check #7's responsive pass — flagging it rather than claiming it.
- RC-06 — migrations: none added. Schema is unchanged from 🟢 Implement the API health check #2.

How to run it

How to run it

```
docker compose up -d && npm --prefix server run prisma:migrate && npm --prefix server run prisma:seed
```

Then start both dev servers, pick a requester, and open [Create Ticket](#).

Chessuker added 2 commits [last week](#)

- feature 3: implement requester context, selector screen, and change r... [3a7f601](#)
- feature 4: implement ticket creation API, resilient create form UI, a... [8fbc0b](#)

TeekhathatTT approved these changes [last week](#) [View reviewed changes](#)

TeekhathatTT left a comment Collaborator

T major [เช็คแล้วตรงกับ PR ทุกจุด:](#)

- Commit history [ตรง](#) ([8fbc0b](#) [บน](#) [3a7f601](#) [บน](#) [d20a79e](#))
- Ticket number gen [ในทราจแซดชั่นเดียวกับ insert, retry](#)
- `status` [ไม่เขียนจาก request, requesterId มาจาก header](#)
- Validation [ปฏิเสธ status / ticketNumber ที่ client ส่งมา](#)
- `/api/categories` [→ { data }, เพิ่ม /api/related-systems](#)
- [ไม่มี migration ใหม่, ไม่มี secret หลุด](#)

TeekhathatTT merged commit [7f597a1](#) into [lab2-staging](#) [last week](#) [Revert](#)

Chessuker mentioned this pull request [last week](#) [Merged](#)

[feature 5: implement My Tickets listing with search, filtering, pagination, and empty/no-results states #30](#)

Pull request successfully merged and closed [Delete branch](#)

You're all set — the `feature/4-create-ticket` branch can be safely deleted.

PR #29 — ticket creation — 3 of 3

PR #26 carries only documentation — `specification.md`, `api-spec.md`, `ui-spec.md`, `tests.md` — and it was merged before any endpoint, component or test file existed. Everything in PR #29 was written against criteria that were already fixed and already reviewed.

3. How the specification was actually used

Three places where writing it first changed the code rather than merely describing it.

The API contract fixed the error envelope before any route existed. `api-spec.md` §1.2 defines one shape for every non-2xx response and a table mapping each error code to its status. `server/src/httpErrors.ts` is that table in code, and it is the only way a route can answer an error — which is what makes "the code disagrees with the spec" a reviewable claim instead of an

opinion. It became one during peer review on Issue #6: an upload that broke two rules at once returned 413 where the spec's ordered validation table says 415 wins. The fix and its regression tests are recorded in [tests.md](#) §5.

The acceptance criteria decided the test plan before the tests. [tests.md](#) §2 was written in the same PR as the specification, one row per planned test naming the AC it covers and the file it would eventually live in. The traceability matrix in §3 then caught that AC-15 had no API-level coverage, which is why API-13 exists.

Open questions stayed open instead of being guessed. [api-spec.md](#) §6 lists the decisions that were not settled at specification time — attachment storage, whether search should match category names, rate limiting, cache headers — as explicit TBDs. Each was resolved in the issue that needed it, with the reasoning recorded in place. API-D-01 became "local disk under `server/uploads/`, uuid file names" during Issue #6; API-D-02 became "no, `search` covers summary, description and ticket number only" during Issue #5.

4. Scope held

Section 3 of the specification lists what is excluded, and the exclusions survived the sprint. The ticket status enum still has exactly one value, `New`, because the status lifecycle is another sprint's work; the detail screen has no comments, internal notes or status control for the same reason. `TicketStatus` in `schema.prisma` carries a comment saying so, so the next person to open it does not read a one-value enum as an oversight.

Lab 2 — Sprint Engineering Specification

Field	Detail
Project	TokTickIT
Sprint / Lab	CPE334 Lab 2 — Requester Ticketing MVP
Author	Thawat Boonsuk (67070501024)
Status	Draft
Last updated	2026-09-05 (Issue #7 — sprint complete)
Related documents	api-spec.md , ui-spec.md , tests.md , reviewer.md , ai-use.md

1. Sprint Goal

Deliver the Requester-side Ticketing MVP of TokTickIT: a Development Requester selector that stands in for real authentication, a Create Ticket form, a My Tickets list scoped to the selected requester, a read-only Ticket Detail page, and an attachment lifecycle that supports upload, download, and soft removal — all presented under the Zen Green Theme and responsive from mobile to desktop.

2. Stakeholder Request Interpretation

- **Who the user is:** Requesters (KMUTT end-users, students, and staff) requesting IT support services, interacting through a temporary Development Requester identity.
- **What problem they have today:** Requesters lack a standardized, self-service channel to log IT support tickets, provide supporting attachments, and track the status of their own submitted issues without unauthorized cross-user access.
- **What they expect to be able to do after this sprint:**
 - Select a simulated Development Requester profile to establish their session context.
 - Create and submit IT support tickets with title/summary, detailed description, category, affected system, requested priority, and validated attachments.
 - View a scoped list of their own tickets in "My Tickets" with search, filter, sort, and pagination.

- Inspect ticket details in a read-only view and manage attachments (upload additional files, download active files, and soft-remove existing attachments with a reason).
- **What "good" looks like to them:**
 - A clean, responsive interface adhering to the Zen Green Theme across mobile, tablet, and desktop viewports.
 - Form validation with clear inline errors and resilient form state preservation when API submission fails.
 - Strict data isolation preventing one requester from listing, viewing, or altering another requester's tickets and files.
 - Automated ticket numbering and predictable seed data for testing.
- **What I deliberately read as out of scope, and why:**
 - Real authentication, JWT tokens, and password management (deferred to Lab 3; simulated via Development Requester Selector).
 - IT Staff workflows (dashboard queues, ticket claiming, changing IT priority, reassignments).
 - Ticket lifecycle progression beyond the initial `New` state (e.g., In Progress, Resolved, Closed).
 - Collaboration mechanisms (Public Comments, Internal Notes, Service Actions Taken).

3. Scope

3.1 Included

#	Item	Notes
S-01	Development Requester selector	Dropdown of active requester users; stores the selection for the session.
S-02	Create Ticket	Form with required/optional fields, category and related-system dropdowns, attachments.
S-03	My Tickets	List of tickets belonging to the selected requester only.
S-04	Requester Ticket Detail	Read-only view of one ticket the requester owns.
S-05	Attachment lifecycle	Upload, download, soft removal with reason.
S-06	Search, filter, sort, pagination	Applied to the My Tickets list.
S-07	Ownership protection	Server-side enforcement that a requester can only read their own tickets.
S-08	Change Requester	A control in the application shell that switches the active requester and reloads all requester-scoped data.
S-09	Reference and seed data	Idempotent seed of categories, related systems and requester users (active and inactive).

3.2 Excluded

#	Item	Reason
X-01	Real login, auth tokens, passwords	Out of scope for this lab; replaced by the Development Requester selector.
X-02	IT Staff workflow and dashboard	Belongs to a later sprint.
X-03	Public comments, internal notes, actions taken	Staff-facing features.
X-04	Ticket status lifecycle beyond <code>New</code>	No transitions are implemented this sprint.
X-05	Hard deletion of attachments	Only soft removal is supported.

4. Functional Requirements

ID	Requirement	Description	Priority
FR-01	Create Ticket	A requester can submit a new ticket with summary, description, category, related system, priority and optional attachments. The backend persists it and returns the generated ticket number.	Must

ID	Requirement	Description	Priority
FR-02	Select Requester	The user picks a Development Requester from a dropdown of active users before using any other screen. The selection persists for the session and can be changed.	Must
FR-03	My Tickets List	The requester sees only their own tickets, with search, filter, sort and pagination.	Must
FR-04	Ticket Detail	The requester opens one of their tickets and sees all its fields read-only, plus its attachments.	Must
FR-05	Attachment Upload / Download / Soft Removal	The requester attaches files at creation and from the detail page, downloads active files, and soft-removes a file with a reason.	Must
FR-06	Change Requester	A "Change Requester" control is available in the header/shell on every screen. Switching the requester re-fetches all requester-scoped data (My Tickets, detail access) for the newly selected user.	Must
FR-07	Resilient Create form	If the backend fails while a ticket is being submitted, the form shows an error state and keeps every value the user has already entered so the submission can be retried.	Must
FR-08	Reference and seed data	Categories, related systems and requester users are seeded into the database by a script that can be run repeatedly without creating duplicates.	Must

5. Business Rules

ID	Rule
BR-01	The backend generates a unique Ticket Number for every ticket. The client never supplies or edits it.
BR-02	Every new ticket is created with status <code>New</code> . No other status is reachable in this sprint.
BR-03	The Development Requester selector is a testing mechanism only. It is not authentication and must be clearly labelled as such in the UI.
BR-04	Ownership: a requester may only read, list, or modify tickets and attachments belonging to the requester currently selected. Any other access is rejected by the server with <code>403</code> .
BR-05	Allowed attachment types: JPG, PNG, WEBP, PDF. Any other type is rejected with <code>415</code> .
BR-06	Maximum attachment size: 5 MB per file. Larger files are rejected with <code>413</code> .
BR-07	Maximum of 5 attachments per ticket, counting active files only.
BR-08	Attachments are soft-removed, never hard-deleted. A removal reason is required.
BR-09	A soft-removed attachment can no longer be downloaded; the server rejects requests for its download URL.
BR-10	A soft-removed attachment stays visible on the detail page, marked as removed, with its reason and removal timestamp.
BR-11	Only requesters with <code>isActive = true</code> are returned by the requester API and shown in the selector. Inactive requesters are never selectable.
BR-12	The seed script is idempotent: running it any number of times leaves exactly one row per seeded entity and never fails on re-run.
BR-13	Changing the active requester discards the previous requester's loaded data; no ticket of the previous requester may remain visible after the switch.

6. UI Specification Summary

Full details live in [ui-spec.md](#). Summary:

- **Theme:** Zen Green — primary `#006B3C`, secondary `#0B7A46`, pale `#EAF6EF`, page background `#F5F7F6`, surfaces `#FFFFFF`.
- **Layout:** fixed application header with the app name and the currently selected requester; content area on a neutral background with white cards.
- **Badges:** status badge (`New`) and priority badge on list rows and in the detail header.
- **Navigation:** Requester selection → My Tickets → Create Ticket / Ticket Detail. Any screen entered without a selected requester redirects to the selector.
- **Responsive:** table layout on desktop, stacked card list on mobile; breakpoints per the Responsive Breakpoints section of [ui-spec.md](#).

7. Data Changes

PostgreSQL via Prisma. New and changed models:

RequesterUser

Field	Type	Notes
id	String (uuid)	PK
name	String	Displayed in the selector
email	String	Unique
department	String?	Optional; shown next to the name in the selector
isActive	Boolean	Only active users appear in the selector
createdAt / updatedAt	DateTime	

Ticket

Field	Type	Notes
id	String (uuid)	PK
ticketNumber	String	Unique, server-generated (BR-01)
summary	String	Ticket Summary from the lab sheet
description	String	
status	Enum <code>TicketStatus</code>	<code>New</code> only this sprint (BR-02); default <code>New</code>
priority	Enum <code>TicketPriority</code>	<code>Low</code> / <code>Medium</code> / <code>High</code>
requesterId	String	FK → <code>RequesterUser.id</code>
categoryId	String	FK → <code>Category.id</code>
relatedSystemId	String?	FK → <code>RelatedSystem.id</code>
createdAt / updatedAt	DateTime	

Attachment

Field	Type	Notes
id	String (uuid)	PK
ticketId	String	FK → <code>Ticket.id</code>
fileName	String	Original file name

Field	Type	Notes
contentType	String	Validated against BR-05
sizeBytes	Int	Validated against BR-06
storagePath	String	Path of the stored file; <i>storage location decision TBD</i>
isRemoved	Boolean	Soft-removal flag (BR-08)
removedReason	String?	Required when <code>isRemoved</code> is true
removedAt	DateTime?	
uploadedAt	DateTime	

Category

Existing Lab 1 model, extended.

Field	Type	Notes
id	String (uuid)	PK
name	String	Unique
description	String?	Shown as helper text in the category dropdown
isActive	Boolean	Default <code>true</code> ; the API returns active categories only
createdAt / updatedAt	DateTime	

RelatedSystem

Field	Type	Notes
id	String (uuid)	PK
name	String	Unique
isActive	Boolean	Default <code>true</code> ; the API returns active systems only
createdAt / updatedAt	DateTime	

Seed data (idempotent)

Entity	Minimum required	Notes
Category	4 categories	Account and Access, Hardware, Software, Network
RelatedSystem	7 systems	Email, Campus Wi-Fi, VPN, LEB2 App, Grade Submission App, Printer, Corporate Laptop
RequesterUser (active)	4 users	Jennifer Anderson, Sarah Johnson, David Lee, Michael Brown — appear in the selector
RequesterUser (inactive)	1 user	Alex Smith (<code>isActive = false</code>) — must not appear in the selector (BR-11)

The seed uses upsert-by-unique-key so it can be re-run safely (BR-12). Implemented in `server/src/seed.ts` , with the data itself in `server/src/seedData.ts` .

Indexes

Index	Purpose
<code>Ticket.ticketNumber</code> (unique)	Enforces BR-01
<code>Ticket.(requesterId, createdAt)</code>	My Tickets listing — filters by owner and sorts by date in one index

Index	Purpose
<code>Ticket.categoryId</code> / <code>Ticket.relatedSystemId</code>	Filtering, and FK lookups
<code>Ticket.status</code> / <code>Ticket.createdAt</code>	Filtering and default sort
<code>Attachment.(ticketId, isRemoved)</code>	Splitting active from removed attachments on the detail page
<code>RequesterUser.isActive</code> , <code>Category.isActive</code> , <code>RelatedSystem.isActive</code>	Selector and dropdown queries return active rows only
<code>RequesterUser.email</code> (unique), <code>Category.name</code> (unique), <code>RelatedSystem.name</code> (unique)	Natural keys the idempotent seed upserts on (BR-12)

8. API Contract

The full wire contract — endpoint paths, methods, parameters, request and response shapes, validation rules, pagination metadata, ownership checks, error envelope, and status codes — lives in [api-spec.md](#). Summary of the surface:

Method	Path	Purpose
GET	<code>/api/requesters</code>	Active Development Requesters for the selector (BR-11)
GET	<code>/api/categories</code>	Active categories
GET	<code>/api/related-systems</code>	Active related systems
POST	<code>/api/tickets</code>	Create one validated ticket (FR-01)
GET	<code>/api/tickets</code>	The selected requester's tickets, with search/filter/sort/pagination (FR-03)
GET	<code>/api/tickets/:id</code>	One owned ticket with attachments (FR-04)
POST	<code>/api/tickets/:id/attachments</code>	Upload an attachment (FR-05)
GET	<code>/api/attachments/:id</code>	Attachment metadata, including removed ones (BR-10)
GET	<code>/api/attachments/:id/download</code>	Download an active attachment
PATCH	<code>/api/attachments/:id/remove</code>	Soft-remove an attachment with a reason (BR-08)

The selected requester is carried by an `X-Requester-Id` request header on every requester-scoped endpoint; ownership (BR-04) is enforced from that header on the server, never from a client-supplied body field.

Status code meanings

Code	Meaning in this API
200	Request succeeded
201	Resource created
400	Validation failure, missing requester header, or attachment limit reached
403	Ownership violation, unknown/inactive requester, or download of a removed attachment
404	Ticket or attachment not found
413	Attachment exceeds 5 MB
415	Unsupported attachment type
500	Unhandled server error — generic message only, details logged server-side

9. Acceptance Criteria

ID	Criterion
AC-01	Given a requester is selected and the Create Ticket form is filled with valid data, when the form is submitted, then the ticket is persisted with status <code>New</code> and the UI shows the generated Ticket Number.
AC-02	Given no requester has been selected, when any application screen is opened, then the user is redirected to the Development Requester selection screen; and given a requester is already selected, when the user switches to another requester via "Change Requester", then all requester-scoped data reloads for the new requester and none of the previous requester's data remains visible.
AC-03	Given a requester is selected, when they request a ticket or attachment that belongs to another requester, then the server responds <code>403</code> and the UI shows an access-denied message.
AC-04	Given a requester is selected, when My Tickets is opened, then only tickets owned by that requester are listed.
AC-05	Given a ticket was just created, when its detail page is opened, then all ticket fields are shown read-only.
AC-06	Given a file larger than 5 MB, when it is uploaded, then the server responds <code>413</code> and the UI shows a size error under the upload control.
AC-07	Given a file whose type is not JPG, PNG, WEBP or PDF, when it is uploaded, then the server responds <code>415</code> and the UI shows a type error.
AC-08	Given a ticket already has 5 active attachments, when a sixth is uploaded, then the upload is rejected and the UI explains the limit.
AC-09	Given an active attachment, when the requester removes it with a reason, then it is marked removed with reason and timestamp, remains visible as removed, and is no longer downloadable.
AC-10	Given the My Tickets list, when search, filter, sort or pagination is applied, then the list updates accordingly and stays scoped to the selected requester.
AC-11	Given the app is viewed at mobile, tablet and desktop widths, when each screen is inspected, then the layout matches the responsive rules in <code>ui-spec.md</code> with no clipping or horizontal overflow.
AC-12	Given the Create Ticket form has been filled in, when the backend fails during submit, then an error state is shown and every entered value is preserved so the user can retry without re-typing.
AC-13	Given a seeded database, when the seed script is run again, then it completes successfully and creates no duplicate rows.
AC-14	Given an inactive requester exists, when the selector is opened, then that requester is not listed.
AC-15	Given a search or filter that matches nothing, when it is applied to My Tickets, then a no-results state is shown, distinct from the empty state shown when the requester has no tickets at all.

10. Definition of Done

10.1 Product

- ☐ All functional requirements FR-01 ... FR-08 implemented.
- ☐ All acceptance criteria AC-01 ... AC-15 verified.
- ☐ All automated tests listed in `tests.md` pass on `main`.
- ☐ Responsive behaviour verified at the three breakpoints.
- ☐ No console errors and no unhandled server errors on the happy path.
- ☐ Zen Green Theme applied per `ui-spec.md`.
- ☐ Ownership rule enforced server-side, not only in the UI.

10.2 Course delivery

- ☐ Work delivered through a pull request from a feature branch.

- ☐ Pull request reviewed by an assigned reviewer.
- ☐ Evidence attached: screenshots, test run output, traceability matrix.
- ☐ Screenshot of the `docs/lab-02/` commit kept as evidence that the specification was written before any implementation code (Issue #1).
- ☐ Every Kanban issue (#1–#7) closed with its acceptance criteria met.
- ☐ All `docs/lab-02/` documents complete and committed: `specification.md`, `api-spec.md`, `ui-spec.md`, `tests.md`, `reviewer.md`, `ai-use.md`.
- ☐ `reviewer.md` records reviewer identity, PR links, comments given and received, responses, and approvals.

11. Assumptions and Decisions

ID	Assumption / Decision	Rationale
AD-01	Ticket Number format is generated as <code>TKT-YYYY-XXXXXX</code> (e.g., <code>TKT-2026-000001</code>), where <code>YYYY</code> is the current creation year and <code>XXXXXX</code> is a sequential zero-padded integer.	Human-readable, sortable, prevents collision, and strictly enforced by the backend per BR-01.
AD-02	The active Development Requester is maintained in client-side state/session storage and transmitted via custom header (<code>X-Requester-Id</code>) or query parameter on every API request.	Keeps backend services stateless while strictly enforcing server-side ownership checks before Lab 3 auth is introduced.
AD-03	Attachment files are stored in the local server filesystem under <code>uploads/attachments/</code> using unique UUID-prefixed filenames, while saving original names and metadata in PostgreSQL.	Simplifies MVP deployment, avoids filename collisions, and enables safe file streaming without direct public URL exposure.
AD-04	Default pagination for My Tickets is set to <code>10</code> tickets per page, with selectable options of 5, 10, and 25 per page.	Provides optimal balance between initial load performance and usable vertical scrolling across mobile and desktop.
AD-05	Default sort order for My Tickets is <code>createdAt</code> descending (newest tickets first).	Ensures users immediately see their most recently created support tickets at the top of the list.
AD-06	Soft removal retains the database record with <code>isRemoved = true</code> , <code>removedAt = NOW()</code> , and a non-empty <code>removedReason</code> , permanently revoking file download/streaming routes.	Complies with audit trail requirements while fulfilling BR-08 and BR-09.
AD-07	<code>Ticket</code> carries a composite index on (<code>requesterId</code> , <code>createdAt</code>) and <code>Attachment</code> a composite index on (<code>ticketId</code> , <code>isRemoved</code>), in addition to single-column indexes on <code>Ticket.categoryId</code> , <code>Ticket.relatedSystemId</code> , <code>Ticket.status</code> , and <code>Ticket.createdAt</code> .	My Tickets always filters by owner and sorts by date, so one composite index serves both halves of that query; the attachment index separates active from removed files without a table scan.
AD-08	Foreign keys use differentiated delete behaviour: <code>Ticket.requesterId</code> and <code>Ticket.categoryId</code> are <code>RESTRICT</code> , <code>Ticket.relatedSystemId</code> is nullable with <code>SET NULL</code> , and <code>Attachment.ticketId</code> is <code>CASCADE</code> .	A ticket must never lose its owner or category, a related system is optional because the requester may not know which system failed, and attachments have no meaning once their ticket is gone.
AD-09	Every write in the seed script is an <code>upsert</code> keyed on a unique column (<code>Category.name</code> , <code>RelatedSystem.name</code> , <code>RequesterUser.email</code>); the seed contains no <code>deleteMany</code> and no <code>create</code> .	Makes the seed idempotent per AC-13 — repeated runs converge on the same rows instead of duplicating data or destroying records other developers depend on.
AD-10	Reference data is retired with an <code>isActive</code> flag (indexed on <code>Category</code> , <code>RelatedSystem</code> , and <code>RequesterUser</code>) rather than by deleting rows, and the seed ships four active requesters plus one inactive requester.	Existing tickets keep pointing at valid rows after retirement, and the inactive requester makes the BR-11 rule "inactive requesters must not appear in the selector" testable from seed data alone.
AD-11	Local development runs PostgreSQL 16 from the repository <code>docker-compose.yml</code> , published on host port <code>5433</code> with a healthcheck and a named volume.	Gives every developer a reproducible database matching <code>server/.env.example</code> , while avoiding a port clash with a PostgreSQL already installed on the default 5432.

Appendix — GitHub Issue Traceability

Mapping between the Kanban issues and the requirements in this document.

Issue	Title	Covers
#1	Sprint Specifications, Architecture & Test Planning	This document, <code>api-spec.md</code> , <code>ui-spec.md</code> , <code>tests.md</code>
#2	Database Schema & Idempotent Seed Data	FR-08, BR-11, BR-12, §7 Data Changes and Seed data, AC-13
#3	Development Requester Context & Simulated Login	FR-02, FR-06, BR-03, BR-11, BR-13, AC-02, AC-14
#4	Ticket Creation API & UI with Validation	FR-01, FR-07, BR-01, BR-02, AC-01, AC-12
#5	My Tickets List with Filtering, Search & Pagination	FR-03, BR-04, AC-03, AC-04, AC-10, AC-15
#6	Requester Ticket Detail & Soft Attachment Lifecycle	FR-04, FR-05, BR-04 ... BR-10, AC-03, AC-05 ... AC-09
#7	UI Refinement, Responsive Layout & End-to-End Tests	AC-11, <code>ui-spec.md</code> checklist, §5 of <code>tests.md</code> , <code>ai-use.md</code>

Answer Part 3:

Test-Driven Development and Traceability · 10 points

Answer Part 3 — Automated Test Run Evidence

All three suites, run on 2026-09-05 with the stack up. 226 tests, 226 passing, nothing skipped.

Suite	Command	Files	Tests	Passed	Failed
Backend — unit and API	<code>npm run test:server</code>	6	130	130	0
Frontend — component	<code>npm run test:client</code>	6	80	80	0
End-to-end — Playwright	<code>npx playwright test</code>	4	16	16	0
Total		16	226	226	0

The verbatim output of each command is kept in `test-runs/` — `server.txt`, `client.txt` and `e2e.txt` are the redirected stdout of the three runs, with colour codes stripped and nothing else changed. The images below are those same files rendered by `screenshots/capture-test-runs.mjs`; the text in each image is the log file, character for character.

1. Backend — unit and API tests

```
TokTickIT - Backend - unit and API tests

$ git rev-parse --abbrev-ref HEAD
main
$ git log --oneline -1
9936b8c Merge pull request #33 from Chessuker/lab2-staging

$ npm run test:server

> toktickit@1.0.0 test:server
> npm --prefix server run test

> toktickit-server@1.0.0 test
> vitest run

RUN v3.2.7 D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server

✓ tests/ticketNumber.test.ts (8 tests) 15ms
✓ tests/seed.test.ts (9 tests) 9ms
✓ tests/requesters.api.test.ts (6 tests) 71ms
stderr | tests/app.test.ts > Express API Endpoints > GET /api/categories returns a safe 500 envelope when the database call fails
[Oxm5ibhgmto5u3p3] GET /api/categories: Error: connection refused
    at D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/tests/app.test.ts:100:63
    at file:///D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/node_modules/@vitest/runner/dist/chunk-hooks.js:155:11
    at file:///D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/node_modules/@vitest/runner/dist/chunk-hooks.js:752:26
    at file:///D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/node_modules/@vitest/runner/dist/chunk-hooks.js:1897:20
    at new Promise (<anonymous>)
    at runWithTimeout (file:///D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/node_modules/@vitest/runner/dist/chunk-hooks.js:1863:10)
    at runTest (file:///D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/node_modules/@vitest/runner/dist/chunk-hooks.js:1574:12)
    at processTicksAndRejections (node:internal/process/task_queues:104:5)
    at runSuite (file:///D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/node_modules/@vitest/runner/dist/chunk-hooks.js:1729:8)
    at runSuite (file:///D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/node_modules/@vitest/runner/dist/chunk-hooks.js:1729:8)

✓ tests/app.test.ts (7 tests) 380ms
✓ Express API Endpoints > POST /api/users creates a new user 306ms
stderr | tests/attachments.api.test.ts > GET /api/attachments/:id/download (API-11, AC-09, BR-09, BR-10) > answers 404 when the row exists
but the stored file is gone
[attachment-integrity] missing stored file d4e5f6a7-1111-4222-8333-444455556666 Error: ENOENT: no such file or directory, open
'C:\Users\Lenovo\AppData\Local\Temp\toktickit-uploads-CAYI95\never-written.pdf'
    at open (node:internal/fs/promises:637:25)
    at readFile (node:internal/fs/promises:1269:14)
    at D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/src/app.ts:593:17 {
  errno: -4058,
  code: 'ENOENT',
  syscall: 'open',
  path: 'C:\Users\Lenovo\AppData\Local\Temp\toktickit-uploads-CAYI95\never-written.pdf'
}

stderr | tests/tickets.api.test.ts > POST /api/tickets - failure handling (api-spec $1.3) > returns a safe 500 envelope when the insert fails
[4f989gxzmto5u3p3] POST /api/tickets: Error: relation "Ticket" does not exist
    at D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/tests/tickets.api.test.ts:281:58
    at file:///D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/node_modules/@vitest/runner/dist/chunk-hooks.js:155:11
    at file:///D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/node_modules/@vitest/runner/dist/chunk-hooks.js:752:26
    at file:///D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/node_modules/@vitest/runner/dist/chunk-hooks.js:1897:20
    at new Promise (<anonymous>)
    at runWithTimeout (file:///D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/node_modules/@vitest/runner/dist/chunk-hooks.js:1863:10)
    at runTest (file:///D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/node_modules/@vitest/runner/dist/chunk-hooks.js:1574:12)
    at processTicksAndRejections (node:internal/process/task_queues:104:5)
    at runSuite (file:///D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/node_modules/@vitest/runner/dist/chunk-hooks.js:1729:8)
    at runSuite (file:///D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/node_modules/@vitest/runner/dist/chunk-hooks.js:1729:8)

stderr | tests/tickets.api.test.ts > GET /api/tickets - search, filter, sort, pagination (API-06, AC-10) > returns a safe 500 envelope when
the query fails
[ybcaadb2omto5u3xz] GET /api/tickets: Error: relation "Ticket" does not exist
    at D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/tests/tickets.api.test.ts:558:7
    at file:///D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/node_modules/@vitest/runner/dist/chunk-hooks.js:155:11
    at file:///D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/node_modules/@vitest/runner/dist/chunk-hooks.js:752:26
    at file:///D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/node_modules/@vitest/runner/dist/chunk-hooks.js:1897:20
    at new Promise (<anonymous>)
    at runWithTimeout (file:///D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/node_modules/@vitest/runner/dist/chunk-hooks.js:1863:10)
    at runTest (file:///D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/node_modules/@vitest/runner/dist/chunk-hooks.js:1574:12)
    at processTicksAndRejections (node:internal/process/task_queues:104:5)
    at runSuite (file:///D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/node_modules/@vitest/runner/dist/chunk-hooks.js:1729:8)
    at runSuite (file:///D:/OpalFolder/RMUTT/3-1/CPE334/TikTokIT/server/node_modules/@vitest/runner/dist/chunk-hooks.js:1729:8)

✓ tests/tickets.api.test.ts (64 tests) 731ms
✓ POST /api/tickets - valid create (API-01, AC-01) > returns 201 with the created ticket and its ticket number 353ms
✓ tests/attachments.api.test.ts (36 tests) 734ms

Test Files 6 passed (6)
Tests 130 passed (130)
Start at 16:08:04
Duration 4.85s (transform 1.06s, setup 0ms, collect 9.92s, tests 1.94s, environment 1ms, prepare 5.86s)
```

Six files: ticket creation and listing, the attachment lifecycle, the requester directory, the seed, and the ticket-number generator. The suite needs no database — `src/db.js` is mocked — but `attachments.api.test.ts` deliberately does not mock the file system, writing and reading real bytes in a temporary directory.

2. Frontend — component tests

```
TokTickIT - Frontend - component tests

$ git rev-parse --abbrev-ref HEAD
main
$ git log --oneline -1
9936b8c Merge pull request #33 from Chessuker/lab2-staging

$ npm run test:client

> toktickit@1.0.0 test:client
> npm --prefix client run test

> client@0.0.0 test
> vitest run

RUN v4.1.10 D:/OpalFolder/KMUTT/3-1/CPE334/TikTokIT/client

Not implemented: navigation to another Document

Test Files  6 passed (6)
Tests      80 passed (80)
Start at   16:08:22
Duration   50.18s (transform 1.97s, setup 32.82s, import 16.25s, tests 23.38s, environment 168.15s)
```

Six files covering the selector and its guard, the create form and its validation, the list with its search, filter and two zero-result states, the read-only detail screen with its removal modal, and the app shell's Change Requester behaviour.

3. End-to-end — Playwright

```
TokTickIT - End-to-end - Playwright

$ git rev-parse --abbrev-ref HEAD
main
$ git log --oneline -1
9936b8c Merge pull request #33 from Chessuker/lab2-staging

$ npx playwright test

Running 16 tests using 1 worker

  ok 1 [chromium] › e2e\attachments.spec.ts:18:7 › Attachment lifecycle (E2E-02) › upload, download, remove with a reason, then download is refused (5.0s)
  ok 2 [chromium] › e2e\attachments.spec.ts:58:7 › Attachment lifecycle (E2E-02) › a removed attachment is refused by the API as well as hidden in the UI (2.0s)
  ok 3 [chromium] › e2e\attachments.spec.ts:109:7 › Attachment lifecycle (E2E-02) › rejects a disallowed file type without attaching it (AC-07) (1.6s)
  ok 4 [chromium] › e2e\create-ticket.spec.ts:19:7 › Create ticket (E2E-01) › create a ticket, then open it and find the same data (1.8s)
  ok 5 [chromium] › e2e\create-ticket.spec.ts:58:7 › Create ticket (E2E-01) › the new ticket appears in My Tickets under its number (1.6s)
  ok 6 [chromium] › e2e\create-ticket.spec.ts:80:7 › Create ticket (E2E-01) › an invalid form is refused with errors under the offending fields (1.4s)
  ok 7 [chromium] › e2e\ownership.spec.ts:19:7 › Ownership guard (E2E-03) › opening another requester's ticket shows access denied (2.0s)
  ok 8 [chromium] › e2e\ownership.spec.ts:35:7 › Ownership guard (E2E-03) › another requester cannot download the owner's attachment (1.8s)
  ok 9 [chromium] › e2e\ownership.spec.ts:75:7 › Ownership guard (E2E-03) › the requester still sees their own ticket after the switch back (3.0s)
  ok 10 [chromium] › e2e\requester-context.spec.ts:16:9 › Requester guard (E2E-04, AC-02) › opening /tickets with no requester redirects to the selector (813ms)
  ok 11 [chromium] › e2e\requester-context.spec.ts:16:9 › Requester guard (E2E-04, AC-02) › opening /tickets/new with no requester redirects to the selector (766ms)
  ok 12 [chromium] › e2e\requester-context.spec.ts:16:9 › Requester guard (E2E-04, AC-02) › opening /system with no requester redirects to the selector (791ms)
  ok 13 [chromium] › e2e\requester-context.spec.ts:24:7 › Requester guard (E2E-04, AC-02) › the selector says plainly that it is not a real login (759ms)
  ok 14 [chromium] › e2e\requester-context.spec.ts:32:7 › Requester guard (E2E-04, AC-02) › the selection survives a reload (1.3s)
  ok 15 [chromium] › e2e\requester-context.spec.ts:43:7 › Change Requester (E2E-04, FR-06, BR-13) › switching requester swaps the visible ticket set (3.0s)
  ok 16 [chromium] › e2e\requester-context.spec.ts:79:7 › Change Requester (E2E-04, FR-06, BR-13) › the header shows the selected requester on every screen (2.3s)

16 passed (35.2s)
```

Four specs, one per planned journey:

Spec	Test ID	Journey
create-ticket.spec.ts	E2E-01	Select requester → fill form → submit → read the ticket number → open the detail screen and find the same data
attachments.spec.ts	E2E-02	Upload → download → remove with a reason → the file is no longer downloadable, by the UI or by direct request
ownership.spec.ts	E2E-03	Requester B opens Requester A's ticket URL and gets access denied, at the screen and at both endpoints
requester-context.spec.ts	E2E-04	The guard redirects every route when no requester is selected; switching requester swaps the visible ticket set

These run against the real stack, so they need it up first:

```
docker compose up -d && npm run prisma:migrate && npm run prisma:seed && npm run test:e2e
```

`playwright.config.ts` starts the API and Vite itself, reusing them if they are already running, but never the database — an E2E run that quietly created its own empty schema would pass against nothing.

Reproducing this evidence

```
npm run test:server > docs/lab-02/test-runs/server.txt 2>&1
```

```
npm run test:client > docs/lab-02/test-runs/client.txt 2>&1
```

```
npx playwright test > docs/lab-02/test-runs/e2e.txt 2>&1
```

```
node docs/lab-02/screenshots/capture-test-runs.mjs
```

Lab 2 — Test Plan and Results

Companion to [specification.md](#). Acceptance criteria referenced here (AC-01 ...) are defined in section 9 of that document; the endpoints under test are specified in [api-spec.md](#).

1. Test Strategy

Level	Tooling	What it covers
Unit	Vitest	Pure logic: ticket-number generation, attachment validation (type, size, count), query/pagination helpers.
Integration / API	Vitest + Supertest	Express routes end-to-end against the app, including status codes, validation, and ownership enforcement.
UI Component	Vitest + React Testing Library	Rendering, form validation, guard/redirect behaviour, list and detail states.
End-to-End	Playwright	Full user journeys across real screens: select requester → create ticket → view list → open detail → manage attachments.

Unit and component tests run without a live database: the API suite mocks `src/db.js`, and the seed suite drives `seed()` against an in-memory client that implements `upsert` only — so a seed that reached for `create` or `deleteMany` would fail the suite instead of silently duplicating rows. Integration against real PostgreSQL is verified manually with the migrate/seed commands in section 5.

2. Planned Tests

Test ID	Type	Requirement / AC	What It Tests	Expected Result	Automated Test File Path	Final Status
API-01	API	AC-01	Create a ticket with valid data	201 Created ; row persisted; ticket number returned	<code>server/tests/tickets.api.test.ts</code>	Pass
API-02	API	AC-01, BR-02	New ticket status	Status is <code>New</code>	<code>server/tests/tickets.api.test.ts</code>	Pass
API-03	API	AC-01	Create with missing required fields	400 with field errors	<code>server/tests/tickets.api.test.ts</code>	Pass
API-04	API	AC-04	List tickets for a requester	Only that requester's tickets returned	<code>server/tests/tickets.api.test.ts</code>	Pass
API-05	API	AC-03, BR-04	Read another requester's ticket	403 Forbidden	<code>server/tests/tickets.api.test.ts</code>	Pass

Test ID	Type	Requirement / AC	What It Tests	Expected Result	Automated Test File Path	Final Status
API-06	API	AC-10	search , category , status , sort , page , pageSize	Correct subset, order and page metadata	server/tests/tickets.api.test.ts	Pass
API-12	API	AC-14, BR-11	GET /api/requesters	Active requesters only; inactive one absent	server/tests/requesters.api.test.ts	Pass
API-13	API	AC-15	Filter that matches nothing	200 with an empty page and correct total	server/tests/tickets.api.test.ts	Pass
API-07	API	AC-06, BR-06	Upload a file larger than 5 MB	413 Payload Too Large	server/tests/attachments.api.test.ts	Pass
API-08	API	AC-07, BR-05	Upload a disallowed file type	415 Unsupported Media Type	server/tests/attachments.api.test.ts	Pass
API-09	API	AC-08, BR-07	Upload a 6th active attachment	Rejected with the limit error	server/tests/attachments.api.test.ts	Pass
API-10	API	AC-09, BR-08	Soft-remove an attachment with a reason	200 ; isRemoved , reason and timestamp set	server/tests/attachments.api.test.ts	Pass
API-11	API	AC-09, BR-09	Download a removed attachment	Rejected; file not served	server/tests/attachments.api.test.ts	Pass
API-14	API	AC-09	Soft-remove without a reason	400 ; attachment unchanged	server/tests/attachments.api.test.ts	Pass
UNIT-01	Unit	BR-01, AD-01	Ticket number generator	Matches TKT-YYYY-XXXXXX , restarts per year, never repeats	server/tests/ticketNumber.test.ts	Pass
UNIT-02	Unit	AC-13, BR-12	Seed run repeatedly	Later runs succeed; row counts unchanged; every write is an upsert on a unique key	server/tests/seed.test.ts	Pass
UNIT-03	Unit	FR-08	Seed content	4 categories, ≥6 related systems, ≥4 active and ≥1 inactive requester	server/tests/seed.test.ts	Pass
UI-01	UI	AC-02	App opened with no requester selected	Redirect to the selector screen	client/src/components/RequesterSelector.test.tsx	Pass

Test ID	Type	Requirement / AC	What It Tests	Expected Result	Automated Test File Path	Final Status
UI-02	UI	AC-14, BR-03	Selector content	Only active users listed; the "not a real login" notice is shown	client/src/components/RequesterSelector.test.tsx	Pass
UI-03	UI	AC-01	Create Ticket form validation	Errors render under the offending fields; submit blocked	client/src/components/CreateTicketForm.test.tsx	Pass
UI-04	UI	AC-04, AC-10	My Tickets list	Rows render; search and filter update the list	client/src/components/MyTickets.test.tsx	Pass
UI-05	UI	AC-05	Ticket detail is read-only	No editable inputs rendered	client/src/components/TicketDetail.test.tsx	Pass
UI-06	UI	AC-09	Removal modal	Confirm disabled until a reason is entered	client/src/components/TicketDetail.test.tsx	Pass
UI-07	UI	AC-02, FR-06, BR-13	Change Requester	Switching requester re-fetches and shows only the new requester's tickets	client/src/components/AppShell.test.tsx	Pass
UI-08	UI	AC-12, FR-07	Create form on backend failure	Error callout shown; entered values still present in the form	client/src/components/CreateTicketForm.test.tsx	Pass
UI-09	UI	AC-15	Empty vs. no-results state	Correct distinct state rendered in each case	client/src/components/MyTickets.test.tsx	Pass
E2E-01	E2E	AC-01, AC-05	Full create flow	Ticket number shown, detail page opens with the same data	e2e/create-ticket.spec.ts	Pass
E2E-02	E2E	AC-09	Attachment lifecycle	Upload, download, soft-remove with reason; removed file no longer downloadable	e2e/attachments.spec.ts	Pass
E2E-03	E2E	AC-03	Ownership guard	Opening another requester's ticket shows access denied	e2e/ownership.spec.ts	Pass

Test ID	Type	Requirement / AC	What It Tests	Expected Result	Automated Test File Path	Final Status
E2E-04	E2E	AC-02	Select → change requester	Guard forces selection; switching requester swaps the visible ticket set	e2e/requester-context.spec.ts	Pass

3. Acceptance-Criterion Traceability Matrix

AC	Description (short)	Covered by
AC-01	Create ticket, number returned	API-01, API-02, API-03, UI-03, E2E-01
AC-02	Guard when no requester selected + reload on change	UI-01, UI-07, E2E-04
AC-03	Ownership violation rejected	API-05, E2E-03
AC-04	List scoped to the requester	API-04, UI-04
AC-05	Detail is read-only	UI-05, E2E-01
AC-06	Oversized upload rejected	API-07
AC-07	Wrong file type rejected	API-08
AC-08	Attachment count limit	API-09
AC-09	Soft removal with reason	API-10, API-11, API-14, UI-06, E2E-02
AC-10	Search / filter / sort / pagination	API-06, UI-04
AC-11	Responsive layout	Section 4 checklist
AC-12	Form values preserved on backend failure	UI-08
AC-13	Seed is idempotent	UNIT-02
AC-14	Inactive requesters hidden	API-12, UI-02
AC-15	No-results state distinct from empty state	API-13, UI-09

4. Responsive and Visual Checklist

#	Screen	Viewport	Result	Screenshot
R-01	Requester Selection	Desktop ≥ 992 px	Pass	docs/lab-02/screenshots/selector-desktop.png
R-02	Requester Selection	Mobile < 768 px	Pass	docs/lab-02/screenshots/selector-mobile.png
R-03	Create Ticket	Desktop	Pass	docs/lab-02/screenshots/create-desktop.png
R-04	Create Ticket	Tablet 768–991 px	Pass	docs/lab-02/screenshots/create-tablet.png
R-05	Create Ticket	Mobile	Pass	docs/lab-02/screenshots/create-mobile.png
R-06	My Tickets (table)	Desktop	Pass	docs/lab-02/screenshots/list-desktop.png
R-06b	My Tickets (table)	Tablet 768–991 px	Pass	docs/lab-02/screenshots/list-tablet.png
R-07	My Tickets (cards)	Mobile	Pass	docs/lab-02/screenshots/list-mobile.png
R-08	Ticket Detail	Desktop	Pass	docs/lab-02/screenshots/detail-desktop.png

#	Screen	Viewport	Result	Screenshot
R-08b	Ticket Detail	Tablet 768–991 px	Pass	docs/lab-02/screenshots/detail-tablet.png
R-09	Ticket Detail	Mobile	Pass	docs/lab-02/screenshots/detail-mobile.png
R-10	Removal modal	Mobile	Pass	docs/lab-02/screenshots/remove-modal-mobile.png
R-11	Header / Change Requester	Mobile	Pass	docs/lab-02/screenshots/header-mobile.png
R-12	My Tickets no-results state	Desktop	Pass	docs/lab-02/screenshots/list-no-results.png

Each row is checked against the Visual Inspection Checklist in [ui-spec.md](#).

The six Create Ticket state screenshots required for Answer Part 6 (Issue #4) are collected in [answer-part6-screenshots.md](#), captured against the running stack and a seeded database.

5. Test Commands and Final Results

```
npm run test:server
```

```
npm run test:client
```

```
npm run test:e2e
```

Migration and seed (Issue #2, run against a live PostgreSQL)

```
npm --prefix server run prisma:migrate
```

```
npm --prefix server run prisma:seed
```

Verified on 2026-08-22 against PostgreSQL 16.15 (Docker, `docker compose up -d`). The seed was run **three** times in a row; every run succeeded and the row counts were identical each time:

Entity	Rows after run 1	after run 2	after run 3
Category	4	4	4
RelatedSystem	7	7	7
RequesterUser (active)	4	4	4
RequesterUser (inactive)	1	1	1
User (Lab 1 admin)	1	1	1

Both migrations applied cleanly (`20260808160909_init` , `20260822000000_lab02_requester_ticketing`), and the resulting `Ticket` table carries all six planned indexes plus the three foreign keys with the intended `Restrict / SetNull / Cascade` behaviour. AC-13 met.

Issue #3 verification (Development Requester context)

`GET /api/requesters` is covered by API-12 against a mocked Prisma client, so it runs without a database. The three Issue #3 screens were additionally exercised by hand in the browser at 1280x720 and 375x812: the guard redirect, the amber "not a real login" callout, the active-only dropdown, the disabled Continue button, the loading and error/retry states, the header requester name, the mobile compact menu, and a Jennifer Anderson to Sarah Johnson switch that replaced the visible requester-scoped

content. Docker was unavailable that day, so the browser pass ran against a stub of `GET /api/requesters` rather than PostgreSQL; the endpoint itself is only proven by API-12 until the next live migrate/seed run.

Issue #5 verification (My Tickets list)

`GET /api/tickets` is covered by API-04, API-06 and API-13 against a mocked Prisma client, so the suite still runs without a database. The endpoint and the screen were then exercised against the **live stack** on 2026-08-29 — PostgreSQL 16 in Docker, `prisma migrate deploy + prisma:seed`, the Express API on :5000 and Vite on :5173 — with 14 tickets created for Jennifer Anderson and 3 for Sarah Johnson through the real `POST /api/tickets`, and Michael Brown left with none so the empty state had a genuine subject.

Confirmed live: ownership isolation (Jennifer's 14 vs. Sarah's 3, and a `requesterId` in the query string ignored), case-insensitive search on ticket number and summary, Category and Status filters, `priority:desc` ordering High → Medium → Low, page metadata across both pages, `200` with `totalItems: 0` for a filter matching nothing, and `400 VALIDATION_FAILED` naming `pageSize` for an out-of-range page size.

Responsive check (V-03), measured rather than eyeballed — `document.body.scrollWidth` against the viewport width on the loaded list:

Viewport	Layout rendered	Body scroll width	Horizontal overflow
375 px	Card list	375 px	0
820 px	Table (scrolls in its own container)	820 px	0
1280 px	Table, all ten columns visible	1280 px	0

The Answer Part 7 screenshots are collected in [answer-part7-screenshots.md](#) and were captured by `docs/Lab-02/screenshots/capture-issue5.mjs` against that same live stack.

Issue #6 verification (Ticket Detail and attachment lifecycle)

The seven attachment and ownership API tests run against a mocked Prisma client, so the suite still needs no database — but the file system is deliberately *not* mocked. `server/tests/attachments.api.test.ts` points `UPLOAD_DIR` at a temporary folder and lets the route write and read real bytes, because "the file reaches disk and comes back" is exactly what the download and soft-removal rules are about. The suite also asserts that a refused upload leaves the directory untouched, and that a soft-removed file is still on disk after the removal.

The stack was then exercised **live** on 2026-09-04 — PostgreSQL 16 on :5433, the Express API on :5000 and Vite on :5173, seeded. `npm run test:e2e` drives the real screens through Playwright and passed 6/6:

E2E test	What it proves
E2E-02 upload → download → remove → refused	The whole lifecycle on one ticket, including a real file download whose bytes match what was uploaded
E2E-02 API-level refusal	After removal, <code>GET /api/attachments/:id/download</code> answers <code>403</code> while <code>GET /api/attachments/:id</code> still answers <code>200</code> with the reason and timestamp
E2E-02 disallowed type	A <code>.txt</code> upload is rejected and never appears in the list
E2E-03 access denied	Requester B opening Requester A's ticket URL gets the access-denied screen with no ticket data on the page
E2E-03 attachment ownership	Requester B gets <code>403</code> from both <code>GET /api/tickets/:id</code> and <code>GET /api/attachments/:id/download</code> , with no <code>Content-Disposition</code>
E2E-03 round trip	Switching back to the owner restores access to the same URL

Peer review on the Issue #6 PR caught a real ordering bug: a file that was both oversized **and** of a disallowed type answered `413` instead of the more specific `415`. The check order in the route was right, but `multer`'s `limits.fileSize` aborted the request before the route could look at the content at all. The fix replaces that limit with a storage engine that retains the first `5 MB + 1` bytes and counts the rest, so the type check always has the signature and the size check always has the true byte count, with

memory still bounded. Six regression tests now pin the order, and the combinations were re-checked over real HTTP against the running API:

Request	Response
6 MB .txt , no signature	415 UNSUPPORTED_MEDIA_TYPE
6 MB valid PNG	413 FILE_TOO_LARGE
50 MB valid PNG	413 FILE_TOO_LARGE
Small .txt	415 UNSUPPORTED_MEDIA_TYPE
6 MB PNG into another requester's ticket	403 FORBIDDEN
Small valid PNG	201 Created

The refused upload also drains the request body before answering. Without that the socket is reset while the client is still sending, and a large refused file surfaces as a network error rather than as the 403 or 404 it should be.

Confirmed live in addition: the type check reads the file's leading bytes rather than its name or Content-Type , so an executable renamed to .pdf is refused with 415 ; stored files are written under a generated uuid name, so a file picked as ../../etc/passwd.png cannot escape the upload directory; and storagePath never appears in any response body.

Responsive check (V-03) on the detail screen, measured rather than eyeballed — document.body.scrollWidth against the viewport width:

Viewport	Layout rendered	Body scroll width	Horizontal overflow
375 px	Single-column fields, stacked attachment actions	375 px	0
1280 px	Four-column field grid	1280 px	0

The Answer Part 8 screenshots are collected in answer-part8-screenshots.md and were captured by docs/Lab-02/screenshots/capture-issue6.mjs against that same live stack.

Issue #7 verification (UI refinement, responsive layout, end-to-end)

The two remaining end-to-end journeys were written and the suite now covers all four: create-ticket.spec.ts (E2E-01) and requester-context.spec.ts (E2E-04) join the attachment and ownership specs from Issue #6. Sixteen tests, all passing against the live stack.

E2E-01 threads the server-generated ticket number through three screens: it is read off the confirmation, then found again on the detail screen beside the same summary, description, category and related system, and finally searched for in My Tickets. E2E-04 covers both halves of AC-02 — every application route redirects to the selector when no requester is chosen, and a requester switch genuinely replaces the visible data rather than leaving the previous requester's rows on screen.

The UI pass was measured, not eyeballed. A script walked all three screens at 1280 px, 820 px and 375 px and read the page rather than looking at it: body scroll width against viewport width, every leaf element's scrollWidth against its clientWidth , every form control's labelling, every button's height, and the computed contrast ratio of each text-on-background pair. The results are in the V-01 ... V-10 table in ui-spec.md. What it caught:

Finding	Fix
Mobile controls between 28 px and 38 px tall — above the 24 px WCAG 2.2 floor but below what a thumb hits reliably	44 px minimum on buttons, page numbers and the header toggle below 768 px, and the tap-target TBD in ui-spec §4 resolved at that number
The Create Ticket form ran one field per row at tablet width, so Category, Related System and Priority each claimed a full 800 px line	Two-column grid from 768 px, with Summary, Description and the upload area opting out via zg-field-wide
The header wrapped to two lines at 820 px, doubling its height	Nav and requester block pinned to one line; the small "Requester" caption is dropped at that width since the name below already says it

Finding	Fix
At tablet width the ticket number ran underneath the Created Date beside it	<code>.zg-table</code> minimum width raised from 56 rem to 66 rem — the page container's width — and the Ticket No. column widened from 12.5% to 13.5%
Pagination pushed Next onto a line of its own at 375 px	Previous and Next share the first row; the numbered pages drop below them

Nothing was found for V-03 (no horizontal overflow at any of the nine screen-and-width combinations), V-06 (no unlabelled control anywhere) or V-08 (every field error below its own control, in `#DC2626`, with a matching border). The lowest contrast ratio measured anywhere is 4.83:1, on muted dates and the italic *Unassigned* placeholder, which clears the 4.5:1 AA requirement; the status and priority badges range from 6.84:1 to 8.21:1.

The nine Answer Part 9 screenshots are collected in [answer-part9-screenshots.md](#) and were captured by `docs/lab-02/screenshots/capture-issue7.mjs`, which prints the measured body width beside every capture.

Final run on `main`

Run on `main` at commit `9936b8c` (the merge of `#33`, which brought `lab2-staging` into `main`), on 2026-09-05. Each log in [test-runs/](#) opens with `git rev-parse --abbrev-ref HEAD` and `git log --oneline -1`, so the branch and commit the suite ran against are part of the evidence rather than a claim beside it.

Suite	Command	Files	Tests	Passed	Failed	Date
Backend (unit + API)	<code>npm run test:server</code>	6	130	130	0	2026-09-05
Frontend (component)	<code>npm run test:client</code>	6	80	80	0	2026-09-05
End-to-end	<code>npm run test:e2e</code>	4	16	16	0	2026-09-05
Total		16	226	226	0	2026-09-05

The raw output of all three commands is kept verbatim in [test-runs/](#) and rendered for Answer Part 3 as `docs/lab-02/screenshots/part3-test-server.png`, `part3-test-client.png` and `part3-test-e2e.png`.

The end-to-end suite needs the stack up: `docker compose up -d`, `npm run prisma:migrate`, `npm run prisma:seed`. `playwright.config.ts` starts the API and Vite itself (reusing them if they are already running) but never the database, so a run cannot silently pass against an empty schema.

6. Issue Coverage

Issue	Covered by tests
#2 Database Schema & Idempotent Seed	UNIT-02, UNIT-03
#3 Requester Context & Simulated Login	API-12, UI-01, UI-02, UI-07, E2E-04
#4 Ticket Creation API & UI	UNIT-01, API-01, API-02, API-03, UI-03, UI-08, E2E-01
#5 My Tickets List	API-04, API-05, API-06, API-13, UI-04, UI-09, E2E-03
#6 Ticket Detail & Attachment Lifecycle	API-05, API-07 ... API-11, API-14, UI-05, UI-06, E2E-02
#7 UI Refinement, Responsive & E2E	Section 4 checklist, E2E-01 ... E2E-04

Answer Part 4:

AI Use with Reflection · 5 points

Lab 2 — AI Use and Reflection

Answer Part 4 deliverable for CPE334 Lab 2.

1. Environment and LLM Info

Field	Detail
Agent / Tool	Claude Code
LLM used	Claude Opus 5, NotebookLM + Gemini Flash 3.7
Thinking level	High for specification, planning and debugging; Medium for implementation
Student name	Thawat Boonsuk
Student ID	67070501024

2. Key Prompts

Ten prompts that materially shaped the sprint, grouped by phase. The Issue column refers to the GitHub Kanban issues #1–#7.

#	Phase	Issue	Prompt (summary or verbatim)	AI Output	How I Used / Adjusted It
P-01	Spec Generation	#1	Turn the lab sheet's requester ticketing brief into a written specification: functional requirements, business rules and acceptance criteria, each one numbered so tests can cite it	<code>specification.md</code> with FR-01...FR-08, BR-01...BR-13 and AC-01...AC-15	Kept the numbering scheme, which every later document and test name references. Cut the status-lifecycle requirements: only <code>New</code> is reachable this sprint, so requirements about later statuses would have been specification for code nobody was writing
P-02	Spec Generation	#1	Write the API contract from the specification: every endpoint, its status codes, and one error envelope shared by all of them	<code>api-spec.md</code> §1–§6, including the ordered validation table for uploads	Fixed the error shape before any route existed, which is what made "the code disagrees with the spec" a reviewable claim later. Left several open decisions as explicit TBDs rather than guessing
P-03	Test Creation	#1	Plan the tests before the code: one row per test with its ID, the AC it covers, the file it will live in, and its expected result	<code>tests.md</code> §2 planned-test table and the AC traceability matrix	This became the sprint's checklist. Every issue since has been "make these rows say Pass". The traceability matrix caught that AC-15 had no API-level test, so API-13 was added
P-04	Implementation	#2	Design the Prisma schema for tickets, attachments and the requester directory, with a seed that can run repeatedly without duplicating rows	<code>schema.prisma</code> plus a seed built entirely from <code>upsert</code>	Kept the soft-removal columns on <code>Attachment</code> from the start, which is why Issue #6 needed no migration. Made the seed test assert that only <code>upsert</code> is ever called, so a future <code>create</code> fails the suite instead of silently duplicating

#	Phase	Issue	Prompt (summary or verbatim)	AI Output	How I Used / Adjusted It
P-05	Implementation	#4	Build the create-ticket endpoint and form, with the ticket number generated server-side and the form keeping the user's input when the backend fails	<code>POST /api/tickets</code> with a transactional number generator, and a form that preserves state on failure	Asked for the collision retry after seeing the generator read-then-write outside a transaction. The "keep the values on failure" behaviour is AC-12 and it needed to be asked for explicitly — the first version cleared the form
P-06	Implementation	#5	Build the My Tickets list where search, filter, sort and pagination are all server-side query parameters, with the empty state and the no-results state as separate screens	<code>GET /api/tickets</code> with the ownership clause written from the header, and a list component that re-fetches on every control change	The two zero-result states were the part worth insisting on: an early version showed one message for both, which loses the distinction AC-15 exists to make
P-07	Implementation	#6	Build the read-only ticket detail screen and the attachment lifecycle: upload, download, soft removal with a mandatory reason, and ownership protection	Five endpoints, the detail screen, and 37 new tests	Rejected rendering the read-only fields as disabled inputs. A greyed-out input still reads as "editable, but not now", which is the wrong promise on a screen where nothing is ever editable, so the fields became a definition list
P-08	Debugging / Refactoring	#6	My reviewer says an oversized file of a disallowed type returns 413 when the spec says 415 — fix the check order	Diagnosis that the ordering in the route was already correct and the real cause was <code>multer's limits.fileSize</code> aborting the request before anything could read the content	This is the prompt I learned the most from. The obvious fix — swapping two <code>if</code> blocks — would have changed nothing. The fix was a storage engine that keeps the first 5 MB + 1 bytes and counts the rest, so both checks have what they need
P-09	Debugging / Refactoring	#7	Check the UI against the Zen Green spec at all three breakpoints by measuring the page, not by looking at screenshots	A script reading body scroll width, element clipping, control labelling, button heights and computed contrast ratios at 1280, 820 and 375 px	Measuring instead of eyeballing found five real problems I would not have noticed by scrolling through screenshots, including a table whose ticket numbers ran under the date column at tablet width
P-10	Completion Review	#7	Write the two missing end-to-end journeys and get all four suites green on <code>main</code>	<code>create-ticket.spec.ts</code> and <code>requester-context.spec.ts</code> , and the final 226-test run	Asked for the create flow to thread the server-generated ticket number through three screens rather than asserting each screen separately. If the number matches on all three, the ticket was really persisted

3. My Reflection

What worked well

Writing the specification first turned out to be the thing that made the AI useful rather than merely fast. Because `api-spec.md` fixed every status code and the exact order the upload validations run in, I could point at a specific line and say the code disagrees with it. Without that document the peer review comment about 413 versus 415 would have been an opinion; with it, it was a defect with an address.

The other thing that worked was making the AI produce evidence instead of claims. Asking it to measure the page — body scroll width against viewport width, every element's scroll width against its client width, computed contrast ratios — gave me numbers I could put in the checklist. "Looks fine on mobile" is not a test result. 375 px body width in a 375 px viewport is.

Where the AI got it wrong

The upload bug is the clearest case. My reviewer found that a file that was both too large and the wrong type came back as 413 when the spec says the more specific 415 wins. The route's check order had been correct since the day it was written. What

nobody noticed, me included, was that `multer` was configured with a size limit that killed the request before the route ever saw the file's contents. The AI wrote that configuration, wrote a comment explaining why it was correct, and wrote tests that passed — because each test broke only one rule at a time. It took a human reading the spec to find the combination nobody had tested.

Smaller version of the same thing: the first pass at the read-only detail screen rendered the ticket fields as disabled inputs. It satisfies "not editable" and it fails what AC-05 is actually about.

How Spec-Driven Development changed the way I used AI

It moved the argument earlier. Instead of reviewing generated code and asking whether it looked right, I was reviewing a specification and asking whether it said the right thing — and once it did, the code either matched or it did not. The numbered acceptance criteria are what made this work in practice: every test file, every commit message and every PR description cites AC or BR numbers, so it is always obvious which requirement a change is supposed to satisfy and which test would notice if it stopped.

It also changed what I ask for. I stopped asking for features and started asking for the failing case: not "add file upload" but "reject a 6 MB PDF with 413 and a renamed executable with 415". The second prompt produces code and tests that are about the same thing.

What I would do differently next time

Test the combinations, not just the cases. Every individual rule in the upload path had a test and every one of them passed. The bug lived in the interaction between two rules, which is exactly where the test plan had no row. Next sprint I would add a line to the test plan for any endpoint with more than one validation: at least one test that breaks two rules at once and asserts which error wins.

I would also run the responsive measurements earlier than the last issue. Three of the five things that pass found — the tablet form, the header wrapping, the table's column widths — had been wrong since the screen was first built, and fixing them at the end meant re-capturing every screenshot.

Answer Part 5:

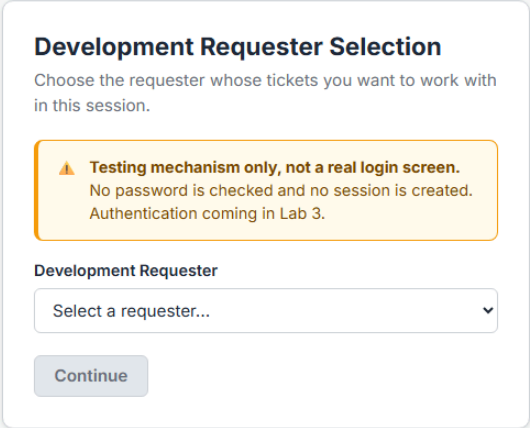
Development Requester Select Screen · graded within Part 6

Answer Part 5 — Development Requester Selection Screen

The simulated login used to choose whose tickets a session works with (FR-02, BR-03, BR-11, AC-02, AC-14). Its points are graded inside Part 6; this page is the evidence for the screen itself.

Captured against the running stack by [screenshots/capture-part5.mjs](#) at 1280 px on 2026-09-05. The loading and failure states are produced by intercepting `GET /api/requesters` — delaying it, then failing it — rather than by editing the component, so what is shown is the real screen reacting to a real slow or broken response.

1. The selector, with active requesters only

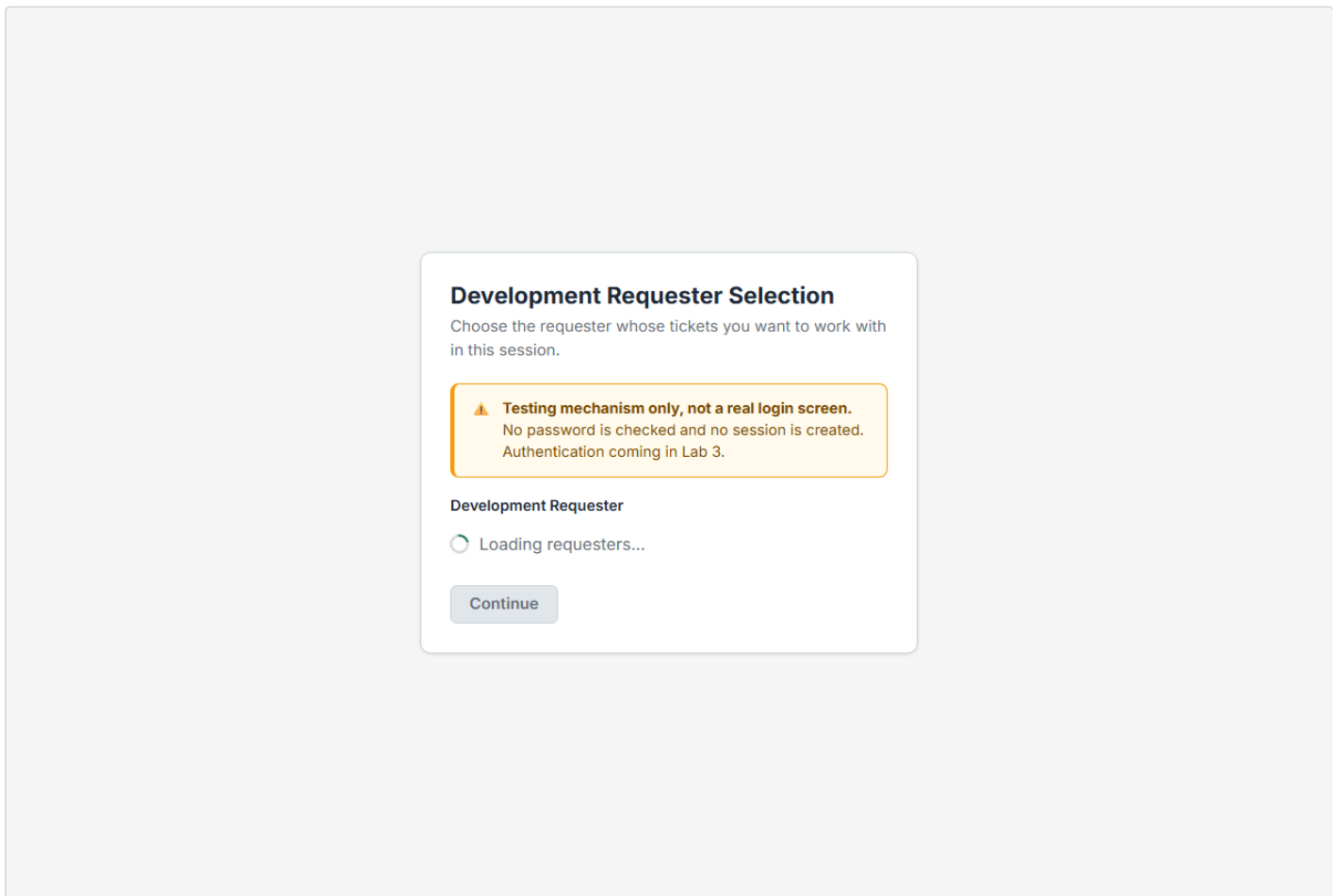


Three things this screen has to say, and does:

- **It is not a login.** The amber callout states it outright: *"Testing mechanism only, not a real login screen. No password is checked and no session is created. Authentication coming in Lab 3."* (BR-03)
- **Only active requesters are listed.** The dropdown holds exactly four options — David Lee (Engineering), Jennifer Anderson (Registrar), Michael Brown (Library), Sarah Johnson (Finance). The seed also creates **Alex Smith (Facilities) with `isActive = false`**, and he is absent. `GET /api/requesters` filters on `isActive` in the query rather than comparing afterwards, so an inactive requester and a non-existent one answer identically (BR-11, AC-14).
- **Continue is disabled until something is chosen.** No default selection, so a session cannot start as "whoever happened to be first".

Covered by **API-12** in `server/tests/requesters.api.test.ts` and **UI-02** in `client/src/components/RequesterSelector.test.tsx`.

2. Loading state



The screenshot shows a modal dialog box titled "Development Requester Selection". The dialog has a light gray background and rounded corners. Inside, the title is in bold. Below the title is a subtitle: "Choose the requester whose tickets you want to work with in this session." Below this is a yellow warning box with a triangle icon and the text: "Testing mechanism only, not a real login screen. No password is checked and no session is created. Authentication coming in Lab 3." Below the warning box is the label "Development Requester" followed by a spinner icon and the text "Loading requesters...". At the bottom is a "Continue" button.

While the request is in flight the dropdown is replaced by a spinner and a status line rather than by an empty select. An empty dropdown would read as "there are no requesters", which is a different fact from "the list has not arrived yet".

3. Failure state

Development Requester Selection

Choose the requester whose tickets you want to work with in this session.

⚠ **Testing mechanism only, not a real login screen.**
No password is checked and no session is created.
Authentication coming in Lab 3.

Development Requester

⚠ Failed to load requesters. Please try again.

Retry

Continue

When the request fails, the screen says so — *"Failed to load requesters. Please try again."* — in an error callout, and offers **Retry** beside a still-disabled **Continue**. The failure is recoverable in place: no reload, no navigation, and no way to continue past a list that was never loaded.

4. The selection in the application shell

TokTickIT

My Tickets

Create Ticket

System Status

REQUESTER
Jennifer Anderson

Change Requester

My Tickets

Tickets raised by Jennifer Anderson.

Create Ticket

Search

Ticket number or summary

Category

All categories

Status

All statuses

Sort by

Newest first

Clear Filters

Showing 1-10 of 71 tickets

TICKET NO.	CREATED DATE	SUMMARY	CATEGORY	REQUESTED PRIORITY	IT PRIORITY	CURRENT STATUS	TICKET OWNER	LAST UPDATED	
TKT-2026-000074	05 n.b. 2569	E2E context — only Jennifer should see this	Account and Access	Low	Unassigned	New	Unassigned	05 n.b. 2569	Detail
TKT-2026-000073	05 n.b. 2569	E2E ownership round trip	Account and Access	Medium	Unassigned	New	Unassigned	05 n.b. 2569	Detail
TKT-2026-000072	05 n.b. 2569	E2E ownership of attachments	Account and Access	Medium	Unassigned	New	Unassigned	05 n.b. 2569	Detail
TKT-2026-000071	05 n.b. 2569	E2E ownership guard	Account and Access	Medium	Unassigned	New	Unassigned	05 n.b. 2569	Detail
TKT-2026-000070	05 n.b. 2569	E2E listed after create	Account and Access	Medium	Unassigned	New	Unassigned	05 n.b. 2569	Detail
TKT-2026-000069	05 n.b. 2569	E2E create flow — projector will not detect the laptop	Hardware	High	Unassigned	New	Unassigned	05 n.b. 2569	Detail
TKT-2026-000068	05 n.b. 2569	E2E rejected file type	Account and Access	Medium	Unassigned	New	Unassigned	05 n.b. 2569	Detail
TKT-2026-000067	05 n.b. 2569	E2E removed attachment stays unreachable	Account and Access	Medium	Unassigned	New	Unassigned	05 n.b. 2569	Detail
TKT-2026-000066	05 n.b. 2569	E2E attachment lifecycle	Account and Access	Medium	Unassigned	New	Unassigned	05 n.b. 2569	Detail
TKT-2026-000065	05 n.b. 2569	E2E context — only Jennifer should see this	Account and Access	Low	Unassigned	New	Unassigned	05 n.b. 2569	Detail

Previous

1

2

3

4

5

Next

Page 1 of 8

Per page

10

After Continue, the header carries the selected requester's name and a **Change Requester** button, and both are present on every screen (FR-06). The list below is scoped to that requester by the server: the id travels in the `X-Requester-Id` header and `GET /api/tickets` writes it into the query's `where` clause, so no client-side filtering is involved (BR-04).

Switching requester unmounts and remounts the routed subtree keyed by requester id, so the previous requester's rows are gone rather than stale (BR-13). That switch is shown in [Answer Part 7](#) and asserted by **UI-07** and end-to-end by **E2E-04**.

What backs this screen in code and tests

Concern	Where
Active-only directory	<code>server/src/app.ts</code> — <code>GET /api/requesters</code> , <code>where: { isActive: true }</code>
Guard when nothing is selected	<code>client/src/components/RequireRequester.tsx</code> — every route sits behind it
Selection persistence	<code>client/src/context/RequesterProvider.tsx</code> — session storage, so a reload does not bounce back to the selector but a new browser session starts clean
Tests	API-12, UI-01, UI-02, UI-07, E2E-04

Answer Part 6:

Working Ticket Screen — Create Mode · 10 points

Answer Part 6 — Create Ticket Screen Evidence (Issue #4)

Six screenshots recording the Create Ticket feature (FR-01, FR-07, AC-01, AC-12, BR-01, BR-02, BR-05 ... BR-07).

Every image was captured against the **running stack with a real database** — Vite dev server, the Express API on port 5000, and seeded PostgreSQL 16 in Docker. The ticket numbers shown are rows that genuinely exist in the `Ticket` table; nothing here is a mock-up.

Item	Value
Captured	2026-08-28
Requester in session	Jennifer Anderson — Registrar
Browser / viewport	Chrome, 1280 px wide, full-page capture
Database	PostgreSQL 16 (Docker), seeded with 4 categories and 7 related systems

0. Before the form — the Development Requester

The lab sheet folds Part 5 into this part's points. The selector screen, its active-only dropdown, its loading and failure states, and the selected requester shown in the application shell with Change Requester are in [Answer Part 5](#).

The link matters for §4 below: the Requester saved on the ticket is the one chosen there, and it is never read from the form.

1. Create Ticket, empty, with master data loaded

TokTickIT

My Tickets

Create Ticket

System Status

REQUESTER
Jennifer Anderson

Change Requester

Create Ticket

Raise a new ticket as the selected requester. The ticket number and status are assigned by the server.

Summary / Title *

Short description of the problem

Description *

What happened, what you expected, and how to reproduce it

Category *

Select a category...

Related System *

Select a related system...

Requested Priority *

Low

Medium

High

Attachments

Drag files here, or

Browse files

JPG, PNG, WEBP, PDF · up to 5.0 MB each · at most 5 files

Cancel

Create Ticket

The screen after `GET /api/categories` and `GET /api/related-systems` have both resolved. Both dropdowns are enabled and populated from the database, each required field carries a red `*`, and the attachment dropzone states the allowed types, size limit and file cap.

Neither Ticket Number nor Status appears as an input — both are server-owned (BR-01, BR-02).

2. Validation errors under the fields

TokTickITMy TicketsCreate TicketSystem Status

REQUESTERJennifer AndersonChange Requester

Create Ticket

Raise a new ticket as the selected requester. The ticket number and status are assigned by the server.

Summary / Title *

Short description of the problem

Summary is required.

Description *

What happened, what you expected, and how to reproduce it

Description is required.

Category *

Select a category...

Category is required.

Related System *

Select a related system...

Related System is required.

Requested Priority *

Low

Medium

High

Attachments

Drag files here, or

Browse files

JPG, PNG, WEBP, PDF · up to 5.0 MB each · at most 5 files

Cancel

Create Ticket

Submitting an empty form. Each message renders **directly under its own field** in `--zg-error`, and the field border turns error-colored (ui-spec §2, V-08). No request is sent while the form is invalid.

Covered by test **UI-03** in `client/src/components/CreateTicketForm.test.tsx`.

3. Submitting / busy state

CPE334 Lab 2 — TokTickIT — Thawat Boonsuk 67070501024

56 / 103

Create Ticket

Raise a new ticket as the selected requester. The ticket number and status are assigned by the server.

Summary / Title *

Laptop battery drains quickly

Description *

The battery drops from 100% to 20% within an hour of unplugging.

Category *

Hardware

Related System *

Corporate Laptop

Requested Priority *

☐ Low

☐ Medium

☒ High

Attachments

Drag files here, or

[Browse files](#)

JPG, PNG, WEBP, PDF · up to 5.0 MB each · at most 5 files

[Cancel](#)

 Creating ticket...

While the `POST /api/tickets` request is in flight the button is disabled, shows a spinner, and its label swaps to *Creating ticket...* — so a second click cannot double-submit.

4. Success, with the Ticket Number from the database

TokTickIT

My Tickets

Create Ticket

System Status

REQUESTER
Jennifer Anderson

Change Requester

Create Ticket

Raise a new ticket as the selected requester. The ticket number and status are assigned by the server.

✔ Ticket created.

Your ticket number is TKT-2026-000005

Status New Priority High

View ticket

Go to My Tickets

Create another ticket

The confirmation shows TKT-2026-000005 , generated server-side in the same transaction as the insert, together with status New and the requested priority. Onward navigation to the ticket detail or to My Tickets is offered.

This satisfies AC-01. Verified directly in the database:

ticketNumber	status	priority
TKT-2026-000001	New	High
TKT-2026-000002	New	Low
TKT-2026-000003	New	High
TKT-2026-000004	New	High
TKT-2026-000005	New	High

The sequence increments and never repeats (BR-01), and every row lands with status New (BR-02).

5. Backend failure — form values preserved

TokTickITMy TicketsCreate TicketSystem Status

REQUESTERJennifer AndersonChange Requester

Create Ticket

Raise a new ticket as the selected requester. The ticket number and status are assigned by the server.

Summary / Title *

Laptop battery drains quickly

Description *

The battery drops from 100% to 20% within an hour of unplugging.

Category *

Hardware

Related System *

Corporate Laptop

Requested Priority *

Low

Medium

High

Attachments

Drag files here, or

Browse files

JPG, PNG, WEBP, PDF · up to 5.0 MB each · at most 5 files

⚠ Could not reach the server. Your details are still here — check your connection and try again.

Cancel

Create Ticket

The API is unreachable when Create Ticket is pressed. An error callout appears above the form actions, **every entered value stays in the form**, and the submit button returns to its idle state so the user can retry immediately.

Read back from the DOM at the moment of capture:

Field	Value still present
Summary / Title	Laptop battery drains quickly
Description	The battery drops from 100% to 20% within an hour of unplugging.
Category	Hardware
Related System	Corporate Laptop
Requested Priority	High

This satisfies **AC-12 / FR-07**, and is covered by test **UI-08**.

The capture reproduces the failure by refusing the connection at the browser. The same behaviour was also confirmed by genuinely stopping the Express server mid-session: the callout appeared, the values survived, and pressing Create Ticket again after restarting the server produced the next number in sequence.

6. Attachment selection — valid and invalid

TokTickITMy TicketsCreate TicketSystem Status

REQUESTERJennifer AndersonChange Requester

Create Ticket

Raise a new ticket as the selected requester. The ticket number and status are assigned by the server.

Summary / Title *

Laptop battery drains quickly

Description *

The battery drops from 100% to 20% within an hour of unplugging.

Category *

Hardware

Related System *

Corporate Laptop

Requested Priority *

Low

Medium

High

Attachments

Drag files here, or

Browse files

JPG, PNG, WEBP, PDF · up to 5.0 MB each · at most 5 files

screenshot.png120 KB

Remove

These files were not added:

huge-scan.pdf — File is 6.0 MB. The limit is 5.0 MB.

payload.exe — Unsupported file type. Allowed: JPG, PNG, WEBP, PDF.

Cancel

Create Ticket

Three files were offered at once:

File	Size	Type	Outcome
screenshot.png	120 KB	image/png	Accepted — listed with size and a Remove control
huge-scan.pdf	6.0 MB	application/pdf	Rejected — over the 5 MB limit (BR-06)
payload.exe	2 KB	application/x-msdownload	Rejected — unsupported type (BR-05)

Each rejection names the file and the rule it broke. The sixth file in a selection is also refused (BR-07).

Scope note: uploading is not part of Issue #4 — `POST /api/tickets/:id/attachments` belongs to Issue #6. Files here are validated and staged in the browser only, and the success screen says plainly that staged files were not attached rather than dropping them silently.

CPE334 Lab 2 — TokTickIT — Thawat Boonsuk 67070501024

60 / 103

Reproducing these screenshots

With Docker running and the database seeded, start both dev servers, then:

```
node docs/lab-02/screenshots/capture.mjs
```

The script drives a real Chrome through all six states and overwrites the PNGs in place. Each run creates real tickets, so the number in screenshot 4 advances.

Answer Part 7:

Working My Tickets Screen · 10 points

Answer Part 7 — My Tickets List Evidence (Issue #5)

Six screenshots recording the My Tickets feature (FR-03, AC-04, AC-10, AC-15, BR-04, BR-13).

Every image was captured against the **running stack with a real database** — Vite dev server, the Express API on port 5000, and seeded PostgreSQL 16 in Docker. The ticket numbers shown are rows that genuinely exist in the `Ticket` table; nothing here is a mock-up.

Item	Value
Captured	2026-08-29
Requester A	Jennifer Anderson — Registrar (14 tickets)
Requester B	Sarah Johnson — Finance (3 tickets)
Requester with no tickets	Michael Brown — Library
Browser / viewport	Chromium, 1280 px wide (375 px for the mobile check), full-page capture
Database	PostgreSQL 16 (Docker), seeded with 4 categories and 7 related systems
Capture script	screenshots/capture-issue5.mjs

The tickets were created through the real `POST /api/tickets`, so every row carries a server-generated ticket number and a server-owned `New` status.

1. Requester A with their tickets listed

TokTickIT

My Tickets

Create Ticket

System Status

REQUESTER
Jennifer Anderson

Change Requester

My Tickets

Tickets raised by Jennifer Anderson.

Create Ticket

Search

Ticket number or summary

Category

All categories

Status

All statuses

Sort by

Newest first

Clear Filters

Showing 1-10 of 14 tickets

Previous

1

2

Next

Page 1 of 2

Per page

10

Jennifer Anderson's 14 tickets, newest first, ten to a page. The table carries all ten columns the issue specifies: Ticket No., Created Date, Summary, Category, Requested Priority, IT Priority, Current Status, Ticket Owner, Last Updated, and a Detail action.

Status renders as a `New` pill and Requested Priority as a `Low` / `Medium` / `High` pill (ui-spec §2). IT Priority and Ticket Owner show an italic `Unassigned` placeholder: both are IT-triage fields with no column in the Lab 2 schema, and an explicit placeholder keeps "not yet assigned" from reading as missing data.

Covered by test **UI-04** in `client/src/components/MyTickets.test.tsx` and **API-04** in `server/tests/tickets.api.test.ts`.

2. Switching to Requester B — Requester A's tickets are gone

TokTickIT

My Tickets

Create Ticket

System Status

REQUESTER
Sarah Johnson

Change Requester

My Tickets

Tickets raised by Sarah Johnson.

Create Ticket

Search

Ticket number or summary

Category

All categories

Status

All statuses

Sort by

Newest first

Clear Filters

Showing 1-3 of 3 tickets

TICKET NO.	CREATED DATE	SUMMARY	CATEGORY	REQUESTED PRIORITY	IT PRIORITY	CURRENT STATUS	TICKET OWNER	LAST UPDATED	
TKT-2026-000017	29 ส.ค. 2569	Desk phone has no dial tone	Hardware	Low	Unassigned	New	Unassigned	29 ส.ค. 2569	Detail
TKT-2026-000016	29 ส.ค. 2569	Finance shared mailbox is read-only	Account and Access	Medium	Unassigned	New	Unassigned	29 ส.ค. 2569	Detail
TKT-2026-000015	29 ส.ค. 2569	Invoice report totals do not match the ledger	Software	High	Unassigned	New	Unassigned	29 ส.ค. 2569	Detail

Previous

Next

Page 1 of 1

Per page 10

The same screen immediately after switching to Sarah Johnson through the header's Change Requester control. Every one of Jennifer's rows is gone and only Sarah's three tickets (TKT-2026-000015 ... 000017) remain — the count line reads *Showing 1–3 of 3 tickets*.

This is BR-04 and BR-13 together. Ownership is enforced server-side: `requesterId` is written into the query's `where` from the resolved `x-Requester-Id` header, and no query parameter can widen it — a `requesterId` supplied in the query string is ignored outright. On the client, AppShell keys the routed subtree by the requester id, so the switch unmounts the list and remounts it empty rather than leaving stale rows on screen.

Covered by **API-04** (ownership scoping assertions) and **UI-07**.

3. Search and the Category / Status filters in use

TokTickIT

My Tickets

Create Ticket

System Status

REQUESTER
Jennifer Anderson

Change Requester

My Tickets

Tickets raised by Jennifer Anderson.

Create Ticket

Search

vpn

Category

Network

Status

New

Sort by

Newest first

Clear Filters

Showing 1-2 of 2 tickets

TICKET NO.	CREATED DATE	SUMMARY	CATEGORY	REQUESTED PRIORITY	IT PRIORITY	CURRENT STATUS	TICKET OWNER	LAST UPDATED	
TKT-2026-000014	29 ส.ค. 2569	VPN disconnects after fifteen minutes	Network	Medium	Unassigned	New	Unassigned	29 ส.ค. 2569	Detail
TKT-2026-000002	29 ส.ค. 2569	Cannot connect to the campus VPN from home	Network	High	Unassigned	New	Unassigned	29 ส.ค. 2569	Detail

Previous

Next

Page 1 of 1

Per page 10

Search `vpn`, Category `Network`, Status `New` applied together. Two of the fourteen tickets match, and the result count updates with them.

The search is case-insensitive — `vpn` matches both *VPN disconnects after fifteen minutes* and *Cannot connect to the campus VPN from home* — and runs against `ticketNumber`, `summary` and `description` on the server, not against the loaded page. Each control maps to one query parameter (`search`, `category`, `status`), and the input is debounced 300 ms so typing does not fire a request per keystroke.

Covered by **API-06** and **UI-04**.

3b. Sorting

TokTickIT

My Tickets

Create Ticket

System Status

REQUESTER
Jennifer Anderson

Change Requester

My Tickets

Tickets raised by Jennifer Anderson.

Create Ticket

Search

Ticket number or summary

Category

All categories

Status

All statuses

Sort by

Priority (High first)

Clear Filters

Showing 1-10 of 71 tickets

Previous

1

2

3

4

5

Next

Page 1 of 8

Per page

10

Priority (High first) selected. Every row on page 1 is **High**, where the default Newest first order interleaved them — the same ten rows read Low, Medium, Medium, Medium, Medium, High, Medium, Medium, Medium, Low before the sort was changed.

TokTickIT

My Tickets

Create Ticket

System Status

REQUESTER
Jennifer Anderson

Change Requester

My Tickets

Tickets raised by Jennifer Anderson.

Create Ticket

Search

Ticket number or summary

Category

All categories

Status

All statuses

Sort by

Priority (Low first)

Clear Filters

Showing 1-10 of 71 tickets

TICKET NO.	CREATED DATE	SUMMARY	CATEGORY	REQUESTED PRIORITY	IT PRIORITY	CURRENT STATUS	TICKET OWNER	LAST UPDATED	
TKT-2026-000056	05 พ.ย. 2569	E2E context — only Jennifer should see this	Account and Access	Low	Unassigned	New	Unassigned	05 พ.ย. 2569	Detail
TKT-2026-000065	05 พ.ย. 2569	E2E context — only Jennifer should see this	Account and Access	Low	Unassigned	New	Unassigned	05 พ.ย. 2569	Detail
TKT-2026-000074	05 พ.ย. 2569	E2E context — only Jennifer should see this	Account and Access	Low	Unassigned	New	Unassigned	05 พ.ย. 2569	Detail
TKT-2026-000012	29 ส.ค. 2569	Email signature is stripped on outgoing mail	Software	Low	Unassigned	New	Unassigned	29 ส.ค. 2569	Detail
TKT-2026-000010	29 ส.ค. 2569	Keyboard keys stick on the front-desk machine	Hardware	Low	Unassigned	New	Unassigned	29 ส.ค. 2569	Detail
TKT-2026-000007	29 ส.ค. 2569	Second monitor is not detected after docking	Hardware	Low	Unassigned	New	Unassigned	29 ส.ค. 2569	Detail
TKT-2026-000024	04 พ.ย. 2569	E2E ownership guard	Account and Access	Medium	Unassigned	New	Unassigned	04 พ.ย. 2569	Detail
TKT-2026-000028	04 พ.ย. 2569	E2E ownership guard	Account and Access	Medium	Unassigned	New	Unassigned	04 พ.ย. 2569	Detail
TKT-2026-000062	05 พ.ย. 2569	E2E ownership guard	Account and Access	Medium	Unassigned	New	Unassigned	05 พ.ย. 2569	Detail
TKT-2026-000023	04 พ.ย. 2569	E2E ownership round trip	Account and Access	Medium	Unassigned	New	Unassigned	04 พ.ย. 2569	Detail

Previous

1

2

3

4

5

Next

Page 1 of 8

Per page

10

Reversed to `Priority (Low first)`, page 1 becomes Low, Low, Low, Low, Low, Low, Medium, Medium, Medium, Medium.

The ordering is the server's: the control writes the `sort` query parameter and the endpoint orders in SQL, so it applies to the whole result set rather than to the ten rows already on screen. Priority sorts by rank rather than alphabetically — High, Medium, Low, not High, Low, Medium — and every sort is tie-broken by `id` ascending so no row can appear on two pages.

These two captures were taken later than the rest of Part 7, after the end-to-end suite had added tickets, so the dataset is larger than the fourteen tickets in the images above.

Covered by **API-06** and **UI-04**.

4. Pagination controls

TokTickITMy TicketsCreate TicketSystem Status

REQUESTERJennifer AndersonChange Requester

My Tickets

Tickets raised by Jennifer Anderson.

Create Ticket

Search

Ticket number or summary

Category

All categories

Status

All statuses

Sort by

Newest first

Clear Filters

Showing 11–14 of 14 tickets

TICKET NO.	CREATED DATE	SUMMARY	CATEGORY	REQUESTED PRIORITY	IT PRIORITY	CURRENT STATUS	TICKET OWNER	LAST UPDATED	
TKT-2026-000004	29 ส.ค. 2569	Wi-Fi drops repeatedly in the Registrar office	Network	Medium	Unassigned	New	Unassigned	29 ส.ค. 2569	Detail
TKT-2026-000003	29 ส.ค. 2569	Printer jams on every print job	Hardware	Medium	Unassigned	New	Unassigned	29 ส.ค. 2569	Detail
TKT-2026-000002	29 ส.ค. 2569	Cannot connect to the campus VPN from home	Network	High	Unassigned	New	Unassigned	29 ส.ค. 2569	Detail
TKT-2026-000001	29 ส.ค. 2569	Laptop battery drains quickly	Hardware	High	Unassigned	New	Unassigned	29 ส.ค. 2569	Detail

Previous

1

2

Next

Page 2 of 2

Per page

10

Page 2 of Jennifer's list. *Showing 11–14 of 14 tickets*, the page-2 button is marked current, Next is disabled because there is no page 3, and Previous is enabled.

The metadata comes from the server (`page` , `pageSize` , `totalItems` , `totalPages` , `hasNextPage` , `hasPreviousPage`); every sort is tie-broken by `id` ascending so no row can appear on two pages. Changing search, filter, sort or page size returns to page 1 — staying on page 3 of a narrower result set is how a working filter comes to look like it matched nothing.

Covered by **API-06** and **UI-04**.

5. Empty state — the requester has no tickets

TokTickIT

My Tickets

Create Ticket

System Status

REQUESTER
Michael Brown

Change Requester

My Tickets

Tickets raised by Michael Brown.

Create Ticket

Search

Ticket number or summary

Category

All categories

Status

All statuses

Sort by

Newest first

Clear Filters



Welcome — you have no tickets yet

When you raise a request for the IT team it appears here, with its ticket number and current status.

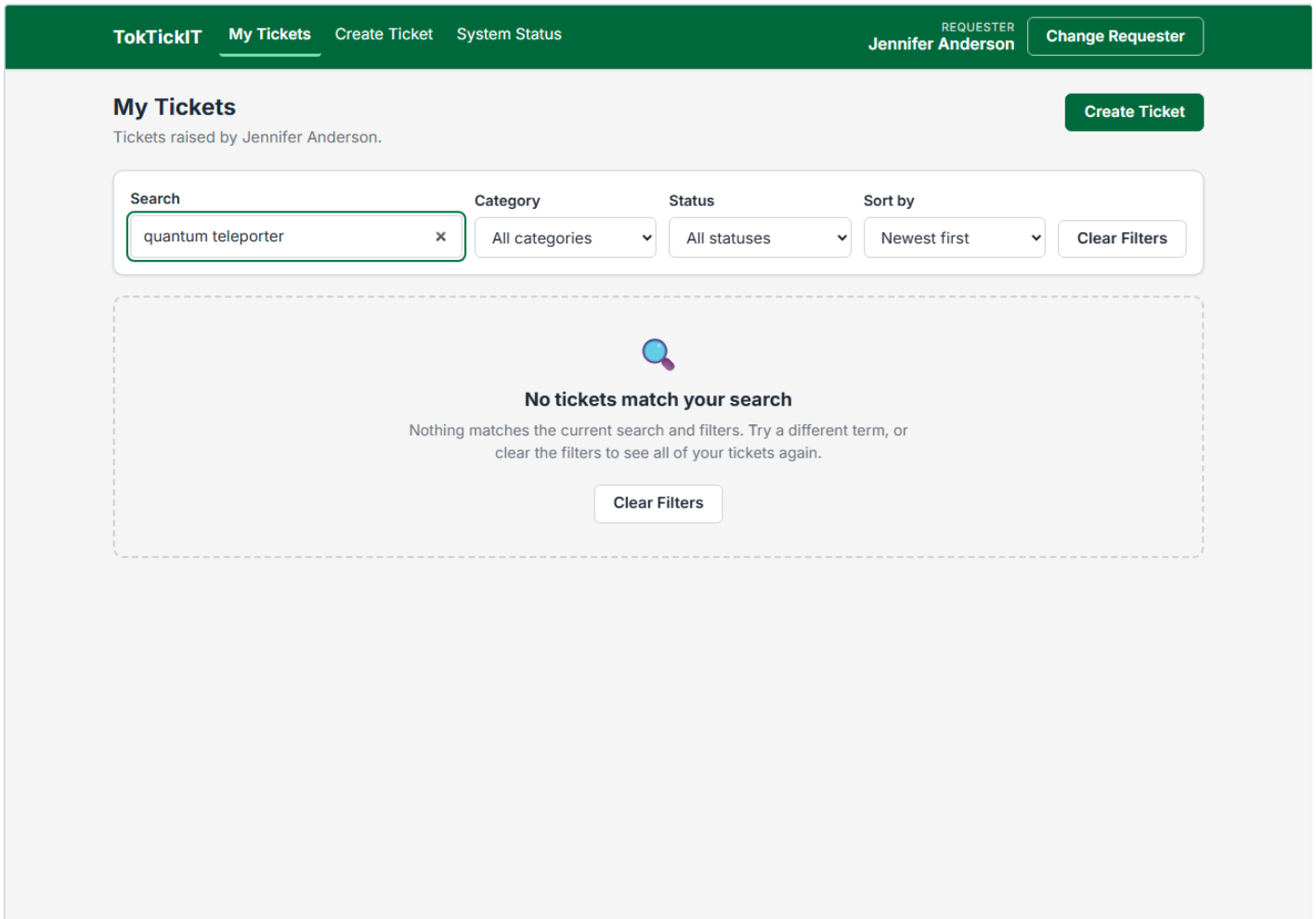
Create your first ticket

Michael Brown has never raised a ticket. The screen shows a welcome message on a solid white card with a single primary call to action, *Create your first ticket*.

There is deliberately no "Clear Filters" button here: no filter is active, so there is nothing to clear.

Covered by **UI-09**.

6. No-results state — the search matched nothing



The same requester as screenshot 1, searching for `quantum teleporter`. The server answers `200` with an empty array and `totalItems: 0` — not a `404` and not an error.

AC-15 requires this to be visibly distinct from the empty state above, and it is, in three ways: a dashed tinted panel instead of a solid white card, a magnifier instead of a seedling, and a secondary *Clear Filters* action instead of a primary *Create* action. The distinction is decided from the response plus whether a filter is active: `totalItems === 0` with no active search or filter is the empty state; `totalItems === 0` with one is the no-results state.

Covered by **API-13** and **UI-09**.

Cross-requester access

Section 2 shows that switching requester replaces the visible list. The refusal that backs it — opening another requester's ticket by URL, and requesting their attachment directly — is in [Answer Part 8](#) §6, where the screen shows Access denied and the API answers `403` to both `GET /api/tickets/:id` and `GET /api/attachments/:id/download`.

Responsive check

TokTickIT

Menu

My Tickets

Tickets raised by Jennifer Anderson.

Create Ticket

Search

Ticket number or summary

Category

All categories

Status

All statuses

Sort by

Newest first

Clear Filters

Mobile card list — 1 of 6

Showing 1–10 of 62 tickets

TKT-2026-000065

New

E2E context — only Jennifer should see this

Low

Account and Access

CREATED

05 n.y. 2569

LAST UPDATED

05 n.y. 2569

IT PRIORITY

Unassigned

TICKET OWNER

Unassigned

TKT-2026-000064

New

E2E ownership round trip

Medium

Account and Access

CREATED

05 n.y. 2569

LAST UPDATED

05 n.y. 2569

IT PRIORITY

Unassigned

TICKET OWNER

Unassigned

TKT-2026-000063

New

E2E ownership of attachments

Low

Account and Access

CREATED

05 n.y. 2569

LAST UPDATED

05 n.y. 2569

IT PRIORITY

Unassigned

TICKET OWNER

Unassigned

Mobile card list — 2 of 6

E2E ownership of attachments

Medium

Account and Access

CREATED

05 n.ย. 2569

IT PRIORITY

Unassigned

LAST UPDATED

05 n.ย. 2569

TICKET OWNER

Unassigned

TKT-2026-000062

New

E2E ownership guard

Medium

Account and Access

CREATED

05 n.ย. 2569

IT PRIORITY

Unassigned

LAST UPDATED

05 n.ย. 2569

TICKET OWNER

Unassigned

TKT-2026-000061

New

E2E listed after create

Medium

Account and Access

CREATED

05 n.ย. 2569

IT PRIORITY

Unassigned

LAST UPDATED

05 n.ย. 2569

TICKET OWNER

Unassigned

Mobile card list — 3 of 6

TKT-2026-000060

New

E2E create flow — projector will not detect the laptop

High

Hardware

CREATED

05 n.ย. 2569

LAST UPDATED

05 n.ย. 2569

IT PRIORITY

Unassigned

TICKET OWNER

Unassigned

TKT-2026-000059

New

E2E rejected file type

Medium

Account and Access

CREATED

05 n.ย. 2569

LAST UPDATED

05 n.ย. 2569

IT PRIORITY

Unassigned

TICKET OWNER

Unassigned

TKT-2026-000058

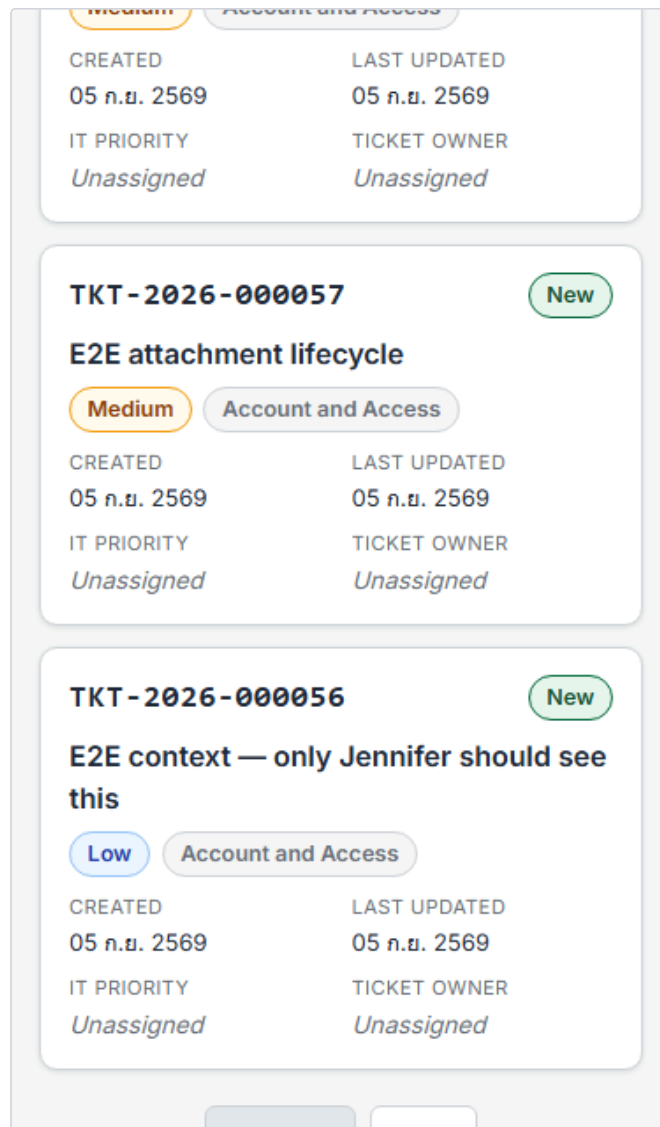
New

E2E removed attachment stays unreachable

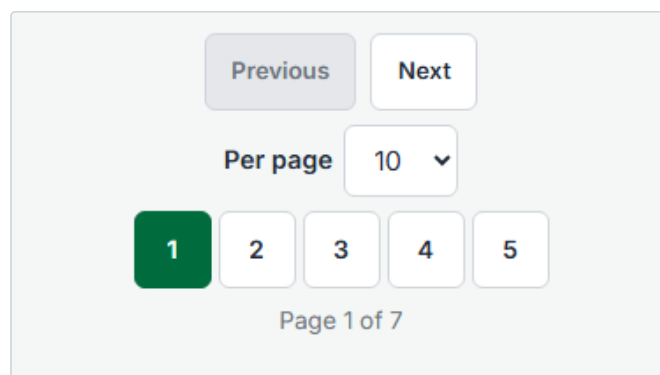
Medium

Account and Access

Mobile card list — 4 of 6



Mobile card list — 5 of 6



Mobile card list — 6 of 6

At 375 px the table is replaced by a stacked card list (ui-spec §4). Each card is a single tap target carrying the ticket number, status badge, summary, priority and category, with the dates and the two triage fields in a two-column definition list. The filters stack full-width above it.

Only one layout is in the DOM at a time — the component picks between table and cards from a media query rather than rendering both and hiding one, so the ticket data is not duplicated for screen readers.

Horizontal overflow was measured rather than eyeballed, comparing `document.body.scrollWidth` with the viewport width on the loaded list:

Viewport	Layout rendered	Body scroll width	Horizontal overflow
375 px	Card list	375 px	0
820 px	Table (scrolls in its own container)	820 px	0
1280 px	Table, all ten columns visible	1280 px	0

Reproducing these captures

```
docker compose up -d
```

```
npm --prefix server run prisma:migrate
```

```
npm --prefix server run prisma:seed
```

Then start both dev servers, create the demo tickets through `POST /api/tickets`, and run:

```
node docs/lab-02/screenshots/capture-issue5.mjs
```

Answer Part 8:

Working Ticket Screen — View Mode and Attachments · 5 points

Answer Part 8 — Ticket Detail and Attachment Lifecycle Evidence (Issue #6)

Six screenshots recording the Requester Ticket Detail screen and the soft attachment lifecycle (FR-04, FR-05, AC-03, AC-05, AC-06 ... AC-09, BR-04 ... BR-10).

Every image was captured against the **running stack with a real database** — Vite on :5173, the Express API on :5000, and PostgreSQL 16 on :5433. The ticket number, the stored files and the removal metadata are rows that genuinely exist in the `Ticket` and `Attachment` tables.

Item	Value
Captured	2026-09-04
Ticket	TKT-2026-000041 — Laptop battery drains within an hour, priority High, status New
Owner (Requester A)	Jennifer Anderson — Registrar
Intruder (Requester B)	Sarah Johnson — Finance
Files used	battery-diagnostics.pdf (42 B, %PDF-1.7 header), battery-graph.png (37 B, real PNG signature)
Browser / viewport	Chromium, 1280 px wide (375 px for the mobile checks)
Capture script	screenshots/capture-issue6.mjs

The ticket was created through the real `POST /api/tickets` and the files through the real `POST /api/tickets/:id/attachments`, so the ticket number and the `New` status are server-generated and the stored files are on disk under `server/uploads/`.

1. Ticket Detail of a ticket the requester owns — read-only

TokTickITMy TicketsCreate TicketSystem Status

REQUESTERJennifer AndersonChange Requester

[My Tickets](#) / TKT-2026-000041

Laptop battery drains within an hour

TKT-2026-000041NewHighCreated 04 n.y. 2569 15:19

Ticket details

This view is read-only. Ticket fields are set when the ticket is created and are changed by the IT team, not here.

TICKET NO.	CREATED DATE	CATEGORY	RELATED SYSTEM
TKT-2026-000041	04 n.y. 2569 15:19	Account and Access	Campus Wi-Fi
REQUESTER	REQUESTED PRIORITY	IT PRIORITY	CURRENT STATUS
Jennifer Anderson · Registrar	High	Unassigned	New
TICKET OWNER	LAST UPDATED		
Unassigned	04 n.y. 2569 15:19		
SUMMARY			
Laptop battery drains within an hour			
DESCRIPTION			
The battery drops from 100% to 20% within an hour of unplugging, even with the screen dimmed. It started after the last firmware update.			

Attachments (0 of 5 active)

No files are attached to this ticket yet.

Add a file

JPG, PNG, WEBP, PDF up to 5.0 MB. A ticket can hold 5 active files; removing one frees a slot.

Choose Files

No file chosen

Every field the issue lists is on the page: Ticket No., Created Date, Category, Related System, Requester, Requested Priority, IT Priority, Current Status, Ticket Owner, Last Updated, Summary and Description.

They are rendered as a definition list, not as a disabled form. That distinction is the point of AC-05: a greyed-out input still reads as *editable, but not right now*, which is the wrong promise on a screen where nothing is ever editable. There is no Edit button, no Save button and no status control anywhere — not disabled, absent. IT Priority and Ticket Owner carry the same italic *Unassigned* placeholder as the list, because both are IT-triage fields with no column in the Lab 2 schema.

Covered by **UI-05** in `client/src/components/TicketDetail.test.tsx`, which asserts that the ticket-fields region contains no `input`, `textarea`, `select` or `button` at all.

2. Adding a new attachment to an existing ticket

TokTickITMy TicketsCreate TicketSystem Status

REQUESTER
Jennifer AndersonChange Requester

[My Tickets](#) / TKT-2026-000041

Laptop battery drains within an hour

TKT-2026-000041NewHigh Created 04 n.u. 2569 15:19

Ticket details

This view is read-only. Ticket fields are set when the ticket is created and are changed by the IT team, not here.

TICKET NO.	CREATED DATE	CATEGORY	RELATED SYSTEM
TKT-2026-000041	04 n.u. 2569 15:19	Account and Access	Campus Wi-Fi
REQUESTER	REQUESTED PRIORITY	IT PRIORITY	CURRENT STATUS
Jennifer Anderson · Registrar	High	Unassigned	New
TICKET OWNER	LAST UPDATED		
Unassigned	04 n.u. 2569 15:19		
SUMMARY			
Laptop battery drains within an hour			
DESCRIPTION			
The battery drops from 100% to 20% within an hour of unplugging, even with the screen dimmed. It started after the last firmware update.			

Attachments (2 of 5 active)

battery-graph.png37 BUploaded 04 n.u. 2569 15:19

DownloadRemove

battery-diagnostics.pdf42 BUploaded 04 n.u. 2569 15:19

DownloadRemove

Add a file

JPG, PNG, WEBP, PDF up to 5.0 MB. A ticket can hold 5 active files; removing one frees a slot.

Choose Files

No file chosen

1 file attached.

battery-diagnostics.pdf and battery-graph.png picked through the Add a file control and uploaded one request each to POST /api/tickets/:id/attachments. The counter reads (2 of 5 active) and each row shows the file name, its size and its upload time beside Download and Remove.

Uploads happen after the ticket exists rather than as part of creating it (api-spec.md §3.7), so a rejected file can never cost the requester the ticket. The helper text states the rules the server enforces: JPG, PNG, WEBP, PDF up to 5.0 MB, five active files per ticket.

The server decides the type from the file's leading bytes, not from its name or its Content-Type, so renaming an executable to .pdf does not get past BR-05. Covered by API-08 (415 for a disallowed type, including the renamed-executable case), API-07 (413 over 5 MB) and API-09 (the sixth active file).

3. Downloading an active attachment

CPE334 Lab 2 — TokTickIT — Thawat Boonsuk 67070501024

79

/

103

TokTickIT

My Tickets

Create Ticket

System Status

REQUESTER
Jennifer Anderson

Change Requester

My Tickets / TKT-2026-000041

Laptop battery drains within an hour

TKT-2026-000041 New High Created 04 n.u. 2569 15:19

Ticket details

This view is read-only. Ticket fields are set when the ticket is created and are changed by the IT team, not here.

TICKET NO.	CREATED DATE	CATEGORY	RELATED SYSTEM
TKT-2026-000041	04 n.u. 2569 15:19	Account and Access	Campus Wi-Fi
REQUESTER	REQUESTED PRIORITY	IT PRIORITY	CURRENT STATUS
Jennifer Anderson · Registrar	High	Unassigned	New
TICKET OWNER	LAST UPDATED		
Unassigned	04 n.u. 2569 15:19		
SUMMARY			
Laptop battery drains within an hour			
DESCRIPTION			
The battery drops from 100% to 20% within an hour of unplugging, even with the screen dimmed. It started after the last firmware update.			

Attachments (2 of 5 active)

battery-graph.png

37 B

Uploaded 04 n.u. 2569 15:19

Download

Remove

battery-diagnostics.pdf

42 B

Uploaded 04 n.u. 2569 15:19

Downloading...

Remove

Add a file

JPG, PNG, WEBP, PDF up to 5.0 MB. A ticket can hold 5 active files; removing one frees a slot.

Choose Files

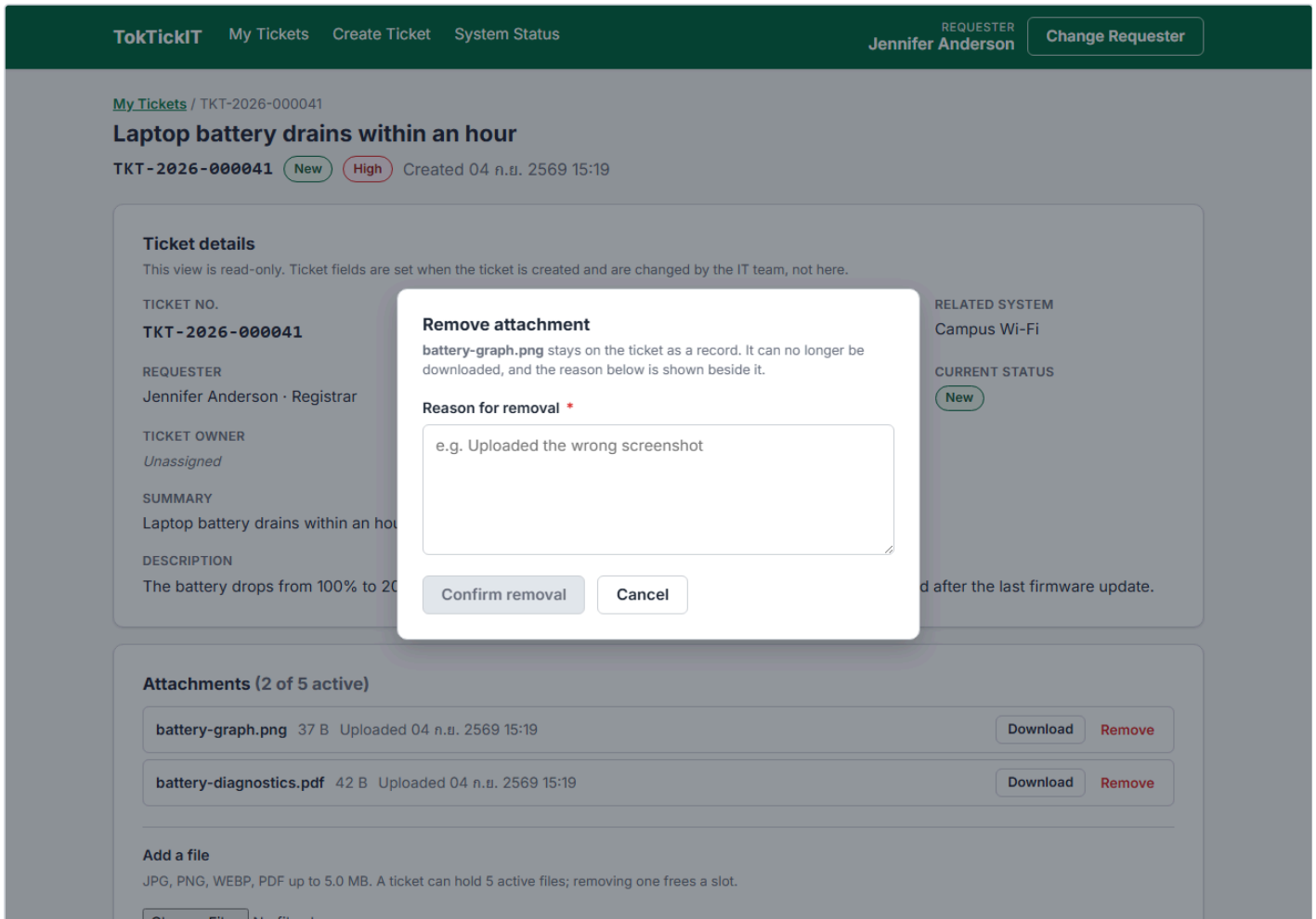
No file chosen

1 file attached.

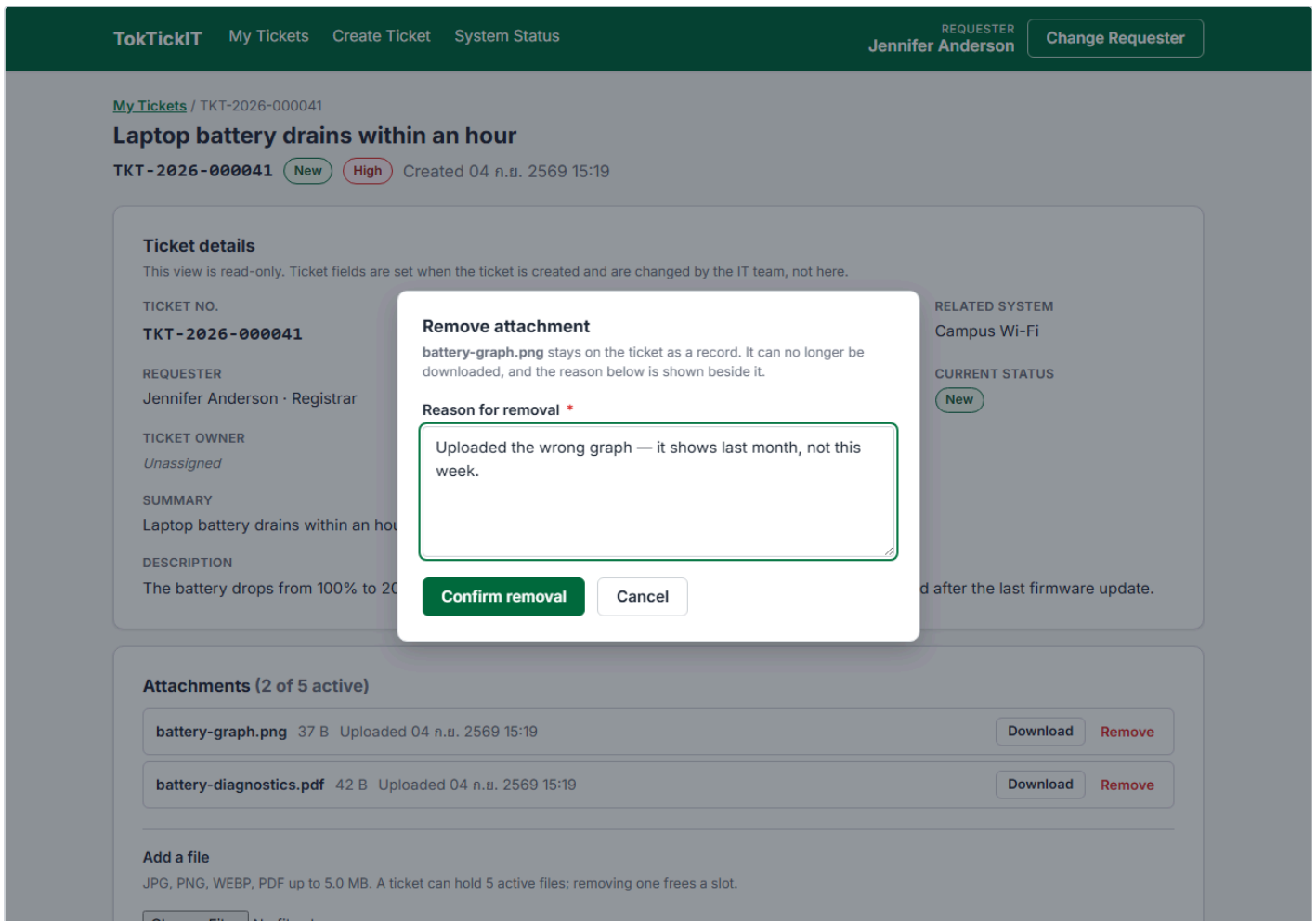
battery-diagnostics.pdf being downloaded. The button shows its busy state — the accessible name changes to *Downloading battery-diagnostics.pdf* as well as the label, so a screen-reader user is not told *Download* while the request is in flight. The response was deliberately delayed by 1.5 s for this capture so the in-flight state is visible; the download itself is a real one.

The download goes through `fetch` rather than a plain link, because the requester context travels in the `x-Requester-Id` header and an `<a href>` cannot carry it. The bytes that came back are saved beside the screenshots as `detail-03-downloaded-file.pdf` — 42 bytes, byte-identical to what was uploaded.

4. The removal modal insists on a reason



Pressing Remove opens the modal with an empty reason box and **Confirm removal disabled**. A removal without a stated reason is not offered rather than merely rejected, which is BR-09 expressed in the interface.



With a reason typed, Confirm enables. Whitespace alone does not count: the button re-disables if the box holds only spaces, and the server repeats the check — `PATCH /api/attachments/:id/remove` answers `400 VALIDATION_FAILED` for a missing, blank or too-short reason.

Covered by **UI-06** (Confirm disabled until a reason is entered, and re-disabled when it is cleared) and **API-14** (`400` with the attachment unchanged).

5. The attachment after a soft removal

TokTickITMy TicketsCreate TicketSystem Status

REQUESTER
Jennifer AndersonChange Requester

My Tickets / TKT-2026-000041

Laptop battery drains within an hour

TKT-2026-000041NewHighCreated 04 n.đ. 2569 15:19

Ticket details

This view is read-only. Ticket fields are set when the ticket is created and are changed by the IT team, not here.

TICKET NO.	CREATED DATE	CATEGORY	RELATED SYSTEM
TKT-2026-000041	04 n.đ. 2569 15:19	Account and Access	Campus Wi-Fi
REQUESTER	REQUESTED PRIORITY	IT PRIORITY	CURRENT STATUS
Jennifer Anderson · Registrar	High	Unassigned	New
TICKET OWNER	LAST UPDATED		
Unassigned	04 n.đ. 2569 15:19		
SUMMARY			
Laptop battery drains within an hour			
DESCRIPTION			
The battery drops from 100% to 20% within an hour of unplugging, even with the screen dimmed. It started after the last firmware update.			

Attachments (1 of 5 active)

battery-diagnostics.pdf42 BUploaded 04 n.đ. 2569 15:19

DownloadRemove

REMOVED FILES

battery-graph.pngRemoved37 B

Removed 04 n.đ. 2569 15:19 — Uploaded the wrong graph — it shows last month, not this week.

Add a file

JPG, PNG, WEBP, PDF up to 5.0 MB. A ticket can hold 5 active files; removing one frees a slot.

Choose Files

No file chosen

1 file attached.

battery-graph.png under a *Removed files* heading: muted, struck through, carrying a **Removed** badge, the removal timestamp and the reason "Uploaded the wrong graph — it shows last month, not this week." There is **no Download button** on the row.

Nothing was deleted. The row is still in the `Attachment` table with `isRemoved = true`, `removedReason` and `removedAt` populated, and the file is still on disk — the record of what was attached survives the file becoming unreachable (BR-08, BR-10). The active counter drops back to (1 of 5 active): removed files do not consume one of the five slots (BR-07).

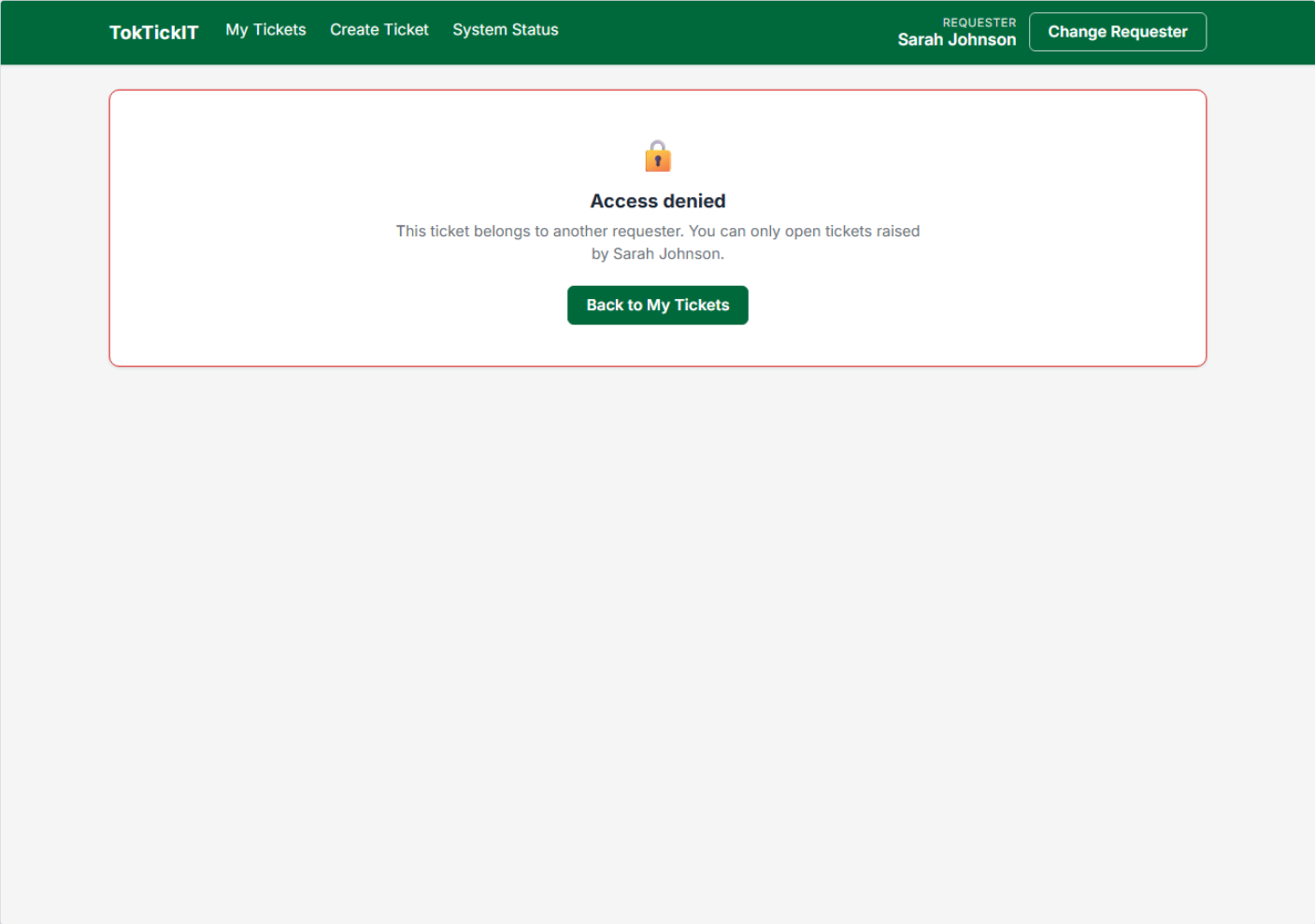
The Download button is driven by the response's `downloadUrl` being `null` rather than by a rule restated on the client, so the screen and the server cannot disagree about what is downloadable.

Covered by **API-10** (200, `isRemoved` true, reason and timestamp stored, no delete issued) and **API-11** (the removed file is refused).

6. Unauthorized access — another requester's ticket and file

CPE334 Lab 2 — TokTickIT — Thawat Boonsuk 67070501024

83 / 103



The same ticket URL opened after switching to Sarah Johnson. The screen is an access-denied panel; none of the ticket's data is on the page, because none of it was returned.

The refusal is the server's, not the screen's. The same request made directly during the capture run:

```
GET /api/tickets/<id> X-Requester-Id: <Sarah>
403 {"error":{"code":"FORBIDDEN","message":"This ticket belongs to another requester."}}
```

GET /api/attachments/:id/download answers 403 the same way for a file on someone else's ticket, and sends no Content-Disposition and no bytes. A removed attachment also answers 403 rather than 404, because the resource genuinely exists and its owner can still see the metadata — what is refused is the state, not the existence.

Covered by **API-05** (Requester B reading Requester A's ticket), **E2E-03** in e2e/ownership.spec.ts (both the screen and the two endpoints) and **UI-05**'s access-denied assertions.

Responsive evidence

Screenshot	Viewport	Note
detail-desktop.png	1280 px	R-08 — four-column field grid, attachment rows on one line
detail-mobile.png	375 px	R-09 — single-column fields, stacked attachment actions
remove-modal-mobile.png	375 px	R-10 — full-width modal buttons

Measured rather than eyeballed on the mobile capture: document.body.scrollHeight is 375 px against a 375 px viewport, so there is no horizontal overflow (V-03).

Answer Part 9:

Zen Green UI and Responsive Evidence · 5 points

Answer Part 9 — Responsive Layout Evidence (Issue #7)

The three main screens at all three breakpoints, plus the selector and the mobile header. Every image was captured against the **running stack with a real database** — Vite on :5173, the Express API on :5000, PostgreSQL 16 on :5433 — by `screenshots/capture-issue7.mjs`, which prints the measured body width beside each capture.

Item	Value
Captured	2026-09-05
Requester	Jennifer Anderson — Registrar (53 tickets)
Breakpoints	Desktop 1280 px, Tablet 820 px, Mobile 375 px
Browser	Chromium, full-page capture
Overflow measurement	<code>document.body.scrollHeight</code> against <code>window.innerWidth</code> at every stop

The ticket summaries beginning `E2E` are rows the end-to-end suite created through the real API. They are genuine tickets, not fixtures: the suite runs against the same database the screenshots were taken from.

1. Create Ticket

Breakpoint	Screenshot	Body width
Desktop 1280 px	<code>create-desktop.png</code>	1280 px, no overflow
Tablet 820 px	<code>create-tablet.png</code>	820 px, no overflow
Mobile 375 px	<code>create-mobile.png</code>	375 px, no overflow

TokTickIT

My Tickets

Create Ticket

System Status

Jennifer Anderson

Change Requester

Create Ticket

Raise a new ticket as the selected requester. The ticket number and status are assigned by the server.

Summary / Title *

Projector in CB2-401 will not detect the laptop

Description *

The projector shows "no signal" with every cable, on two different laptops. It worked last week.

Category *

Hardware

Related System *

Corporate Laptop

Requested Priority *

Low

Medium

High

Attachments

Drag files here, or

Browse files

JPG, PNG, WEBP, PDF · up to 5.0 MB each · at most 5 files

Cancel

Create Ticket

The tablet capture is the one worth looking at closely. From 768 px the form lays its short fields out two to a row — Category beside Related System, Priority beside the space it needs — while Summary, Description and the upload area keep the full width. Those three are the fields whose content benefits from the extra room; a half-width Description would fight the reader for no gain.

Field order follows ui-spec §3.2, every required field carries its red asterisk, and the dropzone states the rules it enforces (JPG, PNG, WEBP, PDF · up to 5.0 MB each · at most 5 files).

At mobile the form returns to a single column and the actions become full-width buttons, with Create Ticket above Cancel so the primary action sits under the thumb.

2. My Tickets

Breakpoint	Screenshot	Body width
Desktop 1280 px	list-desktop.png	1280 px, no overflow
Tablet 820 px	list-tablet.png	820 px, no overflow
Mobile 375 px	list-mobile.png	375 px, no overflow

TokTickIT

My Tickets

Create Ticket

System Status

REQUESTER

Jennifer Anderson

Change Requester

My Tickets

Create Ticket

Tickets raised by Jennifer Anderson.

Search

Ticket number or summary

Category

All categories

Status

All statuses

Sort by

Newest first

Clear Filters

Showing 1–10 of 62 tickets

TICKET NO.	CREATED DATE	SUMMARY	CATEGORY	REQUESTED PRIORITY	IT PRIORITY	CURRENT STATUS	TICKET OWNER	LAST UPDATED	
TKT-2026-000065	05 n.b. 2569	E2E context — only Jennifer should see this	Account and Access	Low	Unassigned	New	Unassigned	05 n.b. 2569	Detail
TKT-2026-000064	05 n.b. 2569	E2E ownership round trip	Account and Access	Medium	Unassigned	New	Unassigned	05 n.b. 2569	Detail
TKT-2026-000063	05 n.b. 2569	E2E ownership of attachments	Account and Access	Medium	Unassigned	New	Unassigned	05 n.b. 2569	Detail
TKT-2026-000062	05 n.b. 2569	E2E ownership guard	Account and Access	Medium	Unassigned	New	Unassigned	05 n.b. 2569	Detail
TKT-2026-000061	05 n.b. 2569	E2E listed after create	Account and Access	Medium	Unassigned	New	Unassigned	05 n.b. 2569	Detail
TKT-2026-000060	05 n.b. 2569	E2E create flow — projector will not detect the laptop	Hardware	High	Unassigned	New	Unassigned	05 n.b. 2569	Detail
TKT-2026-000059	05 n.b. 2569	E2E rejected file type	Account and Access	Medium	Unassigned	New	Unassigned	05 n.b. 2569	Detail
TKT-2026-000058	05 n.b. 2569	E2E removed attachment stays unreachable	Account and Access	Medium	Unassigned	New	Unassigned	05 n.b. 2569	Detail
TKT-2026-000057	05 n.b. 2569	E2E attachment lifecycle	Account and Access	Medium	Unassigned	New	Unassigned	05 n.b. 2569	Detail
TKT-2026-000056	05 n.b. 2569	E2E context — only Jennifer should see this	Account and Access	Low	Unassigned	New	Unassigned	05 n.b. 2569	Detail

Previous

1

2

3

4

5

Next

Page 1 of 7

Per page

10

All ten columns fit the desktop container with the Detail action still on screen. That is what the table's 66 rem minimum width buys: it matches the page container exactly, so the desktop table never scrolls, and at tablet width the same table scrolls inside its own wrapper rather than squeezing its columns until a ticket number runs under the date beside it.

TokTickIT

Menu

My Tickets

Tickets raised by Jennifer Anderson.

Create Ticket

Search

Ticket number or summary

Category

All categories

Status

All statuses

Sort by

Newest first

Clear Filters

My Tickets at mobile width — 1 of 6

Showing 1–10 of 62 tickets

TKT-2026-000065

New

E2E context — only Jennifer should see this

Low

Account and Access

CREATED

05 n.ย. 2569

LAST UPDATED

05 n.ย. 2569

IT PRIORITY

Unassigned

TICKET OWNER

Unassigned

TKT-2026-000064

New

E2E ownership round trip

Medium

Account and Access

CREATED

05 n.ย. 2569

LAST UPDATED

05 n.ย. 2569

IT PRIORITY

Unassigned

TICKET OWNER

Unassigned

TKT-2026-000063

New

E2E ownership of attachments

Low

Account and Access

CREATED

05 n.ย. 2569

LAST UPDATED

05 n.ย. 2569

IT PRIORITY

Unassigned

TICKET OWNER

Unassigned

My Tickets at mobile width — 2 of 6

Medium

Account and Access

CREATED

05 n.ย. 2569

IT PRIORITY

Unassigned

LAST UPDATED

05 n.ย. 2569

TICKET OWNER

Unassigned

TKT-2026-000062

New

E2E ownership guard

Medium

Account and Access

CREATED

05 n.ย. 2569

IT PRIORITY

Unassigned

LAST UPDATED

05 n.ย. 2569

TICKET OWNER

Unassigned

TKT-2026-000061

New

E2E listed after create

Medium

Account and Access

CREATED

05 n.ย. 2569

IT PRIORITY

Unassigned

LAST UPDATED

05 n.ย. 2569

TICKET OWNER

Unassigned

My Tickets at mobile width — 3 of 6

TKT-2026-000060

New

E2E create flow — projector will not detect the laptop

High

Hardware

CREATED

05 n.ย. 2569

LAST UPDATED

05 n.ย. 2569

IT PRIORITY

Unassigned

TICKET OWNER

Unassigned

TKT-2026-000059

New

E2E rejected file type

Medium

Account and Access

CREATED

05 n.ย. 2569

LAST UPDATED

05 n.ย. 2569

IT PRIORITY

Unassigned

TICKET OWNER

Unassigned

TKT-2026-000058

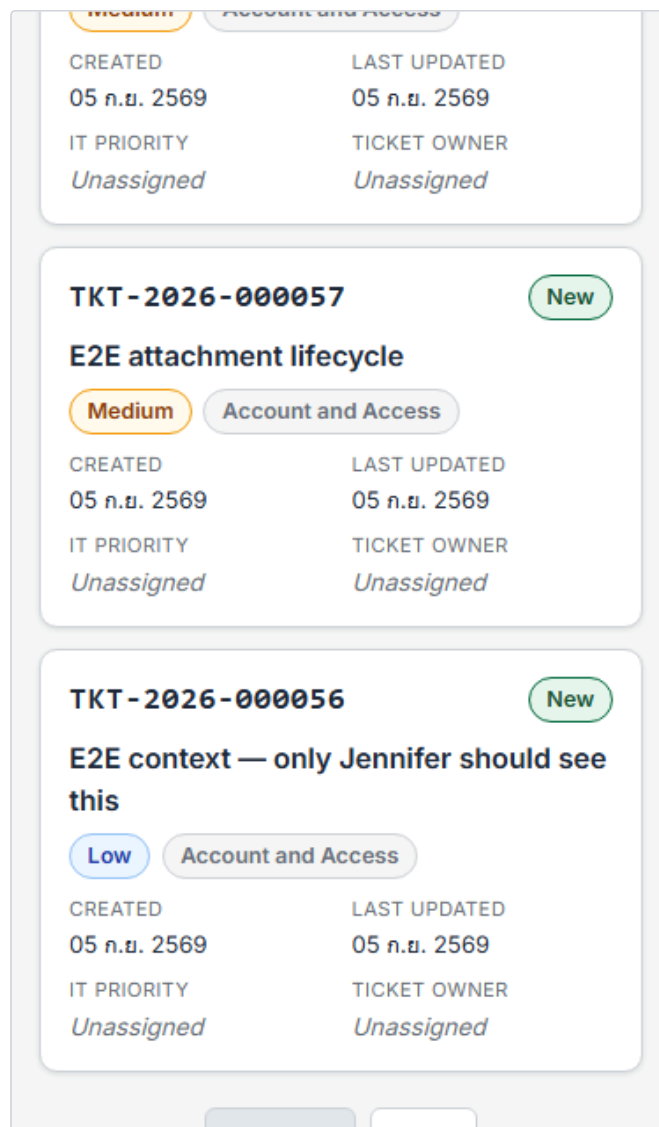
New

E2E removed attachment stays unreachable

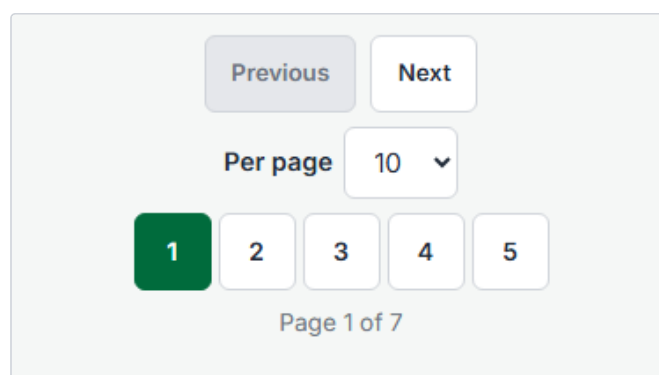
Medium

Account and Access

My Tickets at mobile width — 4 of 6



My Tickets at mobile width — 5 of 6



My Tickets at mobile width — 6 of 6

Below 768 px the table becomes a card list. Each card carries the ticket number, the status badge, the summary, the priority and category chips, and the four date and owner fields as a two-column definition list; the whole card is the tap target. Filters stack full width, and Previous and Next share a row with the numbered pages below them.

3. Ticket Detail

Breakpoint	Screenshot	Body width
Desktop 1280 px	detail-desktop.png	1280 px, no overflow

Breakpoint	Screenshot	Body width
Tablet 820 px	detail-tablet.png	820 px, no overflow
Mobile 375 px	detail-mobile.png	375 px, no overflow

TokTickITMy TicketsCreate TicketSystem StatusJennifer AndersonChange Requester

[My Tickets](#) / TKT-2026-000065

E2E context — only Jennifer should see this

TKT-2026-000065

NewLow

Created 05 n.y. 2569 12:05

Ticket details

This view is read-only. Ticket fields are set when the ticket is created and are changed by the IT team, not here.

TICKET NO.

TKT-2026-000065

CREATED DATE

05 n.y. 2569 12:05

CATEGORY

Account and Access

RELATED SYSTEM

Campus Wi-Fi

REQUESTER

Jennifer Anderson · Registrar

REQUESTED PRIORITY

Low

IT PRIORITY

Unassigned

CURRENT STATUS

New

TICKET OWNER

Unassigned

LAST UPDATED

05 n.y. 2569 12:05

SUMMARY

E2E context — only Jennifer should see this

DESCRIPTION

Created by the requester-context end-to-end test.

Attachments (0 of 5 active)

No files are attached to this ticket yet.

Add a file

JPG, PNG, WEBP, PDF up to 5.0 MB. A ticket can hold 5 active files; removing one frees a slot.

Choose Files

No file chosen

The field grid is `auto-fit` with a 210 px minimum, so it finds four columns at desktop, three at tablet and one at mobile without a breakpoint of its own. Summary and Description always span the full row.

Every value is plain text on the card surface with a muted uppercase label above it — no input chrome anywhere, because there is no input anywhere. That is AC-05, and it is asserted rather than assumed: UI-05 fails if the ticket-fields region contains a single `input`, `textarea`, `select` or button.

CPE334 Lab 2 — TokTickIT — Thawat Boonsuk 67070501024

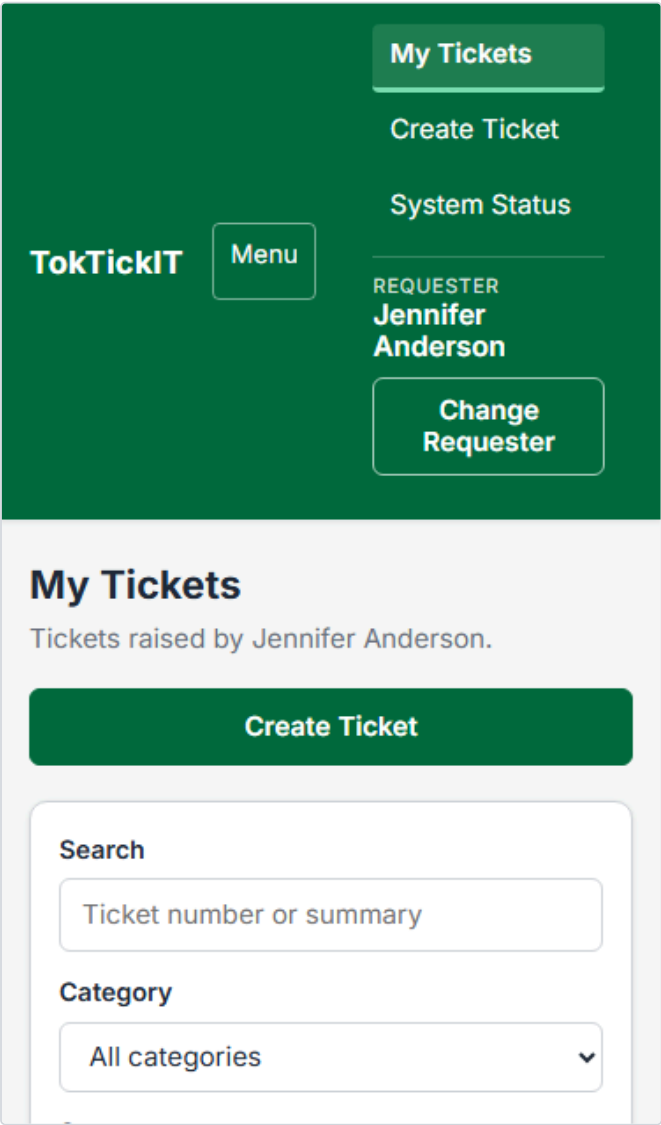
93

/

103

4. Selector and mobile header

Item	Screenshot
Requester Selection, desktop (R-01)	selector-desktop.png
Requester Selection, mobile (R-02)	selector-mobile.png
Header with the compact menu open, mobile (R-11)	header-mobile.png



Mobile header with the menu open — 1 of 6

Status

All statuses

Sort by

Newest first

Clear Filters

Showing 1–10 of 62 tickets

TKT-2026-000065

New

E2E context — only Jennifer should see this

Low

Account and Access

CREATED

LAST UPDATED

05 n.d. 2569

05 n.d. 2569

IT PRIORITY

TICKET OWNER

Unassigned

Unassigned

TKT-2026-000064

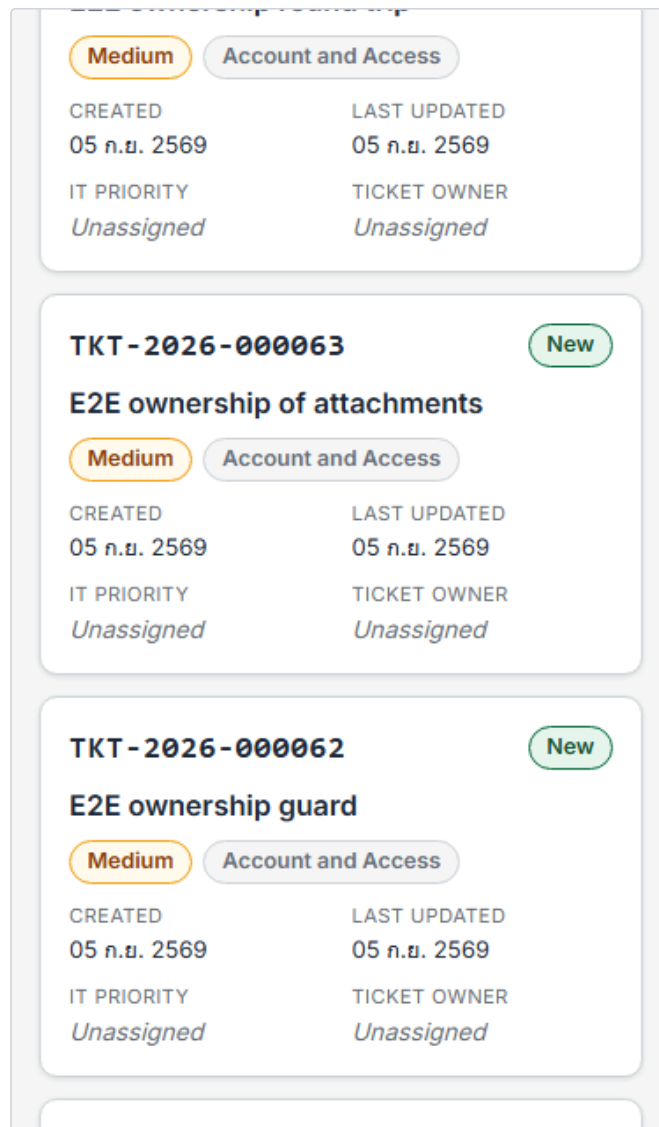
New

E2E ownership round trip

Medium

Account and Access

Mobile header with the menu open — 2 of 6



Mobile header with the menu open — 3 of 6

TKT-2026-000061

New

E2E listed after create

Medium

Account and Access

CREATED

05 n.ย. 2569

LAST UPDATED

05 n.ย. 2569

IT PRIORITY

Unassigned

TICKET OWNER

Unassigned

TKT-2026-000060

New

E2E create flow — projector will not detect the laptop

High

Hardware

CREATED

05 n.ย. 2569

LAST UPDATED

05 n.ย. 2569

IT PRIORITY

Unassigned

TICKET OWNER

Unassigned

TKT-2026-000059

New

E2E rejected file type

Medium

Account and Access

CREATED

05 n.ย. 2569

LAST UPDATED

05 n.ย. 2569

Mobile header with the menu open — 4 of 6

CREATED
05 ก.ย. 2569

LAST UPDATED
05 ก.ย. 2569

IT PRIORITY
Unassigned

TICKET OWNER
Unassigned

TKT-2026-000058

New

E2E removed attachment stays unreachable

Medium

Account and Access

CREATED
05 ก.ย. 2569

LAST UPDATED
05 ก.ย. 2569

IT PRIORITY
Unassigned

TICKET OWNER
Unassigned

TKT-2026-000057

New

E2E attachment lifecycle

Medium

Account and Access

CREATED
05 ก.ย. 2569

LAST UPDATED
05 ก.ย. 2569

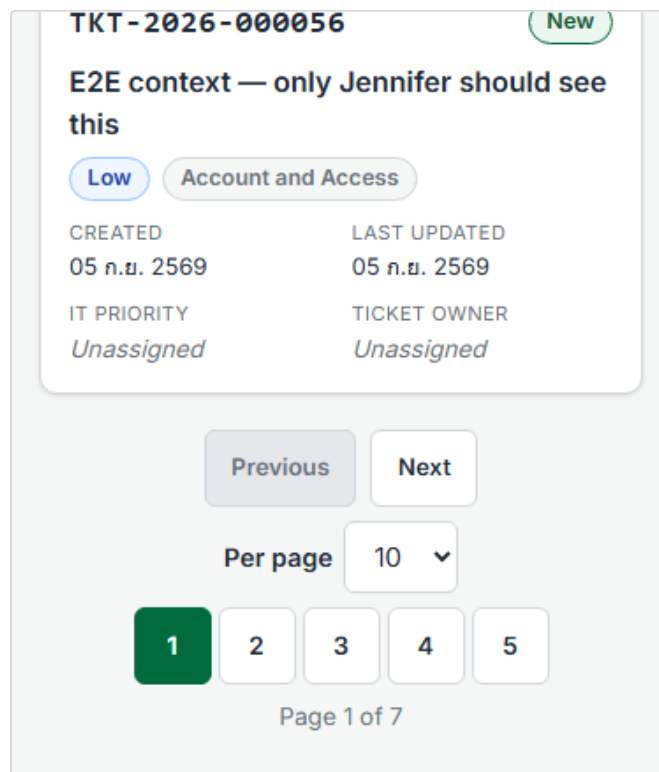
IT PRIORITY
Unassigned

TICKET OWNER
Unassigned

TKT-2026-000056

New

Mobile header with the menu open — 5 of 6



Mobile header with the menu open — 6 of 6

Below 768 px the navigation, the requester name and Change Requester collapse behind a Menu toggle, so the header stays one line high and nothing overflows. Change Requester is reachable from every screen, which is what FR-06 requires.

5. What the responsive pass measured

Nine screen-and-width combinations, checked by reading the page rather than looking at it:

Check	Result
Horizontal overflow (V-03)	<code>body.scrollWidth</code> equals the viewport width in all nine captures
Clipped text (V-04)	Every leaf element's <code>scrollWidth</code> compared against its <code>clientWidth</code> ; the only hit is the deliberately <code>visually-hidden</code> file input
Labelling (V-06)	No <code>input</code> , <code>select</code> or <code>textarea</code> without a label, wrapping label or <code>aria-label</code>
Touch targets	Every control at 375 px is at least 44 px tall, up from 28–38 px before this pass
Contrast (V-07)	Lowest ratio 4.83:1 on muted dates, above the 4.5:1 AA floor; badges 6.84:1 to 8.21:1
Focus ring (V-05)	<code>2px solid rgb(11, 122, 70)</code> , which is <code>--zg-secondary</code>

The full checklist, including what this pass changed rather than confirmed, is in [ui-spec.md](#) §5.

Lab 2 — UI Specification (Zen Green Theme)

Companion to [specification.md](#). This document defines the visual language and screen-level behaviour of the Requester Ticketing MVP; the data behind each screen is specified in [api-spec.md](#).

1. Color Tokens

Token	Value	Used for
<code>--zg-primary</code>	<code>#006B3C</code>	Application header background, primary button background
<code>--zg-secondary</code>	<code>#0B7A46</code>	Active tab underline, focus rings, links, primary button hover
<code>--zg-pale</code>	<code>#EAF6EF</code>	Selected table rows, success callouts, pale badges
<code>--zg-bg</code>	<code>#F5F7F6</code>	Page background
<code>--zg-surface</code>	<code>#FFFFFF</code>	Cards, tables, modals
<code>--zg-border</code>	<code>#D1D5DB</code>	Card and input borders
<code>--zg-shadow</code>	<code>0 1px 3px rgba(0,0,0,0.1)</code>	Card and modal elevation
<code>--zg-text</code>	<code>#1F2937</code>	Body text
<code>--zg-text-muted</code>	<code>#6B7280</code>	Helper text, timestamps, read-only values
<code>--zg-error</code>	<code>#DC2626</code>	Validation errors, destructive confirmations
<code>--zg-warning</code>	<code>#F59E0B</code>	Warnings, e.g. the "not a real login" notice

Editable vs. read-only controls

State	Appearance
Editable input	White surface, 1px <code>--zg-border</code> , focus ring in <code>--zg-secondary</code>
Read-only value	No input chrome; plain text on the card surface, label in <code>--zg-text-muted</code>
Disabled control	Muted background, muted text, no focus ring, <code>cursor: not-allowed</code>

2. Component Rules

- Labels sit **above** their input, never inline or as placeholder-only.
- Required fields carry a red asterisk (*) directly after the label text.
- Validation errors appear **directly under the field they belong to**, in `--zg-error`, and the field border turns error-colored.
- Form-level errors (e.g. a failed submit) appear as a callout above the form actions.
- Buttons show a busy/submitting state: label swaps to a progress label, the button is disabled, and a spinner is shown.
- Primary action per screen is a single filled `--zg-primary` button; secondary actions are outlined or text buttons.
- Every interactive element has a visible focus ring in `--zg-secondary`.
- Badges: status and priority render as pill badges with sufficient contrast against `--zg-surface`.

Badge	Value	Style
Status	New	Pill, <code>--zg-pale</code> fill, <code>--zg-primary</code> border, <code>#14532D</code> text
Priority	Low	Pill, <code>#EFF6FF</code> fill, <code>#93C5FD</code> border, <code>#1E40AF</code> text
Priority	Medium	Pill, <code>--zg-warning-bg</code> fill, <code>--zg-warning</code> border, <code>#92400E</code> text
Priority	High	Pill, <code>#FEF2F2</code> fill, <code>--zg-error</code> border, <code>#991B1B</code> text

Priority is never encoded by colour alone — the level is written in the badge — so the ordering survives a monochrome print and a red/green colour deficiency.

3. Screens

3.0 Application Shell (header)

Element	Specification
Background	<code>--zg-primary</code> , full width, fixed at the top
App name	Left-aligned, on-primary text color
Current requester	Shows the name of the selected Development Requester
Change Requester	Button next to the requester name, available on every screen; returns to the selector and, once a new requester is chosen, clears and re-fetches all requester-scoped data (FR-06, BR-13)
Navigation	Links to My Tickets and Create Ticket; active item underlined in <code>--zg-secondary</code>
Mobile	Requester name and Change Requester collapse into a compact menu; no horizontal overflow

3.1 Development Requester Selection

Element	Specification
Heading	Names the screen and states its purpose
Warning callout	Amber; states clearly that this is not a real login and exists only for development/testing
Dropdown	Lists active requester users only; shows name (and department/email if available)
Continue button	Primary; disabled until a requester is chosen; navigates to My Tickets
Layout	Single centered card on <code>--zg-bg</code>

3.2 Create Ticket

Element	Specification
Field order	Summary → Description → Category → Related System → Priority → Attachments
Required fields	Summary/Title, Description, Category, Related System and Requested Priority — each carries a red <code>*</code> after its label. Attachments are optional.
Dropdowns	Category and Related System populated from the API; placeholder option shown until chosen
Upload area	Dropzone with an explicit "Browse files" button; helper text lists allowed types (JPG, PNG, WEBP, PDF), size limit (5 MB) and file cap (5)
File preview list	One row per staged file: name, size, type icon, remove control
Actions	Primary "Create Ticket" with busy state; secondary "Cancel" returns to My Tickets
Success	Confirmation showing the generated Ticket Number, with a link to the ticket detail
Backend failure	Error callout above the form actions; all entered values stay in the form and the submit button returns to its idle state so the user can retry (FR-07, AC-12)

3.3 My Tickets

Element	Specification
Desktop (≥ 768 px)	Table: Ticket No., Created Date, Summary, Category, Requested Priority, IT Priority, Current Status, Ticket Owner, Last Updated, and a Detail action. Fixed column layout so all ten columns fit the container; the table scrolls inside its own wrapper below 66 rem, the width at which every fixed column still holds its content
IT Priority / Ticket Owner	Rendered as an italic muted <i>Unassigned</i> placeholder. Both are IT-triage fields with no column in the Lab 2 schema; an explicit placeholder keeps "not yet assigned" from reading as missing data

Element	Specification
Mobile (< 768 px)	Card list; each card shows Ticket No., Status badge, Summary, Priority and Category, and the four dates/owner fields as a two-column definition list. The whole card is the tap target
Search bar	Free-text, case-insensitive, matched against <code>summary</code> , <code>description</code> and <code>ticketNumber</code> — maps to the <code>search</code> query parameter. Debounced 300 ms so typing does not fire a request per keystroke
Filters	Category and Status — map to the <code>category</code> and <code>status</code> query parameters. A "Clear Filters" button resets search, category and status together, and is disabled when none is active
Sort	<code>sort</code> parameter; Created At descending by default, plus Created At ascending, Ticket No. either way, and Priority either way (High-first for descending)
Pagination	Previous / Next, a window of at most five numbered page buttons, "Page <i>n</i> of <i>m</i> ", and a page-size selector of 10 / 20 / 50; <code>page</code> / <code>pageSize</code> parameters, default page size 10. Any change to search, filter, sort or page size returns to page 1
Empty state	Requester has no tickets at all: message plus a call to action to create the first ticket
No-results state	Search/filter matched nothing: message offering to clear the filters — visually distinct from the empty state (AC-15)
Selected row	Background <code>--zg-pale</code>

3.4 Requester Ticket Detail (read-only)

Element	Specification
Header	Ticket Number, Summary, Status badge, Priority badge, Created At
Fields	All ticket fields rendered read-only — no inputs, no edit affordance
Attachment list — active	File name, size, uploaded at, Download action
Attachment list — removed	Rendered muted with a "Removed" marker, removal reason and removal timestamp; no Download action
Remove action	Opens a modal requiring a removal reason; confirm button is disabled until a reason is entered
Removal modal	Title, reason textarea (required), Cancel and Confirm; Confirm shows a busy state
Access denied	If the ticket does not belong to the selected requester, the screen shows an access-denied message instead of ticket data

4. Responsive Breakpoints

Breakpoint	Width	Behaviour
Desktop	≥ 992 px	Table layouts, multi-column forms, side-by-side filters
Tablet	768 – 991 px	Condensed table or two-column form; filters wrap
Mobile	< 768 px	Single-column forms, card lists instead of tables, full-width buttons, stacked filters

Rules that hold at every width: no horizontal page overflow; wide content (tables) scrolls inside its own container.

Tap targets — resolved (Issue #7): 44 px minimum below 768 px. WCAG 2.2 AA (2.5.8) sets the floor at 24 px, which every control already cleared, but 44 px is the size a thumb hits reliably and it is what the mobile layout now enforces on buttons, page numbers and the header toggle. Above 768 px the denser 28–38 px controls stay: a pointer does not need the slack, and inflating the table’s Detail action would cost a row of vertical space on every screen to solve a problem that only exists on touch.

5. Visual Inspection Checklist

Checked on 2026-09-05 against the running stack at 1280 px, 820 px and 375 px. Where a row could be measured rather than eyeballed it was: the evidence column names what was actually read off the page.

#	Item	Desktop	Tablet	Mobile	Evidence
V-01	Font family, sizes and weights match the theme	☑	☑	☑	One <code>--zg-font</code> stack on <code>body</code> ; no component overrides the family
V-02	Consistent padding and spacing inside cards and tables	☑	☑	☑	Card and cell padding come from <code>.zg-card</code> / <code>.zg-table td</code> , not from per-screen rules
V-03	No horizontal overflow on the page body	☑	☑	☑	<code>document.body.scrollWidth</code> equals the viewport width on all three screens at all three widths
V-04	No clipped or truncated text, badges or buttons	☑	☑	☑	Every leaf element compared <code>scrollWidth</code> against <code>clientWidth</code> ; the only hit is the intentionally <code>visually-hidden</code> file input
V-05	Focus ring visible on every interactive element	☑	☑	☑	<code>.zg-app :focus-visible</code> computes to <code>2px solid rgb(11, 122, 70)</code> = <code>--zg-secondary</code>
V-06	Every input has an accessible label	☑	☑	☑	Every <code>input</code> / <code>select</code> / <code>textarea</code> has a <code>label[for]</code> , a wrapping label, or <code>aria-label(ledby)</code> ; zero unlabelled
V-07	Color contrast sufficient for text and badges	☑	☑	☑	Lowest measured ratio 4.83:1 (muted dates and the <i>Unassigned</i> placeholder), above the 4.5:1 AA floor; badges 6.84 – 8.21:1
V-08	Error messages appear under the correct field	☑	☑	☑	Each <code>.zg-field-error</code> sits below its own control's bounding box, in <code>#DC2626</code> , with the control's border matching
V-09	Busy/submitting states visible on all submit buttons	☑	☑	☑	Create Ticket, Confirm removal and Download each swap label, disable, and show a spinner (UI-08, UI-06)
V-10	Removed attachments visually distinct from active ones	☑	☑	☑	Muted surface, dashed border, struck-through name, <code>Removed</code> badge, no Download control

Two things this pass changed rather than confirmed:

- **Tablet.** The Create Ticket form now lays its short fields out two to a row from 768 px, with Summary, Description and the upload area still spanning the full width. The header keeps its nav and requester block on one line at that width by dropping the small "Requester" caption, whose meaning the name below it already carries.
- **Table minimum width.** `.zg-table` is pinned to 66 rem, the width of the page container. Wider makes the desktop table scroll and hides the Detail action; narrower squeezes the percentage columns until a ticket number runs under the date beside it. Below 992 px the wrapper scrolls within that width, which is the documented tablet behaviour.

Screenshots recording this checklist are collected in [answer-part9-screenshots.md](#); the run that produced them is recorded in [tests.md](#).